UNIVERSIDADE FEDERAL DO PAMPA

FRANCESCO PAGANI GALVÃO

VERIFICAÇÃO FORMAL DE INVARIANTES DE SESSÃO EM DATA GRIDS: REPLICAÇÃO ASSÍNCRONA, FAILOVER E CONTROLE ORIENTADO A SLO

Bagé
2026

FRANCESCO PAGANI GALVÃO

VERIFICAÇÃO FORMAL DE INVARIANTES DE SESSÃO EM DATA GRIDS: REPLICAÇÃO ASSÍNCRONA, FAILOVER E CONTROLE ORIENTADO A SLO

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Engenharia de Computação como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação.

Orientador: Leonardo Bidese de Pinho

Bagé
2026

— Galvão, Francesco Pagani

Verificação formal de invariantes de sessão em data grids: replicação assíncrona, failover e controle orientado a SLO / Francesco Pagani Galvão. – Outubro, 2026.

103 f.: il.

Trabalho de Conclusão de Curso (Graduação) – Universidade Federal do Pampa, Campus Bagé, Engenharia de Computação, 2026.

"Orientação: Leonardo Bidese de Pinho".

1. Data grid. 2. Replicação assíncrona. 3. Failover. 4. Model checking. 5. Invariantes de sessão. 6. Objetivos de nível de serviço. I. Título.

FRANCESCO PAGANI GALVÃO

VERIFICAÇÃO FORMAL DE INVARIANTES DE SESSÃO EM DATA GRIDS: REPLICAÇÃO ASSÍNCRONA, FAILOVER E CONTROLE ORIENTADO A SLO

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Engenharia de Computação como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação.

Trabalho de Conclusão de Curso defendido e aprovado em: _____ de _____ de _____.

Banca examinadora:

_____

Prof. Dr. Leonardo Bidese de Pinho
Orientador

_____

Prof. Dr. Bruno Silveira Neves
UNIPAMPA

_____

Prof. Dr. Diego Leonel Cadette Dutra
UFRJ

RESUMO

Aplicações distribuídas costumam delegar o estado de sessão de seus usuários a um *data grid* replicado, que funciona como camada de sessão compartilhada entre vários nós. Como a replicação é assíncrona para sustentar o desempenho dessa camada, uma atualização aplicada em um nó leva algum tempo até alcançar todas as réplicas, e nesse intervalo as réplicas permanecem temporariamente divergentes. Se um nó cai antes que a última atualização chegue às demais réplicas, o *failover* pode promover uma réplica desatualizada e reverter efeitos que já deveriam ser definitivos, fazendo um *logout* voltar a valer, um contador de tentativas regredir ou um bloqueio desaparecer. Essas falhas comprometem segurança e cumprimento de SLO e escapam ao teste convencional porque dependem de entrelaçamentos específicos entre operações concorrentes e eventos de falha. Este trabalho formaliza tais propriedades como invariantes de sessão, entre elas a irreversibilidade do *logout*, a persistência do bloqueio, a não regressão do contador de tentativas e a convergência do estado, e investiga sob que condições essas garantias resistem ou se rompem diante de replicação assíncrona e *failover*. A proposta especifica formalmente o protocolo e o estado de sessão, aplica verificação de modelos para identificar as condições sob as quais cada invariante falha e, a partir dessas condições, projeta um algoritmo de decisão que atua em tempo de execução sobre as políticas e os parâmetros de configuração do *cluster*, ajustando-os conforme as falhas observadas para reduzir violações e controlar a cauda de latência. Esse controle adaptativo da configuração em *runtime* é o eixo da proposta, pela qual o trabalho intervém sobre o compromisso entre correção do estado e latência. A avaliação apoia-se em verificação de modelos e em experimentação sobre um *cluster* real submetido a falhas induzidas, confrontando o comportamento observado com o que a especificação indica. O trabalho pretende oferecer garantias de correção do estado de sessão sob falhas em *data grids* replicados sem sacrificar os SLO de latência acordados.

Palavras-chave: Data grid. replicação assíncrona. failover. model checking. invariantes de sessão. objetivos de nível de serviço.

ABSTRACT

Distributed applications often delegate their users' session state to a replicated *data grid* that acts as a shared session layer across several nodes. Because replication runs asynchronously to sustain the performance of that layer, an update applied on one node takes some time to reach every replica, and during this interval the replicas remain temporarily divergent. If a node fails before the latest update reaches the other replicas, *failover* may promote a stale replica and revert effects that should already be final, making a *logout* valid again, a login attempt counter regress or a lockout disappear. Such faults compromise security and SLO compliance and escape conventional testing because they depend on specific interleavings between concurrent operations and failure events. This work formalizes these properties as session invariants, among them the irreversibility of *logout*, the persistence of lockout, the non-regression of the attempt counter and state convergence, and investigates under which conditions these guarantees hold or break under asynchronous replication and *failover*. The proposal formally specifies the session protocol and state, applies model checking to identify the conditions under which each invariant fails and, from these conditions, designs a decision algorithm that acts at run time over the policies and configuration parameters of the *cluster*, tuning them according to the observed faults to reduce violations and control the latency tail. This adaptive control of the cluster configuration at runtime is the core of the proposal, through which the work intervenes on the trade-off between state correctness and latency. The evaluation relies on model checking and on experimentation over a real *cluster* subjected to induced faults, confronting the observed behavior with what the specification indicates. The work aims to offer guarantees of session state correctness under faults in replicated *data grids* without sacrificing the agreed latency SLO.

Keywords: Data grid, asynchronous replication, failover, model checking, session invariants, service level objectives.

LISTA DE FIGURAS

Figura 1 Topologia cliente-servidor de um *cluster* Infinispan, em que as aplicações acessam os servidores por protocolo remoto. 22
Figura 2 Replicação de uma escrita entre os nós de um *data grid*, com atualização dos índices por nó e leitura por consulta. 23
Figura 3 Exploração do espaço de estados pelo verificador TLC, em que a relação de transição gera os estados sucessores a partir do estado inicial 26
Figura 4 Ciclo de vida de uma sessão autenticada com estado externalizado em repositório de sessões compartilhado 31

Figura 5 Modelo de fluxograma PRISMA 2020 para o relato do processo de seleção de estudos. 39

LISTA DE TABELAS

Tabela 1 Questões de mapeamento e eixos temáticos correspondentes 34
Tabela 2 Síntese do fluxo PRISMA 2020 do mapeamento sistemático até a etapa atual 38
Tabela 3 Parâmetros do *baseline* experimental: faixa observada, fonte e valor adotado 46

Tabela 4 Sinais observados pelo controlador heurístico 75

Tabela 5 Cronograma do Trabalho de Conclusão de Curso I 87
Tabela 6 Cronograma do Trabalho de Conclusão de Curso II 89

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ACM	*Association for Computing Machinery*
CAP	*Consistency, Availability, Partition tolerance*
CPU	*Central Processing Unit*
CRDT	*Conflict-free Replicated Data Type*
FLP	Fischer, Lynch e Paterson
IEEE	*Institute of Electrical and Electronics Engineers*
IMDG	*In-Memory Data Grid*
JVM	*Java Virtual Machine*
LWW	*Last Writer Wins*
MAPE-K	*Monitor, Analyze, Plan, Execute over Knowledge*
NIST	*National Institute of Standards and Technology*
OWASP	*Open Worldwide Application Security Project*
PRISMA	*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*
RTT	*Round-Trip Time*
SEC	*Strong Eventual Consistency*
SLA	*Service Level Agreement*
SLI	*Service Level Indicator*
SLO	*Service Level Objective*
SMR	*State Machine Replication*
TCC	Trabalho de Conclusão de Curso
TLA	*Temporal Logic of Actions*
TLC	verificador de modelos de TLA+
UNIPAMPA	Universidade Federal do Pampa

SUMÁRIO

1 INTRODUÇÃO 10
1.1 Problema de pesquisa 11
1.2 Justificativa 12
1.3 Objetivos 14
1.4 Organização do texto 15
2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Sistemas distribuídos: consistência, replicação e falhas 16
2.2 In-Memory Data Grids e a plataforma Infinispan 20
2.3 Especificação formal e *model checking* 24
2.4 Latência de cauda, objetivos de nível de serviço e observabilidade 27
2.5 Sistemas autoadaptativos e controle por realimentação 29
2.6 Gerência de sessão como domínio 30
3 METODOLOGIA 33
3.1 Mapeamento sistemático da literatura 33
3.2 Especificação formal e validação experimental 40
4 DESENVOLVIMENTO E RESULTADOS PARCIAIS 60
4.1 Arquitetura experimental 60
4.2 Especificação formal inicial 61
4.3 Execução do *model checking* e estado do *baseline* experimental 64
4.4 Estado dos artefatos 70
4.5 Resultados parciais e limitações conhecidas 77
5 CONSIDERAÇÕES PARCIAIS 83
5.1 Reflexões sobre as atividades do Trabalho de Conclusão de Curso I 86
5.2 Próximas etapas e plano para o Trabalho de Conclusão de Curso II 88
REFERÊNCIAS 91
APÊNDICE A — PLANO DE TRABALHO PARA O TCC II 99

1 INTRODUÇÃO

Aplicações que servem requisições autenticadas mantêm, para cada identidade, um conjunto de estados de curta duração. Esses estados incluem sessões abertas, contadores de tentativas falhas, metadados de renovação e marcas de bloqueio, e existem para que cada nova requisição possa ser validada como continuação legítima de uma autenticação anterior (TEMOSHOK *et al.*, 2025; OWASP Foundation, 2026). Em arquiteturas com várias instâncias atrás de um balanceador de carga, esse estado não pode ficar restrito ao processo que o criou, sob pena de afinidade rígida do cliente ao processo ou de perda de sessão em qualquer reinício. A resposta corrente é externalizar o estado para um armazenamento compartilhado com acesso de baixa latência. Os *In-Memory Data Grids* (IMDG) ocupam esse papel: oferecem armazenamento chave-valor em memória, particionamento por chave, replicação configurável e mecanismos de realocação quando a composição do *cluster* muda (Infinispan Project, 2026f; Infinispan Project, 2026c).

A replicação dentro de um IMDG admite dois desenhos. Na replicação síncrona, a confirmação de uma escrita depende da confirmação remota das demais cópias, o que atrela cada operação ao *Round-Trip Time* (RTT) e à variabilidade da rede. Na replicação assíncrona, ou otimista, a escrita é confirmada localmente e propagada em segundo plano, com ganho de latência percebida e o custo de admitir réplicas que observam o mesmo objeto em estados diferentes. Trabalhos seminais demonstram que essa segunda família é uma escolha consciente de armazenamentos chave-valor de produção, com mecanismos como *vector clocks*, *sloppy quorum* e *hinted handoff* para tolerar falhas sem coordenação síncrona (DECANDIA *et al.*, 2007). Para tratar a reconciliação entre réplicas divergentes, uma alternativa consolidada são os tipos de dados replicados livres de conflito, que garantem convergência determinística do estado quando as operações satisfazem propriedades algébricas adequadas (SHAPIRO *et al.*, 2011).

Além do funcionamento sob condições normais, este trabalho considera explicitamente três classes de evento que rompem a estabilidade do *cluster*. A primeira é o *failover*: um nó deixa de responder e o atendimento passa a uma réplica, possivelmente com dados defasados. A segunda é a partição de rede, em que sub-conjuntos de nós deixam de trocar mensagens por um período. A terceira reúne o rebalanceamento e o *state transfer*, eventos planejados ou reativos que alteram o mapeamento entre chaves e nós e podem coexistir com falhas (Infinispan Project, 2026c). Cada uma dessas três classes pode afetar tanto propriedades funcionais, relativas ao estado final do sistema, quanto

propriedades temporais, relativas à ordem dos eventos observados pelos clientes. Quando o trabalho exige replicação entre localidades distintas, são considerados os mecanismos documentados para replicação *cross-site* (Infinispan Project, 2026b).

O problema também tem leitura no eixo da computação sustentável. Consumo de energia e resiliência estão entre os desafios centrais da computação de alto desempenho (NAVAUX; LORENZON; SERPA, 2023), e falhas cobram recursos em reconstruções de estado, repetição de operações e reprocessamento. O consumo energético desses ambientes está diretamente atrelado à execução de tarefas computacionais. A literatura analisa estratégias de economia de energia em sistemas tolerantes a falhas (MORÁN *et al.*, 2024).

Este trabalho aborda essa preocupação por dois caminhos. O primeiro é a redução de retrabalho, ou seja, ao reduzir violações de invariantes e estabilizar o comportamento sob falhas, o sistema evita reexecuções e reprocessamento, e com eles o gasto computacional associado. O segundo é a camada de armazenamento, o estado de sessão é mantido em memória em um *data grid*, e decisões de *cache* têm impacto de carbono mensurável e podem ser tratadas de forma consciente de carbono (TIAN *et al.*, 2026). O controlador heurístico previsto, que ajusta políticas e parâmetros de configuração em tempo de execução, se alinha a esforços recentes que investigam o autoajuste de parâmetros sob a perspectiva da computação sustentável, com o objetivo de reduzir o custo computacional do processamento (LEMOS *et al.*, 2025). O objetivo de reduzir retrabalho e estabilizar a cauda de latência é coerente com essa preocupação.

1.1 Problema de pesquisa

O problema é formulado em termos de invariantes de domínio do estado de sessão. Um invariante é uma propriedade que deve permanecer verdadeira em todas as execuções permitidas pelo modelo, apesar de concorrência e falhas. Três exemplos guiam o trabalho. A irreversibilidade da invalidação afirma que, depois de um *logout* confirmado, a sessão correspondente não deve retornar ao estado "válida". A monotonicidade de contadores afirma que tentativas falhas registradas não devem diminuir como resultado de reconciliações entre réplicas. A consistência de bloqueio afirma que uma identidade marcada como bloqueada não deve ter sessões validadas como ativas enquanto o bloqueio durar. As três propriedades têm respaldo normativo, com tratamento explícito em Temoshok *et al.* (2025) e em OWASP Foundation (2026), e podem ser violadas quando replicação assíncrona e *failover* interagem. As três propriedades derivam de exigências de Temoshok

et al. (2025), como a invalidação dos segredos de sessão no *logout* e a limitação de tentativas falhas, e de OWASP Foundation (2026), que exige a invalidação da sessão no servidor no *logout*. A forma replicada dessas propriedades é derivação deste trabalho.

Diante disso, este trabalho aborda como questão central de pesquisa especificar e verificar formalmente invariantes do estado de sessão em *data grids* sob replicação assíncrona e falhas e, a partir dos contraexemplos produzidos pela verificação e das métricas observadas em experimentos controlados, definir um controlador heurístico que reduza a taxa de violações e estabilize a cauda da distribuição de latência, representada pelos percentis p95 e p99, sem aumentar indevidamente o custo operacional.

A abordagem proposta combina quatro elementos que se encadeiam. O trabalho começa pela especificação formal do núcleo do protocolo de estado de sessão em um sistema de transições com canal assíncrono. Sobre essa especificação, verifica os invariantes declarados por *model checking*, técnica adequada para entrelaçamentos de eventos cuja reprodução por testes é pouco confiável (NEWCOMBE *et al.*, 2015). A partir dos contraexemplos obtidos, desenha e implementa um mecanismo de decisão heurístico que observa métricas do sistema e escolhe políticas de mitigação. Por fim, valida o conjunto em um *cluster* Infinispan sob cenários de falha reproduzíveis, com instrumentação que mede a latência, em particular os percentis p95 e p99 (DEAN; BARROSO, 2013; Google, 2016), e que detecta violações de invariantes por verificações no nível da aplicação.

1.2 Justificativa

Externalizar o estado de sessão para um *data grid* replicado faz com que a interação entre concorrência, replicação e falhas se torne um fator determinante da correção observada pelo usuário. DeCandia *et al.* (2007) consolidaram, em produção, o uso de replicação assíncrona em armazenamentos chave-valor de larga escala. No Dynamo, uma escrita pode ser confirmada ao cliente antes de chegar a todas as réplicas, bastando a resposta de parte delas, e essa decisão tem como custo réplicas que observam estados defasados e exigem reconciliação posterior. Quando o dado replicado é o estado de sessão, essa defasagem pode reverter decisões de segurança, como uma invalidação. Shapiro *et al.* (2011) mostram que a convergência sem coordenação é possível quando as operações satisfazem propriedades algébricas como comutatividade ou formação de um semirreticulado, o que torna pertinente investigar quais invariantes do estado de sessão admitem garantias análogas e quais exigem mecanismos adicionais.

A pertinência do estudo das falhas que cercam esses invariantes é sustentada por resultados teóricos clássicos e por evidência empírica de produção. No plano teórico, Fischer, Lynch e Paterson (1985) demonstram que, em um sistema assíncrono no qual processos podem parar silenciosamente, nenhum protocolo determinístico de consenso é totalmente correto, e a prova constrói uma execução que evita a decisão. Na mesma linha, Gilbert e Lynch (2002) formalizam o teorema CAP e mostram que, em presença de partição de rede, um sistema distribuído não pode oferecer simultaneamente consistência forte e disponibilidade. Chandra e Toueg (1996) contornam parcialmente a barreira ao introduzir detectores de falha caracterizados por duas propriedades, completude e acurácia, com a classe $\diamond W$ identificada como suficiente para resolver consenso. No plano empírico, Bailis e Kingsbury (2014) documentam partições, atrasos e perdas de mensagens como eventos recorrentes em infraestruturas de produção, o que reforça que esses cenários devem ser parte do experimento e não exceção dele.

A opção por controlar percentis altos de latência, em vez da média, decorre do impacto desproporcional da cauda da distribuição em serviços compostos por muitos componentes. Um exemplo do livro de engenharia de confiabilidade do Google mostra o problema: a latência típica é da ordem de 50 milissegundos, mas 5% das requisições são vinte vezes mais lentas, e uma média não acusa a variação na cauda (Google, 2016). Dean e Barroso (2013) mostram que o efeito cresce com o número de componentes: se cada servidor responde tipicamente em 10 milissegundos, mas tem p99 de 1 segundo, uma requisição que espera 100 desses servidores passa de 1 segundo em 63% dos casos. A formulação dessas metas como objetivos de nível de serviço, com indicadores correspondentes e *error budgets* associados, é prática consolidada de engenharia de confiabilidade (Google, 2016). O uso de *model checking*, por sua vez, justifica-se pela capacidade de explorar sistematicamente entrelaçamentos de eventos e produzir contraexemplos concretos para violações, classe de defeitos difícil de reproduzir apenas por testes (NEWCOMBE *et al.*, 2015). Há registros de aplicação industrial dessa técnica a protocolos de replicação em sistemas distribuídos de larga escala, com defeitos sutis identificados antes do código entrar em produção (NEWCOMBE, 2014; NEWCOMBE *et al.*, 2015).

O domínio de aplicação reforça a importância dos invariantes escolhidos. A gerência de sessão em arquiteturas distribuídas em nuvem é cercada de orientações normativas que descrevem propriedades operacionais do ciclo de vida da sessão, com tratamento detalhado em Temoshok *et al.* (2025) e em OWASP Foundation (2026). As três propriedades alvo deste trabalho, irreversibilidade de *logout*, monotonicidade de contadores e

consistência de bloqueio, têm respaldo direto nessas fontes normativas, e a formulação delas para o estado replicado é derivação do trabalho. A combinação proposta, em um mesmo trabalho, de especificação formal, verificação por *model checking*, implementação experimental em um *data grid* de uso corrente e um controlador heurístico orientado a invariantes e a SLO constitui a contribuição pretendida. O posicionamento dessa combinação em relação aos trabalhos correlatos aparece no Capítulo 3, a partir do mapeamento sistemático ali descrito.

1.3 Objetivos

Objetivo geral

Especificar e verificar invariantes do estado de sessão sob replicação assíncrona e *failover* em *data grids*, e avaliar um mecanismo de decisão heurístico que modifique políticas ou ações operacionais para reduzir a ocorrência de violações desses invariantes e controlar métricas de latência sob falhas.

Objetivos específicos

Constituem objetivos específicos deste trabalho:

(1) definir um modelo formal do estado de sessão e das operações relevantes (*login*, validação, renovação, invalidação e atualização de contadores e marcadores), incluindo atrasos e reordenação de propagação de atualizações e eventos de falha ou recuperação;

(2) declarar invariantes de segurança, correção e propriedades de convergência, e verificar essas propriedades via *model checking*, registrando contraexemplos quando existirem;

(3) estabelecer uma arquitetura experimental com Infinispan em *cluster*, com suporte a alterações de topologia e cenários de replicação, instrumentando um programa de carga que gera operações concorrentes e detecta violações de invariantes;

(4) implementar um controlador heurístico que, com base em taxas e parâmetros observáveis, selecione entre políticas e aplique ações de mitigação;

(5) comparar experimentalmente o modelo estático com o modelo acrescido do controlador em cenários com injeção de falhas, quantificando taxa de violações, tempo de recuperação e convergência, e estatísticas de latência.

1.4 Organização do texto

O restante deste trabalho está organizado em quatro capítulos. O Capítulo 2 apresenta a fundamentação teórica. O Capítulo 3 detalha a metodologia, organizada em duas linhas de trabalho, sintetiza os trabalhos correlatos derivados do mapeamento sistemático e apresenta o cronograma das duas etapas do TCC. O Capítulo 4 reúne o desenvolvimento e os resultados parciais correspondentes ao Trabalho de Conclusão de Curso I, em particular a arquitetura experimental documentada e a especificação formal inicial. O Capítulo 5 sintetiza as considerações finais parciais, registra as limitações conhecidas e o planejamento para o Trabalho de Conclusão de Curso II, cujo plano expandido aparece no Apêndice A.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo consolida a base teórica e técnica da pesquisa, descrevendo as restrições fundamentais dos sistemas distribuídos e seus modelos de consistência com a arquitetura prática dos *In-Memory Data Grids* materializada pela plataforma Infinispan. A validação estrutural desse ambiente concorrente é explorada por meio das técnicas de especificação formal e *model checking* com TLA+, cuja garantia a priori é complementada pela análise do comportamento dinâmico do sistema através do estudo da latência de cauda e da definição de Objetivos de Nível de Serviço (SLO). A estabilização dessas métricas operacionais recorre aos princípios teóricos de sistemas autoadaptativos e controle por realimentação, estratégias de mitigação que culminam na caracterização do domínio prático da gerência de sessão sob normativas de segurança.

2.1 Sistemas distribuídos: consistência, replicação e falhas

Um sistema distribuído é um conjunto de processos sequenciais que se comunicam por troca de mensagens, sem relógio global comum. Para raciocinar sobre as execuções desse sistema, Lamport (1978) introduz a relação de precedência causal, denotada $\rightarrow$, definida como a menor ordem parcial irreflexiva que satisfaz duas regras: se duas ações a e b pertencem ao mesmo processo e a ocorre antes de b, então $a \rightarrow b$; e se a é o envio de uma mensagem m e b é o seu recebimento, então $a \rightarrow b$. A relação é transitiva, e dois eventos a e b não ligados por $a \rightarrow b$ ou $b \rightarrow a$ são considerados concorrentes, denotados $a \parallel b$. Para tornar essa ordem parcial observável em cada processo, o algoritmo de relógios lógicos atribui a cada evento a um valor inteiro $C(a) \in \mathbb{N}$ que respeita a precedência causal, ou seja, $a \rightarrow b$ implica $C(a) < C(b)$. Cada processo incrementa seu relógio a cada evento local e, ao receber uma mensagem m com marca de tempo $C(m)$, atualiza o relógio para $\max(C(p), C(m)) + 1$, o que mantém a contagem coerente com as mensagens recebidas. Essa construção é a base sobre a qual modelos de consistência são definidos como restrições sobre quais ordens totais dos eventos são admissíveis como observação externa do sistema.

A literatura distingue uma hierarquia de modelos com garantias decrescentes. A linearizabilidade, definida por Herlihy e Wing (1990), exige que toda execução concorrente seja equivalente a alguma ordenação sequencial das operações que preserva a ordem temporal de operações não sobrepostas: se a resposta de uma operação a ocorre antes do

início de b em tempo real, então a precede b na ordem sequencial equivalente. Cada operação tem efeito em um único instante situado entre sua invocação e sua resposta, e o efeito visível corresponde ao de uma cópia única do objeto. Duas propriedades adicionais distinguem o modelo. A primeira é a localidade: um sistema composto por múltiplos objetos é linearizável se cada objeto individualmente o for, propriedade que sustenta composição modular. A segunda é o caráter não-bloqueante: processos que invocam operações totalmente definidas não são forçados a esperar. A consistência sequencial relaxa a exigência temporal: admite-se qualquer ordem total que preserve a ordem de programa de cada processo, sem vínculo com o tempo real. A consistência causal preserva apenas $\rightarrow$, e a consistência de sessão garante propriedades relativas a uma sequência ligada a um cliente, como leitura das próprias escritas e leituras monotônicas. A consistência eventual apenas exige convergência das réplicas na ausência de novas operações (RUIZ-FUERTES; PALLARDÓ-LOZOYA; MUÑOZ-ESCOÍ, 2012; LIU *et al.*, 2015).

Esses modelos não podem coexistir todos ao mesmo tempo na presença de falhas e atrasos não limitados; duas barreiras teóricas delimitam o que é alcançável. A primeira é o teorema CAP, formalizado por Gilbert e Lynch (2002) a partir da conjectura de Brewer. O teorema afirma que, em presença de partição de rede, um sistema distribuído não pode oferecer ao mesmo tempo consistência forte e disponibilidade. A segunda barreira opera no plano da computabilidade: Fischer, Lynch e Paterson (1985) demonstram que nenhum protocolo de consenso é totalmente correto apesar de uma falha em um sistema assíncrono no qual cada processo dispõe de registradores de entrada e saída e a única falha admitida é a parada silenciosa. A demonstração constrói uma execução infinita que evita a decisão por argumento de bivalência: a partir de uma configuração inicial bivalente, na qual ainda existe extensão que decide 0 e extensão que decide 1, sempre se pode escolher uma transição que mantém a bivalência, o que produz uma execução infinita sem decisão. Em conjunto, CAP e FLP delimitam o espaço de soluções: ou se enfraquecem as hipóteses do modelo, com sincronia parcial, aleatorização ou detectores de falha, ou se relaxa a exigência de consistência, ou se aceita perda de disponibilidade enquanto a anomalia perdura.

A literatura responde a essas duas barreiras de formas complementares. No plano teórico, Chandra e Toueg (1996) contornam o resultado de impossibilidade ao introduzir detectores de falha caracterizados por duas propriedades. A primeira é a completude, que pode ser forte, quando todo processo que falha é permanentemente suspeito por todo processo correto, ou fraca, quando todo processo que falha é permanentemente suspeito

por algum processo correto. A segunda é a acurácia, admitindo quatro variantes (perpétua forte, perpétua fraca, eventual forte e eventual fraca), conforme nenhum processo correto seja jamais suspeito ou nenhum o seja após algum instante. A combinação das duas dimensões produz oito classes que se reduzem a quatro pela relação de redução entre detectores; entre elas, a classe $\diamond W$ é a mais fraca conhecida que admite solução para o consenso quando uma maioria de processos é correta. Na prática, um detector que aumenta o tempo-limite a cada engano aproxima a classe $\diamond W$, pois tolera um número ilimitado de enganos desde que, a partir de algum instante, nenhum processo correto continue sob suspeita. No plano empírico, Bailis e Kingsbury (2014) documentam partições, atrasos e perdas de mensagens como eventos recorrentes em infraestruturas de produção, com exemplos publicados que incluem perdas de conectividade em centros de dados de provedores de larga escala, falhas em comutadores de núcleo e episódios em que congestionamento e falhas de configuração produzem comportamento equivalente a partições. Essa observação sustenta a inclusão de partições e variabilidade de rede nos cenários experimentais deste trabalho. Para nomear essas falhas, adota-se a taxonomia de Avižienis *et al.* (2004), que distingue falhas de conteúdo e de temporização e descreve a falha por parada, em que o serviço deixa de responder, e a falha silenciosa, em que nenhuma saída chega à interface do serviço. Nessa terminologia, a queda de nó estudada aqui é uma parada silenciosa, o atraso é uma falha de temporização tardia, e a perda parcial e a partição aparecem nos nós como falhas de temporização ou como silêncio do canal entre eles.

Sobre essa base teórica, algoritmos de consenso baseados em líder eleito constituem a resposta prática quando a maioria dos nós permanece operacional. Ongaro e Ousterhout (2014) apresentam o algoritmo Raft com o objetivo declarado de inteligibilidade, decompondo o problema em três sub-problemas: eleição de líder, replicação do registro de operações e segurança. O tempo é particionado em mandatos numerados monotonicamente; cada nó vota em no máximo um candidato por mandato, e o nó que recebe votos da maioria torna-se líder. A detecção de ausência de líder se dá por tempo-limite de eleição aleatorizado, recomendado pelos autores na faixa de 150 a 300 milissegundos, com sorteio independente por nó para reduzir colisão de candidaturas; o líder envia periodicamente mensagens de presença (`AppendEntries` sem entradas), e os autores recomendam que o tempo de difusão de uma mensagem, estimado entre 0,5 e 20 milissegundos, fique uma ordem de grandeza abaixo do tempo-limite de eleição. A replicação ocorre pela operação `AppendEntries`: cada entrada inclui o mandato em que foi criada e um índice no registro, e o líder considera uma entrada confirmada quando ela está

armazenada na maioria dos nós, momento em que pode ser aplicada à máquina de estado replicada. Cinco propriedades sustentam a segurança do algoritmo. A primeira garante no máximo um líder por mandato (*election safety*). A segunda proíbe o líder de sobrescrever entradas de seu próprio registro (*leader append-only*). A terceira garante que dois registros com a mesma entrada no mesmo índice são idênticos em todas as posições anteriores (*log matching*). A quarta afirma que qualquer entrada confirmada em um mandato está presente no registro de todo líder de mandato superior (*leader completeness*). A quinta afirma que dois nós nunca aplicam comandos distintos em um mesmo índice (*state machine safety*). Raft fornece, portanto, replicação síncrona com tolerância a até $\lfloor (n-1)/2 \rfloor$ falhas em um *cluster* de n nós, e fica como ponto de comparação para os mecanismos otimistas tratados a seguir e para o desenho do canal de replicação modelado na vertente formal deste trabalho.

Os algoritmos de consenso resolvem a coordenação quando a maioria dos nós permanece ativa, mas a confirmação síncrona pode ultrapassar o limite de latência que a aplicação admite. Nesses casos, a replicação otimista oferece um caminho alternativo, e o sistema Dynamo é o exemplo de referência desse desenho em produção. A replicação síncrona exige confirmação remota por um quórum de réplicas antes de retornar ao cliente, o que atrela a latência da operação ao tempo de ida e volta. A replicação assíncrona, ou otimista, confirma a escrita localmente e a propaga em segundo plano. O Dynamo combina cinco mecanismos, e a replicação é regulada pela tripla (N, R, W), em que N é o número de réplicas, R é o número mínimo de nós em uma leitura e W o número mínimo em uma escrita. Com $R + W > N$, o sistema se comporta como um sistema de quórum; como a latência é ditada pela mais lenta das R ou W réplicas, R e W costumam ficar abaixo de N (DECANDIA *et al.*, 2007, seção 4.5). Em produção, a Amazon adota tipicamente $N = 3$ com $R = W = 2$, configuração que equilibra durabilidade, latência e tolerância a falhas (DECANDIA *et al.*, 2007, seção 6).

Cada um dos cinco mecanismos cumpre um papel específico. Relógios vetoriais (*vector clocks*) associados a cada versão de um objeto capturam a causalidade entre versões, pois uma versão é ancestral de outra quando todos os contadores do seu vetor são menores ou iguais aos da outra. Quando não há essa relação, há conflito, e o sistema devolve as versões concorrentes à aplicação na leitura seguinte para que sejam fundidas. O quórum relaxado (*sloppy quorum*) permite que uma requisição seja atendida pelos W ou R nós saudáveis mais próximos na lista de preferência do objeto, e não pelos N originalmente designados, o que preserva a disponibilidade quando réplicas estão inacessíveis.

O repasse condicionado (*hinted handoff*) registra em outro nó as atualizações destinadas a um nó inacessível, com um indicador da intenção, e entrega o registro quando o destino retorna. A antientropia em árvores de Merkle reconcilia divergências de longo prazo em segundo plano, por comparação de resumos criptográficos em ramos da árvore, o que reduz a quantidade de chaves comparadas. Um protocolo de disseminação por boato mantém a visão de membros do *cluster* sem servidor central de coordenação. O caso de uso central do Dynamo é o carrinho de compras da Amazon, em que leituras e escritas continuam disponíveis mesmo diante de discos falhando, rotas de rede oscilando ou centros de dados parcialmente indisponíveis, ao custo de eventual reconciliação na aplicação.

Quando a aplicação prefere convergência automática à reconciliação por código de aplicação, os tipos de dados replicados livres de conflito (CRDT) oferecem uma alternativa com base formal. Shapiro *et al.* (2011) formalizam a Consistência Eventual Forte (SEC), que exige convergência determinística do estado das réplicas a partir do conjunto de operações aplicadas, sem coordenação durante a escrita. Duas formas suficientes para SEC são identificadas. Na abordagem por estado (CvRDT), o estado da réplica forma um semirreticulado, a fusão calcula o supremo $\sqcup$ de dois estados, que é comutativo, associativo e idempotente, e as atualizações só fazem o estado crescer na ordem parcial, o que garante que toda sequência de fusões converge para o mesmo estado independentemente da ordem em que ocorre. Na abordagem por operação (CmRDT), as operações concorrentes comutam, e o artigo supõe uma difusão confiável em ordem causal, que entrega cada operação exatamente uma vez a cada réplica. Dois exemplos do artigo ilustram o conceito. O contador puro de crescimento (G-Counter) mantém, em cada réplica i, um inteiro x_i não-decrescente, e o valor agregado é a soma $\sum_i x_i$. O contador com decremento (PN-Counter) combina dois G-Counters, e seu valor lógico é a diferença entre eles. Trabalhos formais subsequentes demonstram que invariantes de objetos com alta disponibilidade admitem prova de segurança por composição local quando o desenho convergente é cumprido (NAIR; PETRI; SHAPIRO, 2020), o que liga o conjunto de invariantes alvo deste trabalho à literatura formal sobre replicação otimista.

2.2 In-Memory Data Grids e a plataforma Infinispan

Um *In-Memory Data Grid* é um armazenamento chave-valor mantido em memória principal, distribuído entre nós, com particionamento por chave, replicação configurável e mecanismos de realocação quando a composição do *cluster* muda. A categoria

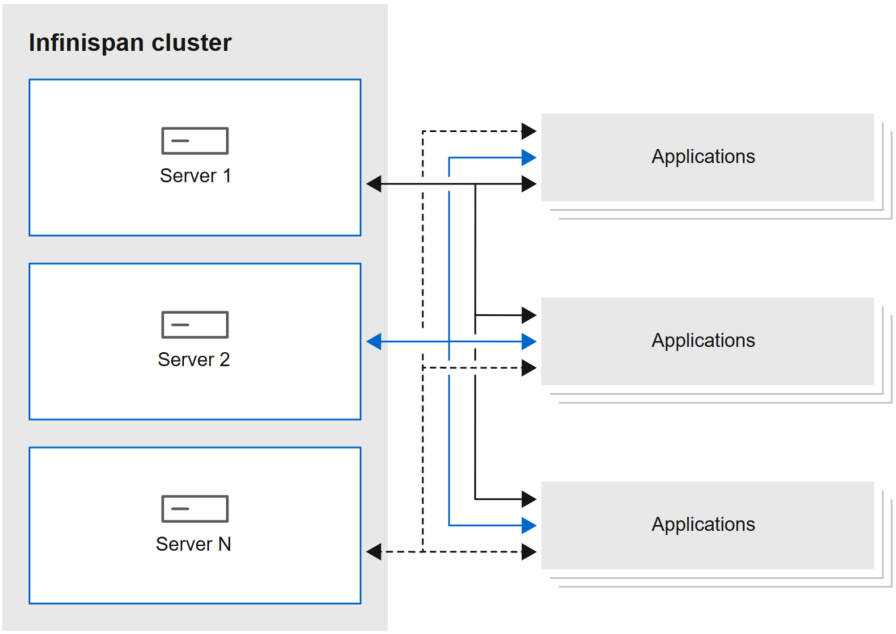
responde a duas demandas: latência inferior à de armazenamentos com persistência em disco e externalização de estado de aplicação sem ponto único de falha. Em arquiteturas em nuvem, separar uma camada de computação sem estado de uma camada de armazenamento com estado é o que permite escalabilidade automática e alta disponibilidade (ARMBRUST *et al.*, 2010), e um IMDG pode ocupar o papel dessa segunda camada.

O Infinispan é a plataforma escolhida para a vertente experimental, essa definição é feita por quatro razões. A primeira e principal é a aderência ao objeto de estudo, o sistema implementa de forma nativa a replicação assíncrona, nos modos `DIST_ASYNC` e `REPL_ASYNC`, além do *failover* e do *state transfer*, que constituem o fenômeno que este trabalho pretende especificar e verificar; a própria documentação registra que o modo assíncrono admite divergência temporária entre réplicas (Infinispan Project, 2026c), ou seja, a ferramenta expõe por configuração as condições sob as quais os invariantes de sessão podem falhar. A segunda é prática: Infinispan Project (2026f) descreve o sistema como um armazenamento chave-valor em memória escrito em Java, de código aberto, com modos de operação que vão do uso como biblioteca embarcada a serviço acessível por Hot Rod ou REST, o que favorece a reprodutibilidade do experimento, a instrumentação do programa de carga e a ausência de barreira de licença. A terceira é a observabilidade nativa, o Infinispan expõe métricas por cache e de *cluster* em JMX (Oracle, 2023) e em formato OpenMetrics (OpenMetrics Project, 2024), insumo necessário para a instrumentação e para o controlador heurístico previsto (Infinispan Project, 2026d). A quarta é a representatividade, por se tratar de um IMDG usado para externalizar estado de sessão em arquiteturas de nuvem (ARMBRUST *et al.*, 2010), o Infinispan figura, ao lado do Hazelcast, entre os sistemas de cache distribuído avaliados comparativamente em desempenho na literatura (SALHI *et al.*, 2018). O guia de configuração descreve quatro modos de *cache*: local, replicado, distribuído e invalidação (Infinispan Project, 2026a, seção 1.3). O modo local (`LOCAL`) mantém o *cache* apenas no nó atual. O replicado mantém cada entrada em todos os nós do *cluster*, e o distribuído, em um número fixo de cópias, configurado pelo parâmetro `numOwners`; ambos admitem confirmação síncrona ou propagação assíncrona, o que dá os nomes `REPL_SYNC`, `REPL_ASYNC`, `DIST_SYNC` e `DIST_ASYNC`. O modo de invalidação (`INVALIDATION`) dispara apenas mensagens para que outros nós descartem cópias locais quando há atualização. Em modo distribuído, o dono primário coordena escritas e $\texttt{numOwners} - 1$ nós atuam como reservas, e uma escrita custa no máximo $2 \cdot \texttt{numOwners}$ mensagens. A documentação adverte que o modo distribuído assíncrono pode perder atualizações e admitir leituras desatualizadas

mesmo pela *thread* que acabou de escrever a chave, e que o nó de origem não recebe confirmação dos demais nós sobre a operação (Infinispan Project, 2026a, seções 2.2 e 2.4). Esse comportamento afeta a escolha do modo para aplicações que não toleram tais efeitos e é o objeto da comparação entre os modos síncrono e assíncrono neste trabalho.

A Figura 1 ilustra a topologia de implantação cliente-servidor do Infinispan. O *cluster*, à esquerda, reúne os servidores que mantêm o armazenamento distribuído, de `Server 1` a `Server N`, enquanto as aplicações, à direita, atuam como clientes remotos e alcançam qualquer servidor pelo protocolo binário Hot Rod ou por REST. As setas entre aplicações e servidores representam requisições de leitura e escrita atendidas por nós distintos, o que evidencia a ausência de afinidade rígida entre cliente e nó e sustenta a continuidade do atendimento quando um servidor deixa o *cluster*.

Figura 1 – Topologia cliente-servidor de um *cluster* Infinispan, em que as aplicações acessam os servidores por protocolo remoto.



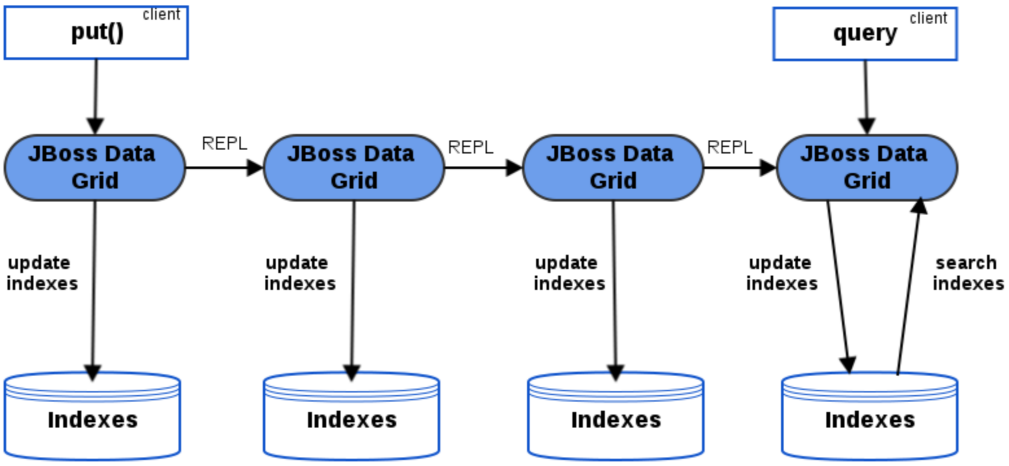
Fonte: Infinispan Project (2026e)

A distribuição de chaves entre nós segue o esquema de *consistent hashing*, onde o conjunto de chaves é dividido em um número fixo de segmentos, definido por `numSegments`, e cada segmento é atribuído a uma lista de donos (Infinispan Project, 2026a, seção 2.2.2). O esquema padrão atribui as chaves aos mesmos nós em todos os *caches*, desde que todos os *caches* existam em todos os nós, ao custo de uma distribuição de carga levemente desigual (Infinispan Project, 2026a, seção 2.2.2); a variante ciente da topologia distribui as cópias por níveis topológicos como sites, racks e máquinas, para

reduzir a chance de perda em falhas correlacionadas; e a variante de equilíbrio uniforme otimiza o balanceamento com a desvantagem de o resultado depender da ordem de entrada dos nós no *cluster*. Quando a composição do *cluster* muda, ocorre rebalanceamento, e segmentos são transferidos entre nós; durante a transição, mudanças de topologia podem causar repetições e mensagens adicionais para operações de leitura e escrita.

Nos modos `REPL_SYNC` e `REPL_ASYNC`, em contraste com a atribuição por segmentos própria dos modos distribuídos, cada nó mantém uma cópia completa de todas as entradas. A Figura 2 representa esse comportamento em um *data grid* replicado. Uma escrita submetida por um cliente, denotada `put()`, é aplicada em um nó e propagada aos demais pela replicação, assinalada como `REPL`, e cada nó atualiza os próprios índices locais a partir da atualização recebida. Uma consulta submetida por outro cliente, denotada `query`, é atendida localmente pelo nó que a recebe, pois todos os nós observam o mesmo conjunto de entradas depois de concluída a propagação. A figura torna visível o custo central da replicação assíncrona, dado que no intervalo entre a escrita local e a entrega da atualização aos demais nós as réplicas permanecem temporariamente divergentes.

Figura 2 – Replicação de uma escrita entre os nós de um *data grid*, com atualização dos índices por nó e leitura por consulta.



Fonte: Red Hat (2016)

Para cenários multisite, a replicação *cross-site*, ou seja, a replicação entre datacenters geograficamente distintos, permite cópias síncronas ou assíncronas entre *clusters* separados (Infinispan Project, 2026b). A documentação registra duas regras importantes. A primeira é operacional: cópias assíncronas não adicionam latência às requisições do cliente, enquanto as síncronas introduzem o RTT entre sites em cada es-

crita, o que torna a opção assíncrona recomendada para configurações ativo-ativo em que dois sítios recebem escritas concorrentes. A segunda diz respeito à reconciliação: a detecção de conflitos por relógios vetoriais aciona políticas de resolução, com a política padrão usando comparação lexicográfica do nome do *cluster*, e alternativas `ALWAYS_REMOVE`, `PREFER_NON_NULL`, `PREFER_NULL` ou implementação customizada pelo SPI `XSiteEntryMergePolicy`. A coleta de métricas operacionais segue documentação dedicada (Infinispan Project, 2026d): contadores por cache, indicadores de *cluster*, sinais de *cross-site*, latências médias e máximas, integração com JMX e exposição como Prometheus (Prometheus Authors, 2026). Esses observáveis sustentam a instrumentação prevista para o controlador heurístico em etapa posterior. Um ponto operacional ressaltado pela documentação merece registro: mesmo com replicação síncrona, caches distribuídos não são linearizáveis, dado que a escrita não é atômica entre os donos (Infinispan Project, 2026a, seção 2.2.1). Essa observação tem implicação direta sobre a formulação dos invariantes de sessão deste trabalho, dado que linearizabilidade não pode ser tomada como premissa nem no modo síncrono.

2.3 Especificação formal e *model checking*

A verificação por *model checking* explora exaustivamente o espaço de estados de um modelo finito do sistema, com o objetivo de certificar a validade de propriedades declaradas ou produzir contraexemplos quando essas propriedades são violadas. Yu, Manolios e Lamport (1999) descrevem TLA+ como uma linguagem de especificação que combina a lógica temporal de ações TLA com lógica de primeira ordem completa e teoria dos conjuntos no estilo de Zermelo-Fraenkel. Uma especificação em TLA+ assume a forma canônica $Spec \triangleq Init \wedge \Box[Next]_v$, em que *Init* é um predicado sobre as variáveis v que caracteriza os estados iniciais, *Next* é a relação de transição expressa como disjunção de ações, e $\Box$ é o operador temporal que afirma que a fórmula a ele subordinada vale em todo estado da execução. Cada ação é um predicado sobre os valores das variáveis no estado corrente e no estado seguinte, denotadas x e x'. Propriedades de invariância são fórmulas $\Box P(v)$ que afirmam que todos os estados alcançáveis satisfazem P; propriedades temporais como $\Box(P \Rightarrow \Diamond Q)$ expressam restrições sobre sequências de estados e afirmam que todo P é eventualmente seguido por um Q. O verificador TLC aceita um subconjunto de TLA+ adequado a descrições de sistemas reais, gera o grafo de estados acessíveis a partir dos estados iniciais por busca em largura, mantém um conjunto de estados visitados

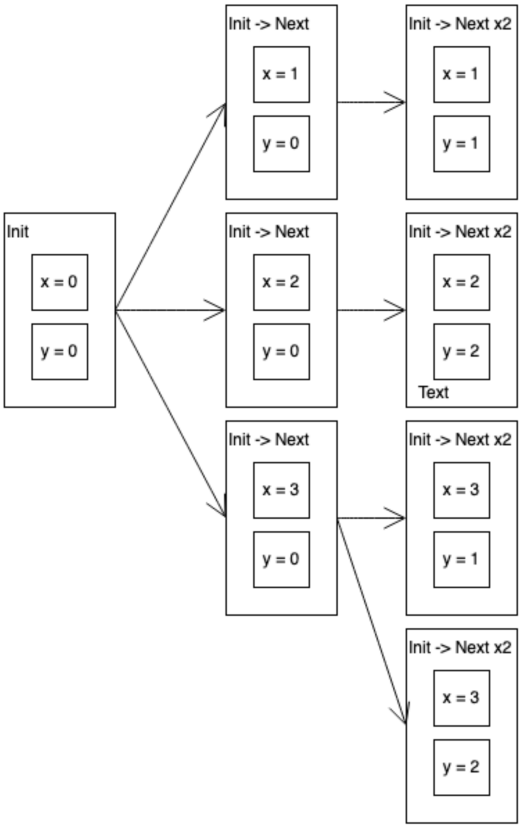
por impressão digital de estado, e, quando encontra um estado que viola um invariante, retorna a sequência de transições que reproduz a violação. Yu, Manolios e Lamport (1999) registram o uso do TLC na especificação do protocolo de coerência de *cache* de um novo multiprocessador da Compaq, com cerca de 1.800 linhas, na qual o verificador encontrou perto de duas dezenas de erros, dois deles presentes também na implementação em RTL, que foi corrigida (YU; MANOLIOS; LAMPORT, 1999, seção 7).

A Figura 3 ilustra, em um exemplo mínimo com duas variáveis inteiras x e y, a forma como o TLC percorre o espaço de estados. O estado inicial, definido por *Init*, atribui o valor zero às duas variáveis. Cada aplicação da relação de transição *Next* produz os estados sucessores admissíveis, e o não determinismo da relação faz um mesmo estado originar mais de um sucessor, como no ramo em que o estado com $x = 3$ dá origem a dois estados distintos após a segunda transição. O verificador repete esse passo sobre cada estado gerado e mantém o conjunto dos estados já visitados para não reexplorá-los. A verificação de um invariante avalia o predicado declarado em cada estado alcançado, e a primeira avaliação falsa interrompe a busca e devolve a sequência de transições que reproduz a violação.

A representação eficiente do espaço de estados é o gargalo central da abordagem. Yu, Manolios e Lamport (1999) relatam estratégias para armazenar conjuntos de estados em disco e usar impressões digitais de 64 bits, únicas com alta probabilidade, e essas opções determinam o tamanho dos modelos tratáveis. Em sistemas distribuídos, o espaço cresce com o número de nós, o tamanho da fila de mensagens em trânsito e a quantidade de tipos de evento, com explosão combinatória que motiva abstrações por simetria, por particionamento ou por redução parcial de ordem (ENEA *et al.*, 2024). Daí a opção recorrente, no desenho de modelos formais para esses sistemas, por abstrações que mantenham a essência da concorrência sem replicar todos os detalhes da implementação. A aplicação industrial subsequente, com destaque para o registro de adoção pela Amazon Web Services, documenta a identificação de defeitos sutis em protocolos de replicação. Newcombe *et al.* (2015) relatam que a verificação formal encontrou, no projeto do DynamoDB, um defeito que poderia levar à perda de dados por meio de uma sequência de 35 passos envolvendo interações entre transações concorrentes, particionamento e recuperação, defeito que escapou a revisões extensivas de desenho, de código e a testes convencionais, e que motivou alteração no protocolo antes da implantação em produção.

A produção contemporânea estende a técnica em três direções. A primeira combina verificação formal e teste de implementações: um modelo formal guia testes in-

Figura 3 – Exploração do espaço de estados pelo verificador TLC, em que a relação de transição gera os estados sucessores a partir do estado inicial.



Fonte: Lee (2021)

crementais sobre o sistema real, com o objetivo de aproximar o que foi verificado do que está executando, por instrumentação que mapeia transições do modelo a observações no código (GAO *et al.*, 2025). A segunda verifica diretamente protocolos distribuídos com semântica de mensagens assíncronas e falhas, em ferramentas projetadas para esse propósito; Must oferece uma interface de concorrência com vários modelos de comunicação, verifica os programas por redução parcial de ordem dinâmica e foi aplicada a sistemas de produção, como *logs* replicados e protocolos de transação distribuída (ENEA *et al.*, 2024). A terceira é a verificação probabilística de modelos: o resultado da análise é uma probabilidade de violação sob distribuições associadas a parâmetros do sistema, e aplica-se a decisões de escalonamento horizontal em bancos NoSQL ciente de custo (NASKOS; GOUNARIS; KATSAROS, 2017). A escolha da ferramenta para este trabalho, entre TLA+/TLC e as alternativas, fica detalhada e justificada na Seção 3.2.

A verificação exaustiva só é tratável com cardinalidades pequenas, e a pertinência desse recorte tem apoio empírico. Yuan *et al.* (2014), ao analisar falhas reportadas em

sistemas distribuídos de produção, constatam que 98% delas se manifestam com três nós ou menos. A observação sustenta tanto os modelos com duas ou três réplicas verificados pelo TLC quanto o *cluster* experimental de três nós, embora não garanta que todo defeito do protocolo apareça nessas cardinalidades.

2.4 Latência de cauda, objetivos de nível de serviço e observabilidade

A distribuição de latência de um serviço composto por muitos componentes tem cauda longa, e o impacto dessa cauda sobre o tempo total observado pelo usuário é desproporcional ao seu peso na distribuição. Dean e Barroso (2013) ilustram o fenômeno com um caso em que a latência típica de uma requisição fica em torno de 10 milissegundos, com 1% das requisições demorando mais de 1 segundo (cem vezes a típica). Em sistemas de *fan-out*, em que uma requisição do usuário se decompõe em n requisições independentes a componentes distintos e depende da resposta mais lenta, a probabilidade de pelo menos uma resposta cair na cauda cresce rapidamente com n: se a probabilidade de uma resposta individual ser lenta é $p = 0{,}01$, a probabilidade de pelo menos uma resposta lenta em $n = 100$ componentes é $1 - (1-p)^n \approx 0{,}63$, ou seja, 63% das requisições do usuário tocam pelo menos uma resposta na cauda. As fontes de variabilidade apontadas pelos autores incluem disputa por recursos compartilhados na máquina e na rede, atividades de manutenção em segundo plano, como a coleta de lixo, filas em servidores e comutadores e mecanismos de economia de energia do *hardware*. Por isso, os percentis p95 e p99, e em sistemas críticos o p99,9, são as métricas primárias em vez da média. As técnicas de tolerância à cauda incluem requisições redundantes (*hedged requests*), em que o cliente envia uma cópia secundária após T milissegundos de espera, com T tipicamente fixado no p95 esperado, e cancela a réplica que perde a corrida; e requisições atadas (*tied requests*), em que servidores que recebem cópias se comunicam para cancelar a duplicata assim que um deles começa o atendimento. Esperar até o p95 limita a carga adicional das requisições redundantes a cerca de 5%, e em um exemplo dos autores 2% de requisições a mais reduziram o p99,9 de 1.800 para 74 milissegundos; nas requisições atadas, a sobrecarga medida em uso de disco ficou abaixo de 1%.

A formulação dessas metas como objetivos de nível de serviço segue prática consolidada (Google, 2016). Um indicador de nível de serviço (SLI) é definido como uma medida quantitativa de algum aspecto do nível de serviço fornecido. Um objetivo de nível de serviço (SLO) é um valor-alvo, ou intervalo de valores, para o nível de serviço me-

dido por um SLI, expresso como SLI $\leq$ alvo ou limite inferior $\leq$ SLI $\leq$ limite superior; um exemplo prescrito pela literatura é "99% das chamadas de `Get` completam em menos de 100 milissegundos". Um acordo de nível de serviço (SLA) é um contrato explícito ou implícito com os usuários, com consequências do cumprimento ou descumprimento dos SLO nele contidos. O orçamento de erros é a taxa com que os SLOs podem deixar de ser cumpridos, acompanhada diariamente ou semanalmente, e seu consumo funciona como entrada para decisões operacionais. Sobre agregação, médias simples obscurecem detalhes importantes, e a combinação de mediana, p99 e p99,9 caracteriza a forma da distribuição com mais fidelidade do que qualquer estatística única.

A pesquisa específica em armazenamentos chave-valor estuda como reduzir a cauda de latência por agendamento de consultas, com algoritmos que tratam heterogeneidade de operações (JAIMAN; BEN MOKHTAR; RIVIÈRE, 2020; BEN MOKHTAR *et al.*, 2021), e estende esse trabalho em versão expandida com avaliação ampla de cenários (BEN MOKHTAR *et al.*, 2024). A observabilidade operacional encontra-se sistematizada em estudos sobre rastreamento e análise em microsserviços, que documentam práticas de instrumentação, retenção e análise de sinais sobre execuções distribuídas (LI *et al.*, 2021). No Infinispan, os contadores e indicadores publicados conforme a documentação de métricas dão acesso aos observáveis necessários para correlacionar variações de latência e eventos operacionais com violações de invariantes (Infinispan Project, 2026d).

A avaliação experimental de sistemas computacionais precisa lidar com a variabilidade entre execuções. Georges, Buytaert e Eeckhout (2007) mostram que o desempenho de programas Java não é determinístico, por efeito de fatores como a compilação dinâmica, a coleta de lixo e o escalonamento de *threads*, e que metodologias de avaliação sem rigor estatístico podem levar a conclusões enganosas. Para médias estimadas a partir de poucas execuções, o intervalo de confiança usa a distribuição *t* de Student (1908). Hoefler e Belli (2015) sistematizam regras para relatar resultados de desempenho, entre elas tratar a execução repetida como unidade de medida, reportar a variabilidade por intervalos de confiança e preferir procedimentos não paramétricos quando os dados não seguem distribuição normal. Para estatísticas sem distribuição amostral conhecida, como os percentis de latência, o *bootstrap* aproxima essa distribuição por reamostragem dos dados observados, e o intervalo percentil toma os quantis da distribuição reamostrada (EFRON; TIBSHIRANI, 1993). O teste de Mann e Whitney (1947) compara duas amostras sem supor normalidade e verifica se uma das variáveis tende a assumir valores maiores que a outra. Para proporções, Brown, Cai e DasGupta (2001) mostram que o intervalo padrão

baseado na aproximação normal da binomial tem cobertura errática, que se afasta do nível nominal de modo irregular, o que recomenda cautela no uso dessa aproximação.

Quando o mesmo pesquisador desenha, executa e analisa um experimento, decisões tomadas depois de ver os dados podem ser apresentadas, sem intenção, como previsões. Nosek *et al.* (2018) defendem o pré-registro como resposta: hipóteses, desfechos e plano de análise são fixados antes da coleta, o que separa a análise confirmatória da exploratória.

A estimativa de energia parte de modelos de potência por utilização. Fan, Weber e Barroso (2007) observam que a potência de um servidor varia de forma aproximadamente linear com a utilização da CPU, o que permite estimá-la por $P(u) = P_{\text{ocioso}} + (P_{\text{máx}} - P_{\text{ocioso}}) \cdot u$, em que u é a utilização. Barroso e Hölzle (2007) mostram que servidores consomem parcela elevada da potência de pico mesmo quando ociosos e passam a maior parte do tempo em utilização baixa ou moderada, de modo que a parcela ociosa pesa na energia por operação. Em máquinas virtuais, a potência não pode ser medida diretamente no *hardware* e precisa ser estimada por modelos alimentados por contadores de uso de recursos (KANSAL *et al.*, 2010). No Linux, a interface de grupos de controle versão 2 (cgroup v2) expõe o tempo de CPU acumulado por grupo de processos, o que permite atribuir o tempo de CPU a cada *container* (HEO, 2026).

2.5 Sistemas autoadaptativos e controle por realimentação

Um sistema autoadaptativo monitora seu próprio comportamento, analisa desvios em relação a metas e atua sobre parâmetros configuráveis para retornar à região operacional. Em ambientes de nuvem, a literatura organiza essas atividades por meio de taxonomias e revisões que descrevem mecanismos de elasticidade sob a ótica da teoria de controle, com critérios como estabilidade, tempo de acomodação e prevenção de oscilação (ULLAH *et al.*, 2018). Trabalhos de implementação demonstram autoescala em armazenamentos de nuvem hierárquicos (WU; SREEKANTI; HELLERSTEIN, 2020) e gestores proativos de elasticidade autotreinados (LIU *et al.*, 2017). Para o desenho de controladores adaptativos, a literatura discute mecanismos de garantia que combinam evidência empírica e propriedades formais (GAMBI; TOFFETTI; PEZZÈ, 2013), e propõe políticas de auto-gestão baseadas em objetivos de alto nível em sistemas distribuídos (ROSA; RODRIGUES; LOPES, 2013). A referência conceitual comum a esses trabalhos é o laço monitorar-analisar-planejar-executar sobre uma base de conhecimento (MAPE-K), do ge-

rente autônomo descrito por Kephart e Chess (2003).

O controlador previsto para etapa posterior deste trabalho é heurístico, com regras determinísticas e mecanismos de histerese para evitar oscilação, e atua sobre um conjunto finito de configurações pré-definidas. A escolha do desenho heurístico, em lugar de soluções baseadas em aprendizado de máquina ou em controle ótimo, segue dois critérios. O primeiro é a auditabilidade das decisões: cada ação aplicada pelo controlador pode ser rastreada para a regra que a motivou, o que sustenta análise de causa raiz após qualquer ocorrência. O segundo é a tratabilidade formal: regras determinísticas com observáveis bem-definidos prestam-se à análise pelo modelo formal do trabalho, conforme delineado no Capítulo 3. Ullah *et al.* (2018) apontam que controladores de elasticidade precisam evitar oscilação e se acomodar rapidamente após perturbações, e três fenômenos operacionais merecem atenção. O primeiro é a alternância rápida entre estados de configuração, que introduz instabilidade e impede que qualquer estado se mantenha por tempo suficiente para produzir efeito. O segundo é a sobre-resposta, em que a reação supera a correção necessária e leva o sistema a oscilar em torno do valor-alvo. O terceiro é o atraso de fase, em que a ação chega depois da janela em que produziria efeito. Regras com histerese e limites de taxa de mudança são respostas diretas a esses três fenômenos.

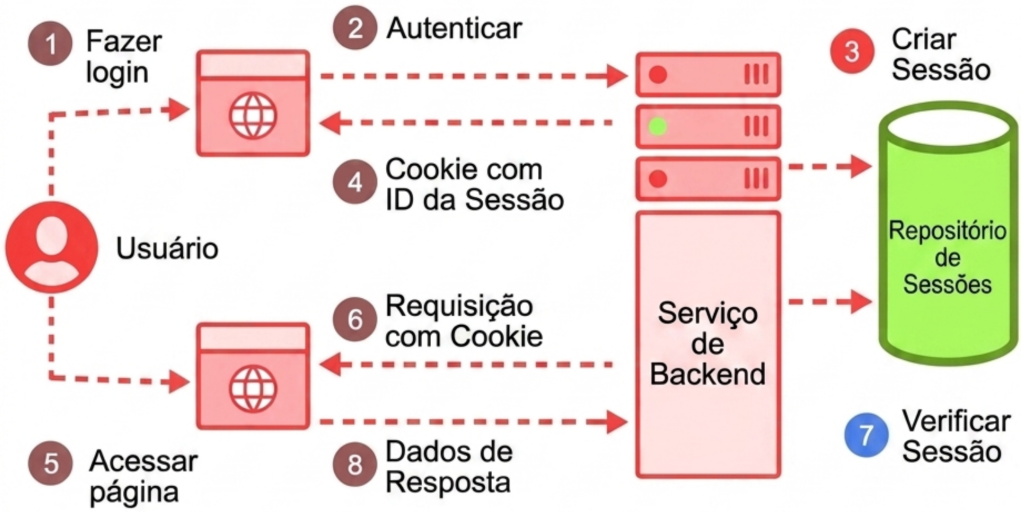
2.6 Gerência de sessão como domínio

A gerência de sessão constitui o estudo de caso do trabalho. O ciclo de vida de uma sessão compreende criação, validação, renovação, invalidação e bloqueio, sobre uma identidade autenticada. A literatura normativa estabelece propriedades que esse ciclo deve satisfazer. A publicação SP 800-63B-4 do NIST, que substituiu a versão de 2017, define níveis de garantia do autenticador com prazos de reautenticação próprios de cada nível, trata da gerência de sessão em seção normativa, na qual exige que os segredos de sessão sejam invalidados no *logout*, e exige a limitação do número de tentativas de autenticação malsucedidas, o chamado *rate limiting* (TEMOSHOK *et al.*, 2025, seções 2, 3.2.2 e 5). Essas duas exigências fundamentam, respectivamente, a irreversibilidade da invalidação e o par formado pelo contador de tentativas e pelo bloqueio da identidade, objetos dos invariantes deste trabalho.

A Figura 4 sintetiza o percurso de uma sessão autenticada quando o estado é externalizado para um repositório compartilhado. O usuário inicia o fluxo ao submeter credenciais em uma operação de *login*, que o serviço de *backend* autentica; confirmada a

identidade, o serviço cria a sessão, registra-a no repositório de sessões e devolve ao cliente um *cookie* que carrega apenas o identificador da sessão. Nas requisições seguintes, o cliente reapresenta esse *cookie*, e o serviço valida a sessão por consulta ao repositório antes de produzir a resposta. A figura evidencia que a correção observada pelo usuário depende do estado mantido no repositório, e essa camada, quando replicada de forma assíncrona, expõe os invariantes de sessão às condições de falha tratadas neste trabalho.

Figura 4 – Ciclo de vida de uma sessão autenticada com estado externalizado em repositório de sessões compartilhado.



Fonte: O autor

O *OWASP Session Management Cheat Sheet* (OWASP Foundation, 2026) traduz essas exigências em requisitos operacionais sobre identificadores e atributos de sessão. O identificador de sessão deve ter pelo menos 64 bits de entropia, gerados por um gerador pseudoaleatório criptograficamente seguro, e deve evitar conteúdo significativo, ou seja, qualquer informação que permita a um observador deduzir o estado da sessão. Sobre o tempo de vida, três mecanismos complementares são prescritos: o tempo-limite absoluto define o tempo máximo de existência da sessão; o tempo-limite por inatividade encerra a sessão após inatividade; o tempo-limite de renovação rotaciona o identificador em intervalos regulares para reduzir a janela de exploração de identificadores capturados. A invalidação deve ser feita no servidor e no cliente, e o identificador deve ser regenerado após qualquer mudança de privilégio, em particular após a autenticação, para impedir ataques de fixação de sessão (*session fixation*). Os atributos do *cookie* devem incluir `Secure`, `HttpOnly`, `SameSite` e o prefixo `__Host-`, para mitigar exposição em canal não seguro, acesso por código executado no cliente e envio em requisições de origem

distinta.

Externalizar o estado de sessão em um IMDG ou armazenamento chave-valor aplica essa separação entre computação sem estado e armazenamento com estado (ARMBRUST *et al.*, 2010): múltiplas instâncias servem requisições de um mesmo cliente sob balanceamento de carga. A externalização desacopla o estado do nó que atende à requisição, com o ganho de continuidade em *failover* e o custo da exposição aos efeitos da replicação assíncrona descritos na Seção 2.1. A combinação dessas duas camadas, exigências normativas e operacionais, justifica a escolha de irreversibilidade de invalidação, consistência de bloqueio e monotonicidade de contadores como invariantes alvo deste trabalho.

3 METODOLOGIA

Este é um trabalho de pesquisa aplicada, de natureza experimental e quantitativa, com objetivos exploratórios e descritivos, e segue estratégia metodológica mista, organizada em duas linhas de trabalho. A primeira aplica um mapeamento sistemático da literatura (PETERSEN *et al.*, 2008) para identificar e classificar abordagens relacionadas a invariantes de estado de sessão, replicação assíncrona, tolerância a falhas, verificação formal por *model checking* e mecanismos de decisão em sistemas distribuídos. A segunda é de pesquisa de projeto: o trabalho constrói e avalia, sob cenários parametrizados de falha, um conjunto de artefatos com recorte derivado dos achados da linha de mapeamento; os artefatos previstos são o modelo formal, o programa de carga instrumentado, o ambiente experimental em *cluster* Infinispan e, em etapa posterior, o controlador heurístico. O relato dos resultados do mapeamento adota elementos do PRISMA 2020 (PAGE *et al.*, 2021), em particular o fluxograma de seleção e a contabilização de itens por etapa. O guia em língua portuguesa de Neiva e Silva (2016) apoia a operação do mapeamento. Por se tratar de um trabalho dividido em duas etapas, Trabalho de Conclusão de Curso I e Trabalho de Conclusão de Curso II, o presente texto reporta o protocolo, a execução inicial do mapeamento e os resultados parciais relativos à primeira etapa.

3.1 Mapeamento sistemático da literatura

O mapeamento sistemático segue a sequência canônica proposta por Petersen *et al.* (2008): define as questões de mapeamento, conduz a busca em bases bibliográficas, triagem os resultados por título e resumo, classifica os trabalhos por palavras-chave extraídas dos resumos, recorrendo a outras partes do texto quando o resumo não basta, e, por fim, extrai e sintetiza os dados, e produz um esquema de classificação acompanhado de uma síntese descritiva. Esta seção sintetiza os elementos principais e relata a execução até a triagem inicial, correspondente ao escopo do presente Trabalho de Conclusão de Curso I.

Pergunta central e questões de mapeamento

A pergunta central de pesquisa, formulada na Seção 1.1, desdobra-se em sete questões de mapeamento. As questões investigam, respectivamente, quais invariantes de domínio do estado de sessão são tratados na literatura, quais modelos de falha são assumidos, quais mecanismos de replicação e reconciliação são empregados, quais abordagens e ferramentas de verificação formal são aplicadas, quais métricas e indicadores operacionais são reportados, quais mecanismos de decisão e adaptação são propostos, e quais plataformas e sistemas-alvo aparecem nos estudos empíricos. Cada questão está associada a um ou mais eixos temáticos do trabalho, conforme detalhado na Tabela 1.

Tabela 1 – Questões de mapeamento e eixos temáticos correspondentes

Identificador	Questão de mapeamento	Eixos
QM1	Quais invariantes de domínio do estado de sessão são tratados na literatura?	sessão, replicação
QM2	Quais modelos de falha são assumidos pelos trabalhos?	falhas, replicação
QM3	Quais mecanismos de replicação e reconciliação são empregados?	replicação, IMDG
QM4	Quais abordagens e ferramentas de verificação formal são aplicadas?	métodos formais
QM5	Quais métricas e indicadores operacionais são reportados?	SLO, latência, observabilidade
QM6	Quais mecanismos de decisão e adaptação são propostos?	autoadaptativo, controle
QM7	Quais plataformas e sistemas-alvo são objeto dos estudos empíricos?	IMDG

Fonte: O autor

A busca é executada nas bases ACM Digital Library, IEEE Xplore, Scopus e SpringerLink, escolhidas por reunirem as principais editoras e indexadores em Ciência da Computação. Conforme descrito na Seção 3.1, a execução inicial empregou o agregador OpenAlex como porta única de acesso, e essa decisão deve ser complementada por execução manual via Portal de Periódicos CAPES na versão definitiva do protocolo.

A pesquisa emprega seis *strings* complementares, denotadas S1 a S6, organizadas por foco temático. Cada *string* combina cláusulas *OR* de termos-foco com cláusulas *AND* de filtros transversais, e resulta de duas rodadas de calibração contra um conjunto-semente nomeado de referências-âncora. As versões finais das *strings* são as seguintes:

- **S1, replicação, consistência e IMDG:** (("in-memory data grid" OR IMDG OR "distributed cache" OR "key-value store" OR "session state" OR "session management" OR "distributed storage" OR "consistent hashing" OR "rebalancing" OR "state transfer") AND ("consistency" OR "invariant" OR "convergence" OR "reconciliation" OR "replication" OR "CRDT" OR "eventual consistency" OR "session consistency"))
- **S2, métodos formais aplicados a sistemas distribuídos:** (("model checking" OR "formal verification" OR "formal specification" OR "temporal logic" OR "TLA+" OR "TLA" OR Alloy OR SPIN OR Coq OR Isabelle OR "bounded model checking" OR "probabilistic model checking" OR "theorem proving") AND ("distributed" OR "replicated" OR "replication" OR "consistency" OR "concurrent" OR "protocol" OR "fault" OR "cache" OR "data grid"))
- **S3, SLO, latência de cauda e observabilidade:** (("SLO" OR "service level objective" OR "service level agreement" OR "tail latency" OR "p95" OR "p99" OR "percentile latency" OR observability OR "distributed tracing" OR "system monitoring") AND ("distributed" OR "data grid" OR "cache" OR "key-value" OR "replicated" OR "scale" OR "latency"))
- **S4, sistemas autoadaptativos e controle:** (("self-adaptive" OR autonomic OR "MAPE-K" OR "feedback control" OR "control theory" OR "autoscaling" OR "auto-scaling" OR "autoscaler" OR elasticity OR "self-healing" OR "self-managing" OR "self-tuning" OR "adaptive control") AND ("distributed" OR "cloud" OR "data grid" OR "key-value" OR "cache" OR "replicated" OR "fault" OR "service" OR "storage"))
- **S5, fundamentos teóricos:** (("consensus" OR "agreement protocol" OR "atomic broadcast" OR "linearizability" OR "failure detector" OR "impossibility" OR "network partition" OR

```
"CAP theorem" OR Byzantine OR Paxos OR Raft OR "state
machine replication" OR "vector clocks" OR "logical
clocks") AND ("distributed" OR "asynchronous" OR
"fault-tolerant" OR "fault tolerance" OR "process" OR
"replicat*"))
```

- **S6, sessão como domínio:** `(("session management" OR "session state" OR "session invariant" OR "session lifecycle" OR "session token" OR "user session" OR "authentication state" OR "session fixation" OR "session hijacking") AND ("distributed" OR "cluster" OR "replicat*" OR "cache" OR "data grid" OR "in-memory" OR "stateful" OR "security"))`

Critérios de inclusão e exclusão

Os critérios de inclusão filtram positivamente os trabalhos a considerar e são cinco. CI1 admite trabalho revisado por pares, obra clássica ou seminal, ou documentação técnica oficial. CI2 exige texto completo acessível. CI3 restringe os idiomas a português ou inglês. CI4 requer que o trabalho aborde, de modo substancial, pelo menos uma das questões de mapeamento. CI5 exige que o estudo trate como tema central sistemas distribuídos, replicação, consistência, falhas, verificação formal, observabilidade, controle adaptativo ou gerência de sessão.

Os critérios de exclusão definem motivos de remoção e são sete. CE1 elimina trabalhos com apenas resumo disponível. CE2 elimina duplicatas, mantida a versão mais completa do mesmo trabalho. CE3 elimina documentos técnicos comerciais sem revisão por pares e sem conteúdo técnico verificável. CE4 elimina trabalhos cujo foco está fora dos eixos relevantes. CE5 elimina idiomas distintos dos aceitos. CE6 elimina fontes sem revisão por pares quando usadas como base teórica principal; relatórios técnicos do tipo *Jepsen* são admitidos apenas como literatura cinza de apoio empírico. CE7 elimina itens anteriores a 1985, ano do resultado de impossibilidade do consenso assíncrono (FISCHER; LYNCH; PATERSON, 1985), ressalvada a manutenção dos estudos seminais desse período pela cláusula CI1. Pela cláusula de inclusão, mantêm-se os estudos seminais anteriores a 1985, ano do resultado de impossibilidade do consenso assíncrono (FISCHER; LYNCH; PATERSON, 1985).

Esquema de extração e classificação

Cada estudo primário incluído é caracterizado pelas oito dimensões a seguir, com as categorias previstas registradas para cada uma. As categorias podem ser refinadas após a etapa de leitura completa, caso achados recorrentes não previstos justifiquem o ajuste.

D1, modelo de consistência alvo:

forte; sequencial; causal; sessão; eventual; consistência eventual forte (CRDT); quórum.

D2, modelo de falhas assumido:

parada silenciosa; parada com recuperação; omissão; particionamento; bizantino; misto; não especificado.

D3, mecanismo de replicação e reconciliação:

síncrona com quórum; assíncrona com gravação por último escritor; assíncrona com resolução pela aplicação; CRDT; relógios vetoriais; consenso (Paxos, Raft); replicação por máquina de estado.

D4, abordagem ou ferramenta formal:

TLA+ com TLC; Alloy; SPIN; Coq; Isabelle; verificação probabilística de modelos; verificação limitada de modelos; sem método formal.

D5, validação empírica realizada:

em IMDG identificado; em outro sistema; em simulação; não-empírico.

D6, domínio de aplicação:

sessão ou autenticação; chave-valor genérico; banco de dados distribuído; mensageria; cache de aplicação; outro.

D7, métricas reportadas:

latência média ou de cauda (p95, p99); vazão; taxa de violação; tempo de recuperação; tempo de convergência; consumo de recursos; outras.

D8, tipo de mecanismo adaptativo:

nenhum; estático configurável; heurístico baseado em regras; MAPE-K; controle de realimentação clássico; aprendizado de máquina.

Procedimento de seleção e ameaças à validade

O procedimento de seleção compreende identificação, triagem por título e resumo, e elegibilidade por leitura completa. A identificação registra, por base, a *string* aplicada, a data de execução e a quantidade de itens retornados. A triagem por título e resumo aplica os critérios de exclusão imediatamente verificáveis. A elegibilidade aplica a leitura completa, com registro do motivo de cada exclusão. Os números obtidos em cada etapa são reportados conforme o padrão PRISMA 2020 (PAGE *et al.*, 2021). A execução inicial do mapeamento, correspondente às semanas 3 e 4 do cronograma do Trabalho de Conclusão de Curso I, foi conduzida em 13 de maio de 2026 e utilizou o agregador OpenAlex como porta única de acesso às bases declaradas, em razão de restrições de acesso programático aos portais. Essa decisão é registrada como limitação metodológica do mapeamento sistemático. A Tabela 2 sintetiza o fluxo nas três primeiras categorias e registra a etapa de Inclusão como em curso, dado que a classificação final dos estudos primários selecionados integra o Trabalho de Conclusão de Curso II.

Tabela 2 – Síntese do fluxo PRISMA 2020 do mapeamento sistemático até a etapa atual

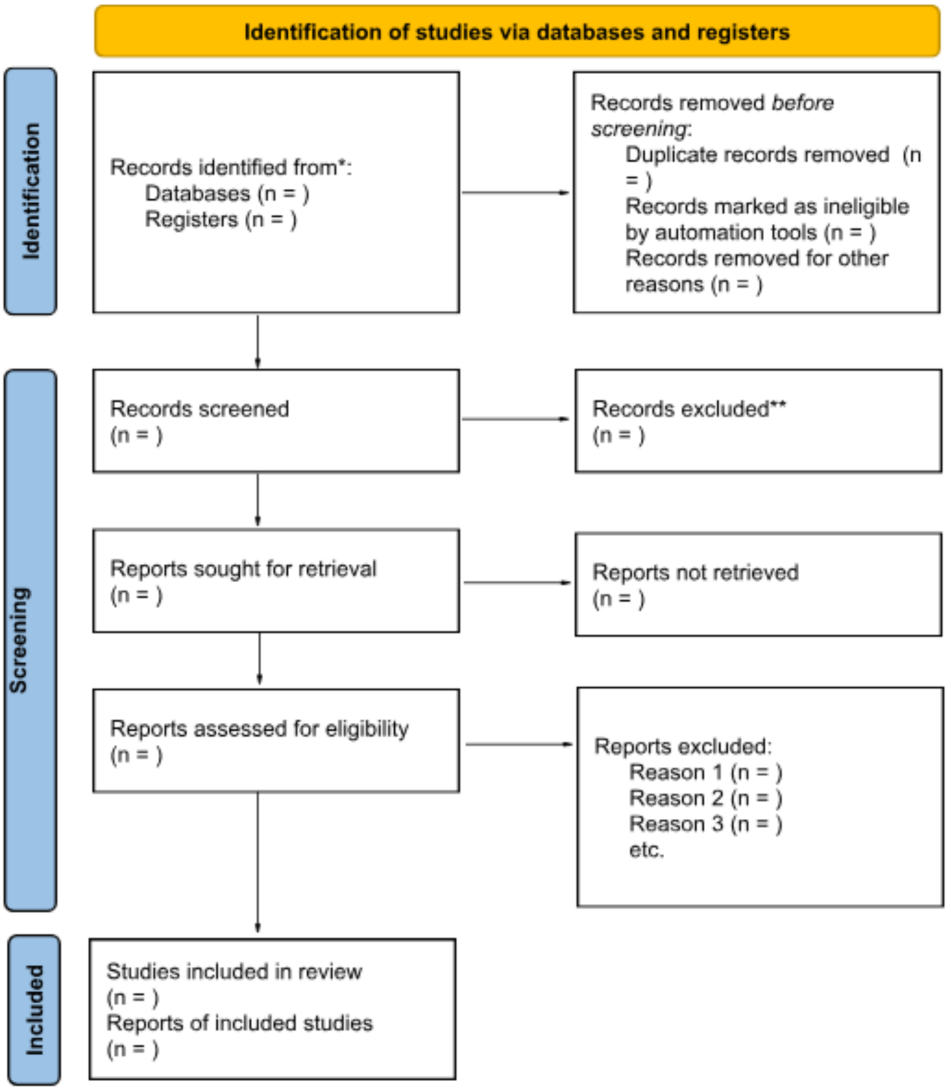
Etapa PRISMA e subetapa	**Quantidade de itens**
Identificação: itens recuperados pela união das *strings* S1 a S6	8.669
Identificação: itens removidos por duplicidade por DOI ou identificador	1.586
Triagem: itens em triagem após deduplicação	7.083
Triagem: itens aceitos por filtros básicos (tipo, idioma, ano)	6.590
Triagem: itens adicionados pelo conjunto-semente de calibração	8
Triagem: itens aceitos por título e resumo	2.536
Elegibilidade: itens selecionados para leitura completa	165
Inclusão: itens classificados nas oito dimensões D1 a D8	em curso

Fonte: O autor

O relato do processo de seleção adota o modelo de fluxograma do PRISMA 2020, reproduzido na Figura 5, que organiza o fluxo de itens em três blocos sucessivos, denominados *Identification*, *Screening* e *Included*, e reserva em cada etapa um campo para a contagem de registros e para os motivos de remoção. A figura apresenta o arcabouço em sua forma de modelo, com os campos de contagem ainda não preenchidos, dado que

a versão povoada com os números de cada etapa, a contagem da etapa de Inclusão e o detalhamento dos motivos de exclusão (CE1 a CE7) por etapa será apresentada na monografia final, após a classificação integral dos 165 estudos primários selecionados. Os quantitativos já apurados até a etapa atual constam da Tabela 2.

Figura 5 – Modelo de fluxograma PRISMA 2020 para o relato do processo de seleção de estudos.



Fonte: Page *et al.* (2021)

São reconhecidas, ainda, cinco ameaças à validade do mapeamento. A primeira é a cobertura incompleta inerente ao conjunto de bases escolhidas, mitigada parcialmente

pela amplitude do agregador empregado na execução inicial. A segunda decorre dos vieses de seleção e extração, mitigada pela documentação completa dos critérios e pela validação contra o conjunto-semente nomeado. A terceira reside em eventual subinclusividade ou superinclusividade das *strings* de busca, mitigada pelas rodadas de calibração registradas na memória técnica do trabalho. A quarta é a datação dos resultados, registrada explicitamente em cada execução de busca. A quinta é a cobertura de literatura cinza, restrita conforme o critério CE6.

3.2 Especificação formal e validação experimental

A linha experimental reúne duas atividades acopladas. A primeira especifica formalmente o núcleo do protocolo de estado de sessão como um sistema de transições, com canal de replicação assíncrona abstrato, e verifica os invariantes declarados por *model checking*. A segunda implementa e valida experimentalmente os mesmos invariantes em um *cluster* Infinispan, com injeção parametrizada de falhas e instrumentação do programa de carga para detectar violações e medir percentis de latência. O recorte aplicado a ambas as atividades é derivado dos achados da linha de mapeamento. No escopo correspondente ao Trabalho de Conclusão de Curso I, esta linha cobre a preparação do ambiente, a implementação do programa de carga, a especificação formal inicial do modelo (ainda sem falhas explicitamente modeladas) e a execução de *baseline* experimental parcial sob um subconjunto dos cenários de falha previstos. Os resultados parciais são reportados no Capítulo 4.

Escolha do ferramental de verificação

A verificação formal do núcleo do protocolo admite ferramentas de naturezas distintas, e a seleção pondera expressividade, custo de prova e ajuste ao recorte do trabalho. A análise relacional limitada por escopo com Alloy (JACKSON, 2012) verifica invariantes estruturais sobre relações, mas opera sob limite de escopo e perde força na representação de um canal assíncrono com muitos passos de execução. O *model checking* explícito de protocolos com SPIN e a linguagem Promela (HOLZMANN, 2004) é maduro para sistemas de processos comunicantes, porém opera em nível mais baixo e oferece menos expressividade para invariantes em teoria de conjuntos sobre os mapas que descrevem o

estado de sessão. Os assistentes de prova como Coq e Isabelle provam propriedades sem limite de estados, mas exigem prova manual interativa, esforço desproporcional ao recorte do protocolo e ao cronograma das duas etapas. As abordagens empíricas para a mesma garantia, como testes baseados em propriedades, engenharia do caos e verificação a posteriori do tipo Jepsen (BAILIS; KINGSBURY, 2014), validam o que o sistema executa, mas não certificam o que é possível no espaço de estados, e no presente trabalho já estão refletidas na vertente experimental descrita nesta seção.

A escolha recai sobre o conjunto TLA+ e TLC por quatro razões. A semântica de especificação de Lamport (2002) mapeia de forma direta os invariantes de segurança como propriedades de invariância, expressas pelo operador temporal $\Box P$, que afirma que P vale em todo estado da execução, e as propriedades de convergência sob quiescência declaradas na Seção 3.2. O canal de replicação assíncrona é modelado como multiconjunto de mensagens com entrega não determinística, o que reproduz reordenação e atraso sem premissas de sincronia. Há maturidade da linguagem e registro de aplicação industrial em sistemas de replicação (NEWCOMBE *et al.*, 2015; NEWCOMBE, 2014), o que reduz risco metodológico. Por fim, a cardinalidade reduzida do recorte (poucas identidades, sessões e réplicas, conforme o parâmetro T21 da Seção 3.2) mantém a verificação exaustiva do TLC tratável. Dentro do mesmo ecossistema TLA+, o tradutor Apalache para verificação simbólica e o detector Must (ENEA *et al.*, 2024) para protocolos assíncronos com falhas figuram como caminhos de escalabilidade reservados à etapa subsequente, quando o espaço de estados crescer com a inclusão explícita de falhas.

Operações modeladas

O protocolo de estado de sessão compreende sete operações sobre um par de mapas globais: o mapa $\sigma : \mathscr{S} \to \{\mathsf{Valid}, \mathsf{Invalid}(\mathsf{reason})\}$, que associa a cada sessão s seu estado lógico, e o mapa $\beta : \mathscr{U} \to \mathbb{N} \times \{\mathsf{Free}, \mathsf{Blocked}\}$, que associa a cada identidade u o contador de tentativas e a marca de bloqueio. As operações ficam definidas a seguir, com precondições e efeitos.

O1, *login*. Cria nova sessão s para uma identidade u após validação de credenciais. Precondição: $\beta(u).\mathsf{marca} = \mathsf{Free}$. Efeito: $\sigma' = \sigma \cup \{s \mapsto \mathsf{Valid}\}$, $\beta'(u).\mathsf{counter} = 0$ (reposição implícita por sucesso). Caso a precondição falhe, a operação retorna o código de erro $\mathsf{IdentityBlocked}$ sem alterar o estado.

O2, validação. Operação de leitura que consulta o estado lógico atual de uma ses-

são. Sem efeitos sobre σ ou β; retorna $\sigma(s)$ se $s \in \mathsf{dom}(\sigma)$ e $\mathsf{NotFound}$ caso contrário.

O3, *logout*. Marca a sessão s como inválida com motivo registrado. Efeito: $\sigma' = \sigma[s \mapsto \mathsf{Invalid}(\mathsf{logout})]$. A operação é idempotente: aplicar O3 duas vezes sobre a mesma sessão produz o mesmo estado.

O4, incremento de tentativas falhas. Adiciona uma unidade ao contador da identidade u após autenticação falha. Efeito: $\beta'(u).\mathsf{counter} = \beta(u).\mathsf{counter} + 1$; se $\beta'(u).\mathsf{counter} \geq \theta$, em que θ é o limiar de bloqueio (parâmetro do sistema), então $\beta'(u).\mathsf{marca} = \mathsf{Blocked}$ e $\sigma' = \sigma[s \mapsto \mathsf{Invalid}(\mathsf{blocked})]$ para toda sessão s de u.

O5, reposição de tentativas. Zera o contador da identidade u após sucesso de *login* dentro da janela. Efeito: $\beta'(u).\mathsf{counter} = 0$, sem alterar a marca de bloqueio.

O6, bloqueio administrativo. Marca a identidade u como bloqueada e invalida toda sessão ativa associada, independentemente do contador. Efeito: $\beta'(u).\mathsf{marca} = \mathsf{Blocked}$ e $\sigma' = \sigma[s \mapsto \mathsf{Invalid}(\mathsf{admin_block})]$ para toda sessão s de u.

O7, desbloqueio. Remove a marca de bloqueio e zera o contador. Efeito: $\beta'(u).\mathsf{marca} = \mathsf{Free}$, $\beta'(u).\mathsf{counter} = 0$. Não recria sessões previamente invalidadas.

Cada operação registra marcas de tempo de início e fim com precisão de nanossegundos, o servidor de conexão do cliente, o código de retorno e a chave, e alimenta as métricas de latência e a detecção de violações descritas na Seção 3.2.

Modelo de falhas

Quatro classes de falha são consideradas no escopo do Trabalho de Conclusão de Curso I, cada uma parametrizada para reprodução experimental controlada. As classes se enquadram na taxonomia de Avižienis *et al.* (2004).

F1, *crash* silencioso de nó. Um nó deixa de responder sem aviso prévio. Parâmetros: identificador do nó alvo, instante de início t_{crash} medido em segundos desde o início da execução, e duração da indisponibilidade Δ_{crash} (com $\Delta_{\mathsf{crash}} = \infty$ correspondendo a falha permanente). A reativação do nó, quando ocorre, expõe o cenário de recuperação (*crash-recovery*) e ativa o procedimento de *state transfer*. A pilha de comunicação do *cluster* tem dois detectores de falha: o FD_ALL3, que suspeita de um nó depois de um intervalo sem mensagens de presença (T16), e o FD_SOCK2, que suspeita assim que a conexão TCP com o nó se fecha. Para que a queda seja silenciosa e detectada pelo FD_ALL3, a interface de rede do nó é desativada antes que o processo seja encerrado (Seção 3.2).

F2, partição de rede entre subconjuntos do *cluster*. O conjunto de nós é dividido em dois ou mais subconjuntos $\{P_1, P_2, \ldots\}$ tais que a comunicação entre subconjuntos distintos é descartada por um intervalo $[t_{\mathsf{part}}, t_{\mathsf{part}} + \Delta_{\mathsf{part}}]$. Dentro de cada subconjunto, a comunicação permanece normal. Esta classe fica reservada para o Trabalho de Conclusão de Curso II (Decisão 011, ponto 11), com F1 e F3 cobrindo o escopo experimental do Trabalho de Conclusão de Curso I. No experimento, um nó é isolado dos outros dois (minoria de um nó), e o fim do intervalo dispara a fusão das partições.

F3, introdução de atraso e *jitter* no canal. O atraso aplicado a cada mensagem é amostrado de uma distribuição com média μ e desvio padrão σ (parâmetros do cenário), com aplicação via `tc-netem` (LUDOVICI; PFEIFER, 2011) no *network namespace* de cada *container*. A distribuição padrão adotada é lognormal, escolhida por aproximar a cauda longa observada empiricamente em redes reais; alternativas exponencial e uniforme ficam configuráveis para análise de sensibilidade.

F4, mudança de topologia que induz *state transfer*. A adição ou remoção controlada de um nó durante carga ativa o rebalanceamento do Infinispan e dispara transferência de segmentos entre nós, com janela de inconsistência transitória durante a propagação. Parâmetros: tipo de evento (entrada ou saída de nó), nó alvo, instante. F4 fica como ampliação opcional no Trabalho de Conclusão de Curso I, com execução completa no Trabalho de Conclusão de Curso II.

Falhas bizantinas (mensagens corrompidas ou maliciosas) estão fora de escopo, dado que o modelo de falhas deste trabalho supõe canal honesto (CE6 do protocolo do mapeamento, Seção 3.1). Cenários multi-site (*cross-site* replication) ficam reservados para a etapa subsequente do trabalho.

Invariantes alvo

O conjunto inicial de invariantes verificados no Trabalho de Conclusão de Curso I tem seis elementos, declarados sobre os mapas σ e β definidos na Seção 3.2, sobre o conjunto de réplicas $\mathscr{R}$, e sobre a sequência temporal de estados globais $\{S_t\}_{t \geq 0}$. Os quatro primeiros são propriedades de segurança e devem valer em todo estado alcançável. Os dois últimos são propriedades de convergência, próprias da consistência eventual, e valem sob hipótese de quiescência, ou seja, no estado em que não há operações em andamento e, cessadas as operações, as réplicas convergem para o mesmo valor.

I1, irreversibilidade da invalidação por *logout*. A partir do momento em que

uma sessão atinge o estado inválido com motivo *logout*, nenhum estado futuro pode ter essa mesma sessão como válida:

$$\forall s \in \mathscr{S}, \forall t \geq 0 : \quad \sigma_t(s) = \mathsf{Invalid}(\mathsf{logout}) \Rightarrow \forall t' \geq t : \sigma_{t'}(s) \neq \mathsf{Valid}.$$

I2, consistência de bloqueio. Enquanto uma identidade estiver marcada como bloqueada, nenhuma sessão associada pode ser validada como ativa:

$$\forall u \in \mathscr{U}, \forall t : \quad \beta_t(u).\mathsf{marca} = \mathsf{Blocked} \Rightarrow \forall s \in \mathsf{sessions}(u) : \sigma_t(s) \neq \mathsf{Valid}.$$

I3, monotonicidade local do contador. Em cada réplica $r \in \mathscr{R}$, na ausência de reposição (O5) e desbloqueio (O7) entre dois instantes $t_1 < t_2$, o contador é não-decrescente:

$$\forall u, \forall r, \forall t_1 < t_2 : \quad \beta_{r,t_1}(u).\mathsf{counter} \leq \beta_{r,t_2}(u).\mathsf{counter},$$

restrito ao intervalo em que nenhuma operação O5 ou O7 sobre u é aplicada em r.

I4, unicidade do motivo de invalidação. Uma vez fixado o motivo de invalidação de uma sessão, ele não é reescrito por outra causa:

$$\forall s, \forall t : \quad \sigma_t(s) = \mathsf{Invalid}(m) \wedge m \neq \bot \Rightarrow \forall t' > t : \sigma_{t'}(s) = \mathsf{Invalid}(m).$$

I5, convergência do estado lógico (sob quiescência). Após o cessar das operações e a entrega de todas as mensagens pendentes pelos canais (estado de quiescência Q), todas as réplicas observam o mesmo estado lógico para cada sessão:

$$\mathsf{quiescent}(Q) \Rightarrow \forall s, \forall r_1, r_2 \in \mathscr{R} : \sigma_{r_1}(s) = \sigma_{r_2}(s).$$

I6, convergência do contador (sob quiescência). Sob a mesma hipótese, o contador converge entre réplicas:

$$\mathsf{quiescent}(Q) \Rightarrow \forall u, \forall r_1, r_2 \in \mathscr{R} : \beta_{r_1}(u).\mathsf{counter} = \beta_{r_2}(u).\mathsf{counter}.$$

Os invariantes I3, I5 e I6 reconhecem explicitamente a separação entre estado local de cada réplica e estado lógico global; I5 e I6 derivam diretamente da Consistência Eventual Forte de Shapiro *et al.* (2011) aplicada aos tipos de dados subjacentes (registrador para σ, contador para $\beta.\mathsf{counter}$).

Métricas e critérios

A avaliação adota cinco métricas primárias coletadas por execução de cenário.

Latência por operação. Coletada como distribuição empírica, com cálculo de percentis p50, p95, p99 e p99,9 conforme prática consolidada para sistemas com cauda longa (DEAN; BARROSO, 2013; Google, 2016). A coleta usa marcas de tempo de início e fim com precisão de nanossegundos, registradas pelo programa de carga, com agregação em janelas configuráveis para análise temporal.

Taxa de violação por invariante. Contabilizada como número de eventos detectados por unidade de tempo e por operação. A detecção segue dois caminhos (Seção 4.1): verificação imediata após cada operação para violações observáveis localmente (I1 a I4), e auditoria periódica em janelas de quiescência para violações de convergência (I5 e I6). Cada violação registrada inclui o invariante atingido, o estado relevante das réplicas e o histórico de operações que o produziu.

Tempo de recuperação. Mede o intervalo $\Delta_{\text{rec}} = t_{\text{base}} - t_{\text{fim-falha}}$ entre o fim da injeção de uma falha (instante $t_{\text{fim-falha}}$) e o retorno da taxa de erro à linha de base (instante t_{base}, definido como o primeiro instante em que a taxa de erro permanece dentro de uma margem ε da pré-falha por uma janela de W segundos).

Tempo de convergência. Mede o intervalo Δ_{conv} entre o fim da falha e o desaparecimento de divergências entre réplicas (detectado pela auditoria periódica reportando $\sigma_{r_1} = \sigma_{r_2}$ e $\beta_{r_1} = \beta_{r_2}$ para todo par (r_1, r_2) de réplicas).

Taxa de erro. Contabiliza as requisições que retornam código distinto de sucesso, sem implicar necessariamente violação de invariante. Inclui erros transitórios de comunicação, recusa por bloqueio, *timeouts* e códigos específicos do cliente Hot Rod.

Métricas secundárias, coletadas via API e logs do Infinispan, incluem uso de CPU por nó, memória do *heap*, número de eventos de rebalanceamento, número de mensagens RPC trocadas e tamanho da fila de propagação assíncrona (Infinispan Project, 2026d). A exposição segue o formato OpenMetrics (OpenMetrics Project, 2024), com integração via JMX (Oracle, 2023), e admite coleta por agente Prometheus opcional (Prometheus Authors, 2026).

Os critérios de comparação entre o *baseline* estático e a configuração com o controlador heurístico, planejados para o Trabalho de Conclusão de Curso II, exigem três condições simultâneas. A primeira é redução estatisticamente significativa da taxa de violação, avaliada pelo teste não paramétrico de Mann-Whitney sobre as amostras de cada

cenário ($\alpha = 0{,}05$). A segunda é manutenção das latências p95 e p99 dentro de margem aceitável, definida como degradação máxima de 10% em relação ao *baseline*, com intervalo de confiança de 95%. A terceira é a auditabilidade das decisões do controlador: cada ação aplicada deve ser rastreável até a regra que a motivou, com registro do gatilho (observável + limiar), do estado anterior e do estado seguinte do controlador. Os três critérios devem ser cumpridos conjuntamente para que a configuração com controlador seja considerada melhoria sobre o *baseline*.

Parâmetros experimentais e fundamentação na literatura

A definição dos parâmetros operacionais do *baseline* segue três princípios. O primeiro é a compatibilidade com a literatura de avaliação de *key-value stores* e *data grids* sob carga com cauda longa, exemplificada por Jaiman, BEN MOKHTAR e Rivière (2020) no caso de Cassandra com YCSB e por DeCandia *et al.* (2007) no caso de Dynamo. O segundo é a aderência à configuração recomendada na documentação oficial do Infinispan (Infinispan Project, 2026c, 2026d, 2026e), de modo que o experimento meça o comportamento do sistema em configuração próxima da praticada em produção. O terceiro é a proporcionalidade ao escopo do Trabalho de Conclusão de Curso I (Decisão de recorte registrada na Seção 3.2), que delimita o tamanho do *cluster*, o conjunto de cenários de falha e o volume de operações para que o tempo de execução em *Sem* 9 e 10 fique dentro da janela disponível.

Tabela 3 – Parâmetros do *baseline* experimental: faixa observada, fonte e valor adotado

ID	Parâmetro	Faixa observada	Fonte	Valor adotado	Justificativa do recorte
T1	Modo de replicação	DIST_SYNC, DIST_ASYNC, REPL_SYNC, REPL_ASYNC	Infinispan Project (2026c)	DIST_ASYNC (alvo) e DIST_SYNC (controle)	objeto de estudo é replicação assíncrona
T2	Número de nós	3 mínimo; 16 em estudos comparáveis	Jaiman, BEN MOKHTAR e Rivière (2020), Yuan *et al.* (2014)	3	escopo TCC-I; tolerância mínima a 1 falha; falhas de produção reproduzíveis com até 3 nós
T3	`numOwners`	2 (padrão); > 2 aumenta durabilidade	Infinispan Project (2026e)	2	padrão da documentação
T4	`numSegments`	256 (padrão); $20 \times N$ para $N > 20$	Infinispan Project (2026e)	256	dentro da faixa padrão para $N = 3$

continua na próxima página

Tabela 3 – continuação

ID	Parâmetro	Faixa observada	Fonte	Valor adotado	Justificativa do recorte
T5	Tamanho do *payload*	1 KB típico de sessão; 1 KB–2 MB em estudos de cauda	Jaiman, BEN MOKHTAR e Rivière (2020)	1 KB fixo	representativo de objeto de sessão
T6	Distribuição de chaves	Zipfian $\rho = 0{,}99$ (YCSB); uniforme; *latest*	Cooper *et al.* (2010), Jaiman, BEN MOKHTAR e Rivière (2020)	Zipfian $\rho = 0{,}99$	expõe a cauda; comparável com a literatura
T7	Mistura de operações	50/50, 95/5, 100R, leitura-modifica-escrita	Cooper *et al.* (2010), Jaiman, BEN MOKHTAR e Rivière (2020)	50/50 (cenário S1) + 95/5 (cenário S2)	50/50 modela *session store* de escrita ativa; 95/5 isola comportamento de validação
T8	Cardinalidade de chaves	50 000 em *benchmarks* sintéticos; 20 M em estudos de cauda	Jaiman, BEN MOKHTAR e Rivière (2020)	100 000	forma cauda Zipfian; cabe em *heap* de poucos GB por nó
T9	Operações por execução	10^6–10^7	Jaiman, BEN MOKHTAR e Rivière (2020)	10^6	viabiliza p99,9 estável ($n = 1\,000$ amostras na cauda)
T10	Repetições por cenário	5 em estudos comparáveis	Jaiman, BEN MOKHTAR e Rivière (2020)	30	conservador para testes não paramétricos
T11	*Warm-up*	fases de *load* e *run* separadas em YCSB	Cooper *et al.* (2010)	60 s ou 10 % do total; descarte das primeiras 5 % das amostras	estabilização de JIT, alocadores e cache
T12	Carga (alvo)	75 % do pico medido	Jaiman, BEN MOKHTAR e Rivière (2020)	75 % do pico do piloto sem falhas	evita saturação; expõe variabilidade
T13	Percentis reportados	p95 e p99 (mínimo da indústria); p99,9 em *key-value stores*	Dean e Barroso (2013), DeCandia *et al.* (2007), Google (2016)	p50, p95, p99 obrigatórios; p99,9 quando $n \geq 10^6$	padrão da literatura; p99,9 é estatisticamente fiável com o volume adotado
T14	Duração por cenário	não reportada (estudos fixam número de operações)	Jaiman, BEN MOKHTAR e Rivière (2020)	10 min sem falha; 30 min com falha	inclui a janela de detecção (T16) e estabilização
T15	MTU	1500 (padrão); até 9000 (*jumbo*)	Infinispan Project (2026e)	1500	infraestrutura em *containers* sem *jumbo* controlado
T16	Detecção de falha (FD_ALL3)	*heartbeat* 8000 ms, *timeout* 40000 ms, padrão da pilha JGroups do servidor (conferido na configuração do *cluster*)	O autor	padrões mantidos	reproduz produção; define duração mínima de F1
T17	Falha F1 (*crash*)	nó silencioso; reentrada com *state transfer*	Infinispan Project (2026c)	1 nó; $\Delta_{\text{crash}} = 60$ s; reentrada ativa	maior que o *timeout* de T16; ativa convergência (I5, I6)

continua na próxima página

Tabela 3 – continuação

ID	Parâmetro	Faixa observada	Fonte	Valor adotado	Justificativa do recorte
T18	Falha F3 (atraso/*jitter*)	atraso com média, desvio e distribuição configuráveis	Ludovici e Pfeifer (2011)	+50 ms no p99 via `tc-netem`	valor de engenharia, sem fonte empírica; calibrável
T19	Métricas coletadas	latência, vazão, *hit ratio*, RPC, *state transfer*	Infinispan Project (2026d)	M1–M7 (Seção 3.2)	mapeamento 1:1 com os medidores expostos
T20	Endpoint de métricas	JMX; OpenMetrics	Infinispan Project (2026d)	OpenMetrics; Prometheus opcional	padrão atual da documentação
T21	Espaço de estados (TLC)	cardinalidades pequenas (2–3 por *sort*) em verificações industriais; falhas de produção com até 3 nós	Yu, Manolios e Lamport (1999), Newcombe (2014), Yuan *et al.* (2014)	2–3 identidades, sessões e réplicas	expõe entrelaçamento concorrente; *model checking* incremental
T22	Análise estatística	Mann-Whitney/Wilcoxon; *bootstrap* para IC de percentis	Mann e Whitney (1947), Efron e Tibshirani (1993)	Mann-Whitney por percentil; *bootstrap* com 10 000 reamostragens para IC 95 %	distribuições assimétricas de latência

Fonte: O autor, com base em Cooper *et al.* (2010), DeCandia *et al.* (2007), Dean e Barroso (2013), Google (2016), Infinispan Project (2026c), Infinispan Project (2026d), Infinispan Project (2026e), Jaiman, BEN MOKHTAR e Rivière (2020), Yu, Manolios e Lamport (1999), Newcombe (2014), Yuan *et al.* (2014), Ludovici e Pfeifer (2011), Mann e Whitney (1947) e Efron e Tibshirani (1993).

A Tabela 3 sintetiza a análise, com a faixa observada na literatura ou na documentação oficial, a fonte principal e o valor adotado, e a justificativa do recorte. A tabela explicita três escolhas que merecem comentário. A combinação 10^6 operações por execução (T9) com 30 repetições por cenário (T10) decorre da necessidade de estimar p99 e, quando aplicável, p99,9 sob incerteza controlada: com 10^6 amostras, o p99,9 dispõe de 10^3 observações na cauda, base para um intervalo de confiança *bootstrap* (EFRON; TIBSHIRANI, 1993). Com 30 repetições, comparações entre cenários por percentil podem ser feitas pelo teste de Mann e Whitney (1947), sem hipótese de normalidade, condição necessária dada a assimetria das distribuições de latência observadas em sistemas com cauda longa. A escolha de Zipfian $\rho = 0{,}99$ (T6) e mistura 50/50 (T7) reproduz o setup de Jaiman, BEN MOKHTAR e Rivière (2020) e a definição de *Workload A* do YCSB, descrito por Cooper *et al.* (2010) como "*session store recording recent actions*", e portanto põe o experimento em registro comparável com o estudo de cauda em Cassandra. A escolha dos valores padrão de detecção de falha (T16) deixa o cenário F1 (T17) com duração mínima alinhada ao *timeout* de 40 s, condição necessária para que o evento de falha seja registrado pelo *cluster* antes do encerramento da janela experimental.

Carga sintética dos cenários S1 e S2

Os cenários S1 e S2 da linha T7 da Tabela 3 sorteiam, a cada iteração de cada *thread* do programa de carga, uma operação com pesos fixos. Em S1, os pesos são 50 % de validação (O2), 25 % de *login* (O1), 10 % de *logout* (O3), 10 % de incremento de tentativas falhas (O4) e 5 % de reposição de tentativas (O5). Em S2, os pesos na mesma ordem são 95 %, 3 %, 1 %, 1 % e 0 %. O *login* abre uma tríade: quando ele é aceito, a mesma *thread* valida e encerra a sessão recém-criada, e os três eventos entram no registro. As validações e os *logouts* sorteados fora da tríade, chamados aqui de isolados, recaem sobre chaves de sessão escolhidas pela distribuição Zipfian de T6, no universo de chaves de T8.

Antes do aquecimento de cada execução, o programa esvazia os dois *caches* do modo de replicação em uso, confere que ambos ficaram vazios e encerra a execução com código de erro próprio se isso não ocorrer. Cada *thread* t usa, no *login*, oito identidades próprias, `user-`t`-`k com $k = 0, \ldots, 7$, tomadas em rodízio, que nunca recebem O4 nem O5. As operações O4 e O5 atuam sobre uma identidade separada por *thread*, `lim-`t, de modo que o bloqueio por limite continua exercitado sem atingir as identidades de *login*. As chaves lidas pelas operações isoladas existem no *cache*, conforme o esquema de gerações descrito a seguir. O rodízio não consome os geradores da mistura nem o de chaves.

A versão da mistura adotada nas medições organiza as chaves de sessão em *slots*. O sorteio Zipfian define o *slot* k, e um vetor compartilhado entre as *threads* guarda a geração corrente g_k de cada *slot*. Antes do aquecimento, uma fase de carga grava, para cada um dos 10^5 *slots*, a geração 0 como sessão já invalidada por *logout* e a geração 1 como sessão válida, o que soma 2×10^5 entradas e fixa $g_k = 1$. A sessão do *slot* k pertence à identidade `user-`$((k-1) \bmod T)$`-`$(\lfloor (k-1)/T \rfloor \bmod 8)$, em que T é o número de *threads*, a mesma em todas as gerações do *slot*. A validação isolada lê a geração corrente, exceto quando um sorteio de Bernoulli independente, com probabilidade $p = 0{,}05$, a desvia para a geração anterior. Esse desvio representa um cliente que ainda apresenta um *token* antigo. O valor de p é uma premissa de engenharia, declarada como tal, e não a medida de um serviço real. O *logout* isolado toma uma trava da faixa do *slot* (k mod 1024), invalida a geração corrente, que é a operação medida e registrada, grava como válida a geração $g_k + 1$ e só então avança g_k. Essa gravação de reposição não emite evento, não entra na latência medida e não passa pelo auditor. Se o *logout* retornar erro,

não há reposição nem avanço. Se a reposição falhar, a geração não avança, e as validações seguem sobre a sessão já invalidada até o próximo *logout* do *slot*.

O desenho tem três consequências declaradas. A fração de respostas Invalid entre as validações isoladas fica estacionária desde o primeiro evento, próxima de p acrescida da corrida entre uma validação e um *logout* do mesmo *slot*. A reposição acrescenta ao *cluster* uma escrita por *logout* isolado, ausente do registro de eventos, em número próximo de 10 % das operações em S1 e de 1 % em S2. A trava serializa no cliente os *logouts* isolados de uma mesma faixa de *slot*, que portanto não concorrem entre si, e a espera pela trava fica fora da latência medida. Os critérios de estacionariedade foram fixados antes de qualquer execução dessa versão da mistura. A fração Invalid entre as validações isoladas pode diferir no máximo 1,0 ponto percentual entre o primeiro e o terceiro terço da janela de medição (critério C1) e deve ficar no intervalo $[4{,}5\,\%;\ 6{,}5\,\%]$ (C2). Com n validações isoladas e k leituras de *token* antigo, a execução deve satisfazer $|k/n - 0{,}05| \leq 3{,}2905\sqrt{0{,}05 \cdot 0{,}95/n}$ (C3), região de aceitação do teste de $H_0\colon\ p = 0{,}05$ ao nível de 0,1 % pela aproximação normal da binomial, aplicada somente quando $n \cdot 0{,}05 \cdot 0{,}95 \geq 100$, limite adotado por cautela diante da cobertura errática dessa aproximação (BROWN; CAI; DASGUPTA, 2001). As contagens de C3 lidas na saída do programa e no registro de eventos podem diferir em até quatro eventos, o número de *threads*, por causa da fronteira da janela de medição. Execução fora dessas tolerâncias é recusada.

O *logout* e a cascata de invalidação do bloqueio administrativo (O6) invalidam sessões por escrita condicional à versão lida. O programa lê a sessão com sua versão e a substitui pela versão invalidada somente se a versão não tiver mudado; em caso de conflito, relê a sessão e tenta de novo, até três tentativas. A primeira invalidação vence, e o motivo registrado não muda depois. Uma invalidação que perde a escrita condicional e relê a sessão já invalidada é contada como invalidação perdida e não altera o estado. Se a sessão continuar válida, com versão alterada, depois da terceira tentativa, a operação termina com o código `ERROR_TRANSPORT`, e um contador próprio de conflitos esgotados separa esse caso dos erros de transporte na análise. Os *caches* permanecem não transacionais. O servidor emite uma vez por processo o aviso `ISPN006010`, segundo o qual operações condicionais devem ser usadas com *caches* transacionais porque, de outro modo, "*data inconsistency issues could arise under failure situations*". O guia do cliente Hot Rod faz a mesma recomendação, para que no *failover* as operações condicionais não fiquem parcialmente aplicadas, com entradas conflitantes em nós diferentes (Infinispan Project, 2026g), e o guia de configuração adverte que a divisão do *cluster* em partições

pode causar perda ou inconsistência de dados qualquer que seja a estratégia de tratamento ou a política de *merge* (Infinispan Project, 2026a). A configuração não transacional foi mantida porque a adoção de *caches* transacionais mudaria o sistema medido. Por isso, uma violação de I4 ou uma invalidação perdida observada sob falha (queda de nó, partição ou fusão de partições) é tratada na análise como efeito do *grid*, e não da aplicação.

O *login* relê a identidade depois de gravar a sessão. Se a releitura encontrar a identidade bloqueada, o próprio *login* invalida a sessão recém-gravada com o motivo de bloqueio administrativo, pelo mesmo caminho condicional da cascata, e retorna recusa por bloqueio. Essa dupla checagem fecha, em `DIST_SYNC`, a corrida entre um *login* e um bloqueio concorrente da mesma identidade: o *login* escreve a sessão e lê a identidade, o bloqueio escreve a identidade e lê as sessões, e, com operações linearizáveis, ao menos um dos dois observa a escrita do outro. Em `DIST_ASYNC`, a releitura pode encontrar uma réplica atrasada, e o que restar dessa corrida é medida do *grid*. A releitura custa uma leitura a mais por *login* aceito, incluída na latência medida, e vale em todos os cenários, para que o *login* meça a mesma operação em todos eles.

O auditor aplica mais três regras à contagem de violações. Uma validação que lê sessão válida sob identidade bloqueada é classificada pelo instante de início da leitura da sessão, comparado com dois marcos do bloqueio (a confirmação da identidade bloqueada e o fim da cascata de invalidação). A leitura anterior à confirmação conta como concorrente, apenas para diagnóstico; a leitura entre a confirmação e o fim da cascata recebe o rótulo de janela de cascata; a leitura sob cascata interrompida por erro recebe o rótulo de cascata incompleta; e somente a leitura posterior ao fim da cascata é registrada como I2. Dentro de uma mesma operação, cada par formado por invariante e sessão é registrado no máximo uma vez para I1 e I4. Um bloqueio cuja cascata esgota as tentativas termina em `ERROR_TRANSPORT` com a cascata parcial, e a contagem de invalidações perdidas inclui apenas a invalidação que perdeu a escrita condicional e releu a sessão invalidada. As tabelas de violação contam I1, I4 e I2 a partir do arquivo de violações de cada execução, e uma I2 só entra como violação quando a análise do rastro do cliente a classifica como real. Os rótulos de janela e de cascata incompleta e os contadores de leituras concorrentes são reportados à parte. A leitura de sessão válida sob identidade bloqueada sem marco de bloqueio deve ser nula em `DIST_SYNC`. Em `DIST_ASYNC`, uma leitura atrasada da identidade depois do desbloqueio produz esse caso por efeito do *grid*, e a contagem é reportada sem servir de critério.

O registro de eventos distingue, na validação, as respostas `VALID`, `INVALID` e

`NOT_FOUND`, e, no *logout*, a chave ausente (`NOT_FOUND`) da invalidação concluída. Ao fim de cada execução, um verificador de sanidade lê esse registro, conta os desfechos por operação na janela de medição e emite veredito sobre a coerência entre os desfechos observados e a mistura declarada. O veredito é desfavorável se não houver *login*, se a fração de *logins* aceitos entre aceitos e recusados por bloqueio ficar abaixo de 0,95, se nenhuma validação retornar `VALID`, se mais de 5 % das validações encontrarem a chave ausente, ou se a fração de alguma operação sorteada se afastar mais de 2,0 pontos percentuais do peso nominal do cenário. Esses limiares são fixados no programa de carga antes do piloto e não são ajustados depois de observados os dados. Como a campanha não registra marcadores de fase, as regras se aplicam a todos os eventos da janela de medição, inclusive aos do intervalo sob falha.

O tipo de cada operação, a chave sorteada e a decisão de ler o *token* antigo vêm de três geradores `java.util.Random` próprios de cada *thread*, derivados da semente da execução, do índice t da *thread* e da repetição r pela expressão $s = \mathit{semente} + t + 1\,000\,003 \cdot r$. O gerador de chaves usa s diretamente. Os outros dois combinam s com um sal próprio por ou exclusivo ($\oplus$) e aplicam a função de mistura de 64 bits do gerador SplitMix (STEELE JR.; LEA; FLOOD, 2014), na variante usada por `java.util.SplittableRandom` ($\mathsf{mix64}$), de modo que as três sequências sejam distintas: o gerador da mistura recebe a semente $\mathsf{mix64}(s \oplus \mathsf{SAL_MISTURA})$, e o do *token* antigo, $\mathsf{mix64}(s \oplus \mathsf{SAL_TOKEN})$. Duas execuções com a mesma semente produzem, em cada *thread*, a mesma sequência de intenções, isto é, os mesmos tipos de operação, *slots*, decisões de *token* antigo e identidades de *login*. A intercalação entre *threads*, as respostas do *grid*, as latências, o número de operações por *thread* e o trecho exato coberto pela janela de medição não são reprodutíveis, porque dependem do tempo. Com mais de uma *thread*, o registro de eventos de duas execuções com a mesma semente não coincide, e a reprodutibilidade vale por *thread*.

Cenário S3: simulação do ciclo de vida de sessões

Os cenários S1 e S2 sorteiam operações com pesos fixos e não contêm bloqueio administrativo nem afinidade entre cliente e nó, de modo que não exercitam a consistência de bloqueio (I2) nem a escrita feita em um nó isolado por partição. O cenário S3 simula uma aplicação web com os defeitos de uma aplicação real: o programa não repara estado, não reconcilia réplicas e não repete operações indefinidamente, e o experimento

registra o que o *data grid* entrega nessas condições. Três servidores de aplicação, cada um ligado a um único nó do *cluster*, atendem usuários virtuais fixados a eles, como sob um balanceador com afinidade; um quarto cliente faz o papel de console administrativo. Cada usuário de um primeiro conjunto de identidades faz *login*, valida a sessão a cada página com tempo de pensar aleatório e, ao fim, encerra a sessão ou a abandona, caso em que a aplicação a considera expirada por inatividade e a sessão permanece válida no *grid*. Depois de um *logout* confirmado, o usuário repete validações da sessão encerrada por alguns segundos, como quem usa o botão de voltar do navegador, o que exercita I1 e I4.

Um ator administrativo bloqueia, em intervalos aleatórios, uma identidade com sessão ativa e a desbloqueia depois de um tempo mínimo, o que exercita I2 com cascata de invalidação. Um segundo conjunto de identidades recebe tentativas falhas até o limite, com reposições ocasionais, e mantém o bloqueio por limite separado do administrativo. O cliente tem tempos-limite curtos e no máximo duas retentativas de aplicação, depois das quais a operação conta como falha percebida pelo usuário. As ações de cada usuário são agregadas em ações S3, com a latência percebida, que inclui as retentativas. Os tempos de pensar, as durações de sessão, os intervalos do administrador e os tamanhos dos conjuntos de identidades são premissas de modelagem, sem fonte empírica, e serão fixados no pré-registro da campanha que usar S3. O cenário S3 é o objeto da campanha com F2 e F4 sob carga e da avaliação do controlador (Seção 3.2), enquanto a nova medição de referência usa apenas S1 e S2.

Protocolo da nova medição de referência

A nova medição de referência (*rebaseline*) mede os dois modos de replicação da linha T1 na mesma campanha e com o mesmo *jar*: `DIST_SYNC` como controle e `DIST_ASYNC` como alvo. O protocolo é pré-registrado (NOSEK *et al.*, 2018) antes de qualquer execução e não admite ajuste depois de observados os dados. Uma mudança gera registro novo, que cita o anterior e explica o motivo. A matriz cruza os cenários S1 e S2, com a carga da Seção 3.2, os dois modos e três condições (sem falha, F1 e F3), em dez repetições de 300 s por célula, o que soma 120 execuções, sem o controlador. Cada execução usa quatro *threads* e um *cluster* de três nós com a imagem `quay.io/infinispan/server:15.0.21.Final`, recriado do zero antes da campanha, com quatro *caches*: um de sessões e um de contadores em cada modo. O ce-

nário S3 e as falhas F2 e F4 ficam fora da campanha.

A janela de medição de cada execução vai de +44 s a +300 s. O aquecimento ocupa os primeiros 30 s, valor de $\max(30\,\text{s},\ 10\,\% \cdot 300\,\text{s})$, e a política de aquecimento descarta ainda 5 % do tempo restante, ou seja, 14 s. As falhas são disparadas no nó `isn2` a +49 s do início do aquecimento, cinco segundos depois do início da medição, e a execução cujo disparo caia fora do intervalo de +48 s a +51 s é recusada. F1 desativa a interface `eth0` no *network namespace* de `isn2` e, em seguida, encerra o processo do nó com o sinal `SIGKILL` (`podman kill`), sem saída ordenada. Após 60 s, o *container* é reiniciado com `podman start`. O nó fica fora de cerca de +49 s a cerca de +109 s, intervalo maior que o *timeout* de 40 s do detector FD_ALL3 (T16). A desativação da interface antes do encerramento impede que o FD_SOCK2 perceba o fechamento das conexões e detecte a queda em poucos segundos. F3 aplica `tc-netem` na interface `eth0` de `isn2`, com atraso de 20 ms e desvio de 13 ms sob distribuição normal, por 251 s, até o fim da execução.

A ordem de execução segue blocos completos aleatorizados. Cada uma das dez rodadas executa as doze células uma vez, em ordem sorteada por gerador com semente fixada no registro, de modo que a deriva do ambiente ao longo da noite se distribua entre as células e que uma interrupção preserve rodadas completas. Uma rodada piloto com as doze células precede a campanha e serve para verificar o orquestrador, os critérios e os tempos; seus dados não entram em resultado. A execução de índice i usa a semente $50\,000 + 100 \cdot i$, e nenhum par formado pela semente da *thread* e pela repetição se repete na campanha, no piloto ou nas reposições. As execuções rodam em máquina virtual Linux sob Hyper-V, sem coleta de uso de CPU durante a campanha.

Cada execução passa por critérios de recusa fixados antes do piloto. Antes da carga, os três nós precisam estar saudáveis há pelo menos 60 s, sem rebalanceamento em curso e sem disciplina `tc-netem` residual, e os dois *caches* precisam começar vazios. Depois da execução, exigem-se término normal do programa de carga, veredito favorável do verificador de sanidade, carga inicial completa, os critérios C1 a C3 da mistura e a confirmação de que a falha ocorreu como previsto. Nas células com falha, C1 e C2 usam como denominador apenas as validações isoladas com resposta `VALID` ou `INVALID`, e os erros são reportados à parte. Nas células sem falha, exigem-se também ausência de validação com `NOT_FOUND`, ao menos 99 % de *logins* aceitos e ausência de reposições com erro, de invalidações perdidas e de conflitos esgotados. Nas células com falha, esses valores são apenas reportados. Em `DIST_SYNC` sem falha, qualquer registro de I1, I2

ou I4, leitura sem marco de bloqueio ou *login* revertido pela dupla checagem interrompe a campanha para investigação. Em `DIST_ASYNC`, essas contagens são objeto de medição e são reportadas. O aviso `ISPN006010` pode aparecer no máximo uma vez por processo de nó, e o total por nó não pode exceder o número de vezes em que o processo foi iniciado, regra que acomoda o reinício de `isn2` em F1. A campanha é interrompida automaticamente diante de duas recusas na mesma célula, de mais de seis recusas no total ou de falha de infraestrutura, e uma recusa isolada é reposta no fim da campanha com o próximo índice livre.

A unidade estatística é a execução. Para cada execução e operação, a análise calcula p50, p95 e p99 das latências registradas na janela de medição, pelo método do posto mais próximo, sobre os desfechos válidos de cada operação: *login* aceito, validação com `VALID` ou `INVALID`, *logout* concluído, incremento contabilizado e reposição de tentativas concluída. Os demais desfechos entram nas taxas de erro por código. O p99 só é calculado com ao menos 1 000 eventos válidos da operação na execução, e o p99,9 só com ao menos 10 000, sem contraste. Os contrastes comparam F1 e F3 com a condição sem falha, por cenário, modo e operação, e `DIST_ASYNC` com `DIST_SYNC`, por cenário, condição de falha e operação. A estimativa de cada contraste é a diferença entre as médias dos p99 por execução das duas células, com intervalo de confiança de 95 % por *bootstrap* percentil (EFRON; TIBSHIRANI, 1993) com 10 000 reamostragens de execuções, feitas com reposição e de forma independente em cada célula. O contraste só é estimado quando cada célula tem ao menos oito execuções aceitas. O teste de Mann e Whitney (1947) bilateral sobre os p99 por execução, com $\alpha = 0{,}05$, é secundário. Os intervalos são reportados sem correção para comparações múltiplas, com a contagem de comparações declarada, e nenhuma conclusão depende de uma comparação isolada no limite da significância.

Estimativa de energia

A máquina virtual Hyper-V que hospeda o *cluster* não expõe ao sistema convidado os contadores de energia do processador, de modo que a energia não pode ser medida, apenas estimada por modelo (KANSAL *et al.*, 2010). O método previsto lê, a cada segundo, o tempo de CPU acumulado e a memória de cada *container* Infinispan pela interface cgroup v2 (HEO, 2026), o tempo de CPU do processo do programa de carga, que inclui o controlador, e o uso total de CPU da máquina virtual, com o tempo tomado pelo hipervisor

(*steal*) registrado à parte. O relógio dessas leituras é o mesmo dos marcadores de falha, o que permite recortar o consumo por fase da execução. A métrica principal é medida, e não estimada: tempo de CPU por mil operações bem-sucedidas em S1 e S2 e por mil ações S3 sem falha percebida, acompanhado do retrabalho, contado em retentativas e operações com erro por ação útil.

A conversão em energia usa o modelo linear de Fan, Weber e Barroso (2007), com a potência ociosa e a potência máxima tomadas da especificação do processador do hospedeiro. A estimativa é reportada em duas parcelas: a dinâmica, proporcional ao tempo de CPU, e a total, que inclui a parcela ociosa, cujo peso Barroso e Hölzle (2007) destacam. A razão entre potência ociosa e máxima varia em $\{0{,}3;\ 0{,}5;\ 0{,}7\}$ como análise de sensibilidade. As conclusões são apenas relativas, entre os braços de uma comparação, e nunca em joules absolutos. O método tem limitações declaradas: o tempo de CPU virtual não equivale ao trabalho do núcleo físico; memória, disco e rede não entram na conta; e reduzir a carga reduz o consumo por definição, razão pela qual a métrica é normalizada por operação útil e a vazão útil tem margem de não inferioridade na avaliação do controlador. Uma execução com *steal* acima de 5 % do tempo fica fora da comparação de CPU.

Desenho da avaliação do controlador

O pré-registro da avaliação do controlador heurístico, com os valores finais, precede qualquer execução. A comparação ocorre em `DIST_ASYNC`, o modo alvo, com o cenário S3, e opõe dois braços que executam o mesmo código: no braço A, o controlador observa e decide sem atuar (modo *shadow*); no braço B, ele aplica as decisões (modo *active*). Assim, o custo de observação é o mesmo nos dois braços, e só a atuação difere. São quatro células: sem falha, para medir sobrecarga e acionamentos indevidos; F2 com o nó `isn3` isolado por 90 s; F4 com perda de 20 % (severidade N3); e F1 sobre `isn2`. Cada célula recebe oito blocos pareados, e cada bloco contém uma execução de cada braço com a mesma semente de carga, em ordem sorteada e com o *cluster* reiniciado entre execuções. Os três primeiros blocos funcionam como piloto e são aproveitados se o protocolo não mudar. Duas células em `DIST_SYNC` (F2 e F4), com quatro blocos cada, servem de referência, sem pretensão de teste, para indicar se o efeito depende do modo. O total é de cerca de 80 execuções, estimadas em 16 a 27 horas de máquina.

O desfecho primário de cada célula é o p99 da latência percebida das ações S3,

que inclui as retentativas, nas fases durante a falha e de recuperação, calculado por execução. Para cada bloco b, calcula-se $d_b = \ln(p99_B / p99_A)$, e a estimativa é a média dos d_b, reportada como razão. O intervalo de confiança primário de 95 % é o intervalo t sobre os d_b (STUDENT, 1908), com o *bootstrap* sobre blocos como análise de sensibilidade (EFRON; TIBSHIRANI, 1993). O teste é de permutação exata por troca de sinal dos d_b (FISHER, 1935, seção 21; GOOD, 2005), com $\alpha = 0{,}05$ por célula; com oito blocos, há $2^8 = 256$ arranjos, e o menor valor-p bilateral possível é $2/256 \approx 0{,}008$. Os desfechos secundários (p99 por operação, taxa de erro, falhas percebidas, violações de I1, I2 e I4, tempo de convergência e divergência persistente de I5 e I6, tempo de CPU e energia estimada) seguem o mesmo procedimento, com correção de Holm (1979) para múltiplas comparações dentro da célula. Como as contagens de violação podem ser nulas, a análise principal dessas contagens é a diferença de taxas entre os braços, e uma contagem nula é relatada como "não observada em N ações", com limite superior de 95 % próximo de $3/N$ (HANLEY; LIPPMAN-HAND, 1983), e nunca como ausência do fenômeno. A vazão útil, em ações S3 sem falha percebida por segundo, entra como critério de não inferioridade: o limite inferior do intervalo de 95 % da razão entre os braços deve ficar acima de $1 - \delta$, com $\delta = 10\,\%$. Esse critério impede que o controlador melhore a cauda apenas por reduzir a carga. A rastreabilidade, terceiro critério da Proposta, permanece: cada decisão grava os sinais, os limiares em vigor, a regra aplicada e o perfil resultante. Os limiares do controlador serão calibrados com dados da campanha com o cenário S3, por regra fixada antes de ver esses dados, e as execuções de calibração não entram na avaliação.

Trabalhos correlatos

Esta seção apresenta os seis estudos de maior aderência ao desenho proposto, identificados entre os 165 itens selecionados pelo mapeamento para leitura completa. A descrição de cada um indica o que o trabalho realiza e o que o presente trabalho aproveita ou como dele se diferencia. A classificação integral dos estudos primários pelas dimensões D1 a D8 do esquema de extração, com o posicionamento sistemático em relação a cada agrupamento, integra a etapa inicial do Trabalho de Conclusão de Curso II, conforme o Apêndice A. A apresentação segue ordem temática, da verificação formal de protocolos à consistência sob monitoramento, à latência de cauda orientada a SLO e à autoadaptação por realimentação.

No eixo da verificação formal, Newcombe (2014) e Newcombe *et al.* (2015) relatam a adoção de TLA+ pela Amazon Web Services para verificar protocolos de replicação, entre eles componentes do DynamoDB, com identificação de defeitos sutis antes da entrada em produção. O estudo legitima o método central deste trabalho, especificação em TLA+ e verificação por TLC, e fundamenta a prática de cardinalidades pequenas no espaço de estados, registrada como parâmetro T21 na Seção 3.2. A diferença está no domínio e no escopo: aqui o objeto é o estado de sessão sob recorte acadêmico, e a especificação formal é acoplada a um controlador heurístico em tempo de execução.

Ainda na verificação formal, Must oferece uma interface de concorrência com vários modelos de comunicação, verifica os programas por redução parcial de ordem dinâmica e foi aplicada a sistemas de produção, como *logs* replicados e protocolos de transação distribuída (ENEA *et al.*, 2024). A diferença é o ponto de partida: o núcleo sem falhas é tratado com TLA+/TLC, e Must figura como caminho de escalabilidade reservado ao Trabalho de Conclusão de Curso II, quando as falhas forem modeladas de forma explícita.

Para fechar o eixo formal, Nair, Petri e Shapiro (2020) provam a segurança de invariantes de objetos de alta disponibilidade por raciocínio composicional e local sob concorrência. O resultado liga os invariantes de convergência deste trabalho, I5 e I6, derivados da Consistência Eventual Forte dos CRDT, à literatura de prova formal de segurança. A diferença é metodológica: aqui a garantia provém de *model checking*, com exploração exaustiva e produção de contraexemplos, em vez de prova dedutiva, e o alvo são invariantes específicos de sessão, I1 a I6.

Considerando consistência sob monitoramento, Nguyen *et al.* (2019) constituem o estudo de maior proximidade com o desenho proposto. Os autores combinam consistência relaxada, monitoramento e recuperação em armazenamentos chave-valor: o projetista identifica um predicado de correção P sobre o estado distribuído, executa o protocolo sobre um armazenamento com consistência eventual ou causal, instrumenta a execução com um monitor que verifica P por detecção de predicados distribuídos e aciona a reversão a um estado que satisfaz P quando há violação. A avaliação sobre Voldemort, conduzida em instâncias EC2 da Amazon Web Services, registra vazão de 50% a 80% maior que a da consistência sequencial, custo de monitoramento tipicamente abaixo de 4% e detecção de mais de 99,9% das violações em menos de 50 milissegundos em redes regionais e em menos de 3 segundos em redes globais; os autores discutem ainda o caso em que reversões frequentes inviabilizam o progresso da aplicação. A taxa de detecção acima foi

medida em cenário projetado para provocar violações frequentes. Para o caso de reversões frequentes, os autores propõem mecanismos alternativos, como adiamento da resolução de violação ou avanço sob consistência mais restrita por janelas curtas. Este trabalho desloca a verificação formal a montante da execução, ou seja, antes de o sistema entrar em operação, no nível do modelo formal, com TLA+/TLC produzindo contraexemplos a priori; o controlador é heurístico e externo ao *data grid*, atua sobre um conjunto finito de perfis com histerese, em vez da reversão reativa da camada de dados, e o estudo de caso é a gerência de sessão, com invariantes I1 a I6 derivados de fontes normativas (TEMOSHOK *et al.*, 2025; OWASP Foundation, 2026).

Sobre latência de cauda orientada a SLO, Jaiman, BEN MOKHTAR e Rivière (2020) propõem o TailX, escalonador de consultas *multiget* heterogêneas para reduzir a latência de cauda em armazenamentos chave-valor, avaliado sobre Cassandra. O estudo fundamenta a adoção da cauda, percentis p95 e p99, como métrica primária deste trabalho e embasa vários valores do *baseline* na Tabela 3, entre eles a distribuição Zipfian, a mistura de operações e o volume por execução. A diferença é o ponto de atuação: o TailX intervém no escalonamento das consultas, enquanto este trabalho atua sobre políticas e parâmetros de configuração em tempo de execução, por meio de um controlador heurístico, e amarra a latência à correção dos invariantes.

No eixo da autoadaptação, Liu *et al.* (2017) apresentam o OnlineElastMan, gestor de elasticidade proativo e autotreinado para serviços de armazenamento em nuvem, orientado a SLO. O estudo representa a adaptação por realimentação sobre métricas observadas. A diferença é a natureza do mecanismo: o controlador deste trabalho é heurístico, baseado em regras com histerese e portanto auditável, opera sobre um conjunto finito de perfis pré-definidos e responde a violações de invariantes e à cauda de latência, em vez de autoescalar por modelo aprendido.

Em conjunto, os seis estudos cobrem os eixos que o presente trabalho articula em um único recorte: a verificação formal de protocolos de replicação, a consistência relaxada sob monitoramento e recuperação, a latência de cauda orientada a SLO e a autoadaptação por realimentação. A literatura trata esses eixos de forma isolada ou em pares; a contribuição pretendida está em reuni-los sobre invariantes de sessão de origem normativa, com verificação a montante por *model checking* e um controlador heurístico auditável a jusante, ou seja, durante a execução, sobre o sistema em operação, lacuna que o mapeamento sistemático, ao classificar os 165 estudos pelas dimensões D1 a D8 no Trabalho de Conclusão de Curso II, deve confirmar.

4 DESENVOLVIMENTO E RESULTADOS PARCIAIS

Este capítulo documenta a construção do ambiente de pesquisa e os achados da etapa atual, estruturando-se a partir da infraestrutura de testes até a análise do comportamento do sistema. A definição da arquitetura experimental estabelece a topologia da rede e os mecanismos de geração de carga, provendo a base para os ensaios práticos. De forma complementar, a especificação formal modela as regras de transição para validar rigorosamente as propriedades do sistema e antecipar divergências de estado. O confronto entre a teoria e a prática ocorre na execução do *model checking* e do *baseline* experimental, etapa que expõe a resposta operacional sob cenários controlados de interrupção e degradação de rede. O conjunto de ferramentas desenvolvidas, incluindo o código de orquestração e os *pipelines* de análise estatística, é consolidado na descrição do estado dos artefatos. O texto atinge então a apresentação dos resultados parciais e das limitações metodológicas, articulando os padrões de violação detectados com as lacunas de avaliação identificadas na revisão da literatura.

4.1 Arquitetura experimental

O ambiente experimental documentado nesta etapa atende a três requisitos. O primeiro é reproduzir os modos de operação do Infinispan relevantes ao trabalho, em particular o modo `DIST_SYNC` para experimentos com replicação síncrona e o modo `DIST_ASYNC` para experimentos com replicação assíncrona. O segundo é permitir injeção parametrizada de falhas das classes definidas na Seção 3.2, sem alteração no código do protótipo. O terceiro é expor os observáveis necessários para detecção de violações e medição de percentis de latência.

O *cluster* é composto por três nós Infinispan, executados em *containers* Linux gerenciados por *Podman* (Podman Project, 2026) com o arquivo de orquestração `cluster/podman-compose.yml` no repositório `tcc-desenvolvimento`. Cada nó executa o servidor Infinispan na versão estável, com configuração de *cache* distribuído baseada na documentação oficial e `numOwners=2`, o que mantém duas cópias por chave (Infinispan Project, 2026c). A rede entre *containers* é uma *bridge* controlada pelo *Podman*, com latência e perda de pacotes injetadas via `tc` no *network namespace* de cada nó, conforme o cenário em execução. Métricas são exportadas em formato OpenMetrics (OpenMetrics Project, 2024) conforme a documentação de métricas (Infinispan Project,

2026d), e podem ser agente Prometheus opcional (Prometheus Authors, 2026).

Na versão atual, os *containers* Podman executam no modo *rootful*, sobre a rede *netavark*, em uma máquina virtual Ubuntu Server 24.04 sob Hyper-V. O ambiente WSL2 usado no início não tinha o módulo de *kernel* `sch_netem`, necessário às falhas de rede, e foi substituído pela máquina virtual. A descoberta de membros usa JGroups com transporte TCP e lista explícita de nós, e a pilha de detecção de falhas inclui FD_ALL3 e FD_SOCK2 (Seção 3.2).

A geração de carga ocorre por um programa Java executado fora do *cluster*, via Hot Rod (Infinispan Project, 2026c), que produz operações concorrentes sobre chaves de sessão. As operações implementadas correspondem ao conjunto definido na Seção 3.2. Para cada operação, o programa de carga registra marca de tempo de início e fim com precisão de nanossegundos, código de retorno, servidor de conexão do cliente e parâmetros de entrada. As checagens de invariantes ocorrem em dois momentos: imediatamente após cada operação, o programa verifica a coerência local (por exemplo, uma validação que retorna sucesso após *logout* confirmado constitui violação registrada de I1); e periodicamente, em intervalos configuráveis, um auditor consulta todas as réplicas e detecta divergências persistentes (violações de I5 e I6 sob a premissa de quiescência). Na versão atual, a verificação de I5 e I6 é feita por um laço de convergência executado depois que a carga cessa (Seção 4.4).

4.2 Especificação formal inicial

Entre as opções de ferramental discutidas na Seção 2.3, a escolha adotada nesta etapa inicial é TLA+ com o verificador TLC (YU; MANOLIOS; LAMPORT, 1999), pela maturidade do ferramental, pelo registro de aplicação industrial a protocolos de replicação (NEWCOMBE, 2014; NEWCOMBE *et al.*, 2015) e pela adequação da semântica de transições e invariantes ao recorte de invariantes I1 a I6 deste trabalho. Ferramentas como Must, que oferece uma interface de concorrência com vários modelos de comunicação e verifica os programas por redução parcial de ordem dinâmica (ENEA *et al.*, 2024), ficaram, no planejamento, como alternativa a revisitar caso o refinamento das transições de canal previsto para o Trabalho de Conclusão de Curso II exigisse primitivas que escapassem a TLA+. A especificação formal inicial está escrita em TLA+ e versionada como `spec/SessionStore.tla` no repositório `tcc-desenvolvimento`. O módulo declara o conjunto de identidades, o conjunto de sessões e dois estados lógi-

cos relevantes (`Valid`, `Invalid`), modelados como funções totais sobre os conjuntos correspondentes. O canal de replicação assíncrona é modelado como uma multiset de mensagens pendentes, com regras de transição que entregam uma mensagem qualquer da multiset em qualquer momento, o que captura atraso não determinístico e reordenação. Cada réplica é um processo com seu próprio estado local, e as transições representam, respectivamente, as operações O1 a O7 da Seção 3.2. A abstração corresponde a um modelo de replicação otimista, distinto de algoritmos de consenso baseados em líder eleito como Raft (ONGARO; OUSTERHOUT, 2014), escolha consistente com o objeto de estudo (replicação assíncrona com reconciliação) declarado no Capítulo 1. O modelo com falhas, descrito adiante, foi escrito no próprio TLA+, e Must e o tradutor Apalache (Seção 3.2) seguem sem uso, como alternativas para ampliar a cobertura.

Na versão atual, a especificação está acompanhada de um módulo de verificação, `spec/MCSession.tla`, que declara os cenários e as propriedades verificadas. Cada sessão e cada identidade é um registro único por réplica, com uma marca de tempo (ts), espelhando o objeto imutável que o programa de carga grava como unidade no *data grid*. As ações correspondem às operações O1 e O3 a O7 da Seção 3.2. Cada escrita local é difundida às demais réplicas por filas de mensagens pendentes, uma por réplica, e qualquer mensagem pendente pode ser entregue a qualquer momento, o que captura atraso e reordenação. Na entrega, a réplica aplica o registro recebido somente se a marca de tempo da mensagem for maior que a local. Essa regra é um dispositivo de modelagem para evitar regressões arbitrárias no canal abstrato, e não o mecanismo de resolução de conflitos do Infinispan, que sob partição segue a política de fusão configurada.

O modelo completa o estado com a função auxiliar $\mathsf{dono} : \mathscr{S} \to \mathscr{U}$, que associa a cada sessão s a identidade u à qual a sessão pertence. A relação $\mathsf{sessions}(u)$, empregada nos invariantes da Seção 3.2, é a inversa de dono: $s \in \mathsf{sessions}(u)$ se e somente se $\mathsf{dono}(s) = u$. No modelo, duas operações diferem da definição da Seção 3.2. O efeito do *login* (O1) inclui $\mathsf{dono}'(s) = u$ e não altera o contador, cuja reposição é a operação O5. O bloqueio por limite (O4), por decisão de domínio, não invalida as sessões existentes, e só o bloqueio administrativo (O6) tem cascata de invalidação.

Os invariantes I1 a I6 são declarados como fórmulas sobre o estado global, com I1 a I4 caracterizadas como propriedades de segurança incondicionais e I5 e I6 como propriedades de convergência condicionadas ao predicado de quiescência (canal de mensagens vazio). A verificação por TLC explora o espaço de estados a partir de configurações iniciais com um número pequeno de identidades, sessões e réplicas (tipicamente duas a três

de cada). Em conformidade com a decisão metodológica registrada na Seção 3.2, a especificação inicial não modelava falhas explícitas; a execução do verificador e a leitura dos contraexemplos produzidos integram a Seção 4.3. O refinamento da especificação com a inclusão de transições de falha e o tratamento explícito dos contraexemplos obtidos estavam planejados para o Trabalho de Conclusão de Curso II, e a inclusão das falhas, descrita adiante, foi feita nessa etapa.

Na versão atual, os invariantes também são declarados sobre a função dono, e a quiescência é o estado em que todas as filas de mensagens estão vazias. O registrador de cada réplica modela também dono e β.marca, e as formas de I5 e I6 foram ampliadas para incluí-los. I5 passa a exigir a convergência do estado lógico e do dono de cada sessão:

$$\mathsf{quiescent}(Q) \Rightarrow \forall s, \forall r_1, r_2 \in \mathscr{R} : \sigma_{r_1}(s) = \sigma_{r_2}(s) \wedge \mathsf{dono}_{r_1}(s) = \mathsf{dono}_{r_2}(s).$$

I6 passa a exigir a convergência do contador e da marca de bloqueio:

$$\mathsf{quiescent}(Q) \Rightarrow \forall u, \forall r_1, r_2 \in \mathscr{R} : \beta_{r_1}(u).\mathsf{counter} = \beta_{r_2}(u).\mathsf{counter} \\ \wedge\ \beta_{r_1}(u).\mathsf{marca} = \beta_{r_2}(u).\mathsf{marca}.$$

As fórmulas de I1 e I2 da Seção 3.2 tratam do estado lógico da sessão. No modelo com várias réplicas, a verificação as complementa com versões avaliadas réplica a réplica, descritas na Seção 4.3. I3 e I4 estão declarados no modelo TLA+ em formas de domínio mais fracas, que são consequência da tipagem e aparecem na mesma seção. A forma temporal de I4 é verificada réplica a réplica, e a de I3 não foi verificada.

A especificação evoluiu em etapas. A versão do Trabalho de Conclusão de Curso I não modelava falhas. Em setembro de 2026, a entrega passou a difundir cada escrita a todas as réplicas, a comparação de marcas de tempo foi corrigida e a cascata de invalidação do bloqueio administrativo foi refeita. No início de outubro, uma revisão do mecanismo de conflito substituiu as variáveis independentes por campo pelo registro único por chave e retirou o desempate entre réplicas, que deixara de ter função. A revisão se apoiou no código-fonte do Infinispan e do programa de carga. Nele, cada sessão e cada identidade são gravadas como um registro único, com um único número de versão, e a mudança visou reproduzir essa estrutura no modelo. Em seguida, o modelo ganhou as ações de falha: `Cair` e `Reviver`, que representam a queda e a recuperação de uma réplica (F1), e `Particionar` e `Mesclar`, que representam a partição e a fusão (F2). Na fusão, o lado com mais réplicas (ou, em empate, qualquer dos lados) copia o registro inteiro de uma de

suas réplicas para as do outro lado, sem comparação de marcas de tempo. Duas variantes posteriores, uma com perda de mensagens no canal e outra com relógio atualizado pelo máximo na entrega, foram verificadas em cópias separadas da especificação, ainda não incorporadas à versão principal. A variante com perda tem caráter ilustrativo, porque o modelo não tem retransmissão, e nenhuma das duas corrige o problema de I5 e I6 descrito adiante.

4.3 Execução do *model checking* e estado do *baseline* experimental

A execução das semanas 9 e 10 do cronograma do Trabalho de Conclusão de Curso I compreende duas atividades. A primeira, verificação formal por *model checking*, foi concluída. A segunda, *baseline* experimental sob falhas selecionadas, foi executada para os cenários F1 e F3 em piloto reduzido, e a sua condução é tratada ao final desta seção. Um diagnóstico dos registros brutos, feito em outubro de 2026, mostrou que essa bateria não mediu o ciclo de vida de sessões, como descrito adiante.

Verificação formal

A verificação formal foi conduzida em cinco rodadas sucessivas do TLC sobre a especificação descrita na Seção 4.2. O modelo foi instanciado nas três configurações de cardinalidade crescente parametrizadas em T21 da Tabela 3: a configuração reduzida `MC-small` (duas identidades, três sessões, duas réplicas), a configuração intermediária `MC-medium` (três identidades, três sessões, duas réplicas) e a configuração ampliada `MC-full` (três identidades, três sessões, três réplicas). As cinco rodadas, de maio a julho de 2026, usaram só essas três configurações; a configuração `tiny` foi acrescentada depois, nas verificações de setembro de 2026, com o módulo `MCSession.tla`. Os *scripts* de execução e os arquivos de configuração estão versionados em `spec/run-tlc.sh`, `spec/MC-small.cfg`, `spec/MC-medium.cfg` e `spec/MC-full.cfg` no repositório `tcc-desenvolvimento`.

Com a inclusão das falhas no modelo, a verificação dos invariantes I1 a I6 passou a usar quatro configurações de cardinalidade, todas com no máximo uma mensagem pendente por fila, relógio limitado a três e no máximo um evento de falha por execução. A configuração `tiny` tem uma identidade, duas sessões, duas réplicas e limite de bloqueio

igual a dois; `small` tem duas identidades, três sessões e duas réplicas; `medium` tem três identidades, três sessões e duas réplicas; e `full` tem três identidades, três sessões e três réplicas. As saídas brutas do TLC estão em `runs/tlc/` no repositório. A versão anterior deste texto reproduziu uma expectativa registrada antes da execução, e não o resultado medido. Os números a seguir vêm dos registros de saída do verificador.

Os invariantes I1 e I2 são violados em traços curtos. I1 é violado em traço de 4 estados nas configurações `tiny`, `small` e `medium` e de 5 estados em `full`; I2, em traço de 4 estados em `tiny`, `small` e `medium` e de 3 estados em `full`. Nenhum desses traços contém ação de falha. As violações decorrem da forma das fórmulas, avaliadas sobre o estado de todas as réplicas: basta que uma réplica tenha aplicado a invalidação ou o bloqueio e outra ainda não, situação normal enquanto há mensagens em trânsito. As violações de I1 e I2 nessa forma são, portanto, consequência direta da divergência transitória e não informam sobre o protocolo.

Para separar esse efeito, o módulo de verificação acrescentou propriedades avaliadas réplica a réplica. Essas propriedades foram verificadas em um cenário semeado, com uma identidade, duas sessões já válidas, três réplicas, *login* desabilitado e cotas maiores (até três mensagens por fila, relógio até cinco e dois eventos de falha). A versão de I1 por réplica afirma que uma sessão inválida em uma réplica não passa a outro estado nessa réplica no passo seguinte. No cenário semeado, ela não foi violada sem falha nem com queda, de forma exaustiva com cotas reduzidas e até o tempo-limite com as cotas do plano, e foi violada sob partição, em traço de 4 estados. Esse resultado é coerente com a perda, na fusão, de escritas feitas na partição menor entre a divisão e sua detecção, descrita pela documentação do Infinispan (Infinispan Project, 2026a, seção 7.1.1), com duas ressalvas: no modelo, a escrita da réplica isolada é anterior à partição e ainda não propagada, aproximação daquela janela, e a fusão do modelo é uma abstração da política real. O comportamento real sob partição com carga ainda não foi medido.

No modelo com *login* habilitado, a versão de I1 por réplica também é violada sem falha e com queda, por artefatos do modelo. O relógio de cada réplica não acompanha as marcas recebidas, e, na variante que corrige esse ponto, a violação vem de *logins* concorrentes da mesma sessão em réplicas diferentes. Nenhum dos dois ocorre no programa de carga, que usa a versão do registro lido e identificador único de sessão. A versão de I2 por réplica, segundo a qual uma identidade bloqueada não tem sessão válida na mesma réplica, vale para qualquer tipo de bloqueio e é violada em traços curtos, de 3 a 6 estados conforme o cenário. Todos os 28 contraexemplos obtidos vêm do bloqueio por limite

(O4), por exemplo no traço formado por um *login* seguido de dois incrementos de tentativas falhas, porque esse bloqueio não invalida as sessões existentes (Seção 4.2). Esse caso é tratado como violação esperada por desenho, e a análise experimental o separa do bloqueio administrativo. A versão de I4 por réplica, que exige que o motivo de invalidação não mude, também é violada.

Os invariantes I5 e I6 são violados nas quatro configurações, em traços de 5 a 7 estados, por dois mecanismos. Em cinco dos oito traços mínimos, duas escritas concorrentes na mesma chave recebem a mesma marca de tempo, porque cada réplica atribui marcas pelo próprio contador, e a regra de entrega, que exige marca estritamente maior, faz cada réplica recusar o registro da outra. A divergência persiste depois que as filas se esvaziam. Esse é um defeito de modelagem, surgido com a retirada do desempate entre réplicas, e sua correção, por reintrodução de um desempate ou por canal com entrega em ordem, está pendente. Nos outros três traços (I5 em `tiny` e I6 em `small` e em `full`), a fusão copia o registro do lado vencedor sobre uma escrita local ainda não propagada, e as mensagens dessa escrita, entregues depois, alcançam só as outras réplicas, o que deixa divergência permanente. Esse segundo mecanismo é um resultado do modelo sob partição, coerente com a seção 7.1.1 da documentação citada acima e independente da correção do empate. Até essa correção, os contraexemplos de I5 e I6 que não passam pela fusão não informam sobre o protocolo.

As formas de I3 e I4 declaradas no modelo, contador dentro da faixa de 0 ao limite e motivo dentro do conjunto declarado, não foram violadas. A verificação foi exaustiva apenas na configuração `tiny`, com 570 089 estados distintos, 11 243 221 estados gerados e profundidade 21. Nas configurações maiores, as execuções foram interrompidas pelo tempo-limite sem violação. Em `small`, por exemplo, a verificação de I3 percorreu 30 474 782 estados distintos em cerca de 9 minutos, e em `full`, 5 466 292 estados distintos em cerca de 4 minutos. Essas formas decorrem da tipagem e não verificam a monotonicidade do contador nem a imutabilidade do motivo. Esta última, avaliada réplica a réplica, é violada, como descrito acima.

Os traços produzidos pelo TLC alimentam o catálogo inicial de combinações operação $\times$ canal $\times$ invariante que orienta a fase subsequente. O refinamento da especificação para reduzir os contraexemplos por mecanismos de confirmação em duplo envio ou *anti-entropy* dirigida fica registrado como ação para o Trabalho de Conclusão de Curso II (Apêndice A, etapa 2). Na versão atual, a pendência da especificação é a correção do empate de marcas de tempo descrita acima, e o catálogo passou a relacionar operação,

falha e invariante, base do alinhamento entre o modelo e os experimentos previsto no Apêndice A.

Baseline experimental

O *baseline* experimental parcial previsto para o Trabalho de Conclusão de Curso I cobre dois cenários de falha: falha de nó (F1), executada como parada ordenada, e atraso com *jitter* no canal (F3). Os parâmetros operacionais de cada cenário (tamanho do *cluster*, modo de replicação, `numOwners`, distribuição de chaves, mistura de operações, volume e duração) estão consolidados na Tabela 3 da Seção 3.2, com as fontes que os fundamentam. Nesta etapa, os dois cenários foram executados na mesma bateria, contra a mesma condição *sem falha* de referência, e seus resultados são reportados nesta subseção.

A execução do cenário F3 exige o módulo de *kernel* `sch_netem` para a injeção de atraso por `tc-netem` (LUDOVICI; PFEIFER, 2011), ausente no ambiente WSL2 utilizado no piloto inicial de F1. Para viabilizar os dois cenários sob o mesmo ambiente, a bateria foi reexecutada em uma máquina virtual Linux (Hyper-V, Ubuntu Server 24.04), com o *cluster* Infinispan em contêineres *Podman* no modo *rootful* sobre a rede *netavark*. O atraso de canal de F3 foi aplicado a partir do *host* da máquina virtual, no *network namespace* do contêiner-alvo `isn2`, acessado via `nsenter`, com a disciplina `tc-netem` configurada em `delay 20ms` e desvio de 13 ms sob `distribution normal`, calibrada para deslocar o percentil 99 em aproximadamente 50 ms, o alvo de T18 (sob a aproximação $p99 \approx \text{média} + 2{,}326 \cdot \text{desvio}$). A falha F1 usou `podman stop` e `podman start` sobre o mesmo contêiner `isn2`, isto é, uma parada ordenada, e não a queda silenciosa definida na Seção 3.2. Com `SIGTERM`, o nó sai de forma ordenada, mecanismo explicado na Seção 4.5, e por isso essa bateria e as verificações intermediárias que usaram o mesmo comando são descritas como parada ordenada.

O *cluster* foi composto por três nós Infinispan, com imagem fixada em `quay.io/infinispan/server:15.0.21.Final`, em `DIST_SYNC` e `numOwners=2`. A configuração do *cluster* só tinha *caches* síncronos, e todas as execuções na máquina virtual até outubro de 2026, incluindo as seis células desta bateria, ocorreram nesse modo. A descoberta de membros adotou JGroups com transporte TCP e protocolo `TCPPING` sobre lista explícita de *hosts*. A formação do *cluster* em três membros foi validada pelo *endpoint* REST de saúde antes de cada repetição, e o nó parado em F1 retornou ao tamanho três entre repetições. A bateria permanece um piloto reduzido em

relação à bateria plena desenhada nos parâmetros T9 e T10 (30 repetições de 10^6 operações). O piloto cobriu seis células, combinação dos dois cenários de mistura S1 (50/50) e S2 (95/5) com as três condições *sem falha*, F1 e F3, em três repetições de 90 s por célula, com janela de injeção de *jitter* de 65 s nas células F3. Os eventos do aquecimento foram descartados em todas as células.

Um diagnóstico dos registros brutos, feito em outubro de 2026, mostrou que a bateria não mediu o ciclo de vida de sessões. Em todas as seis células, 100 % dos *logins* retornaram recusa por bloqueio, e 100 % das validações retornaram ausência de resultado (código `NONE`), que encobria a leitura de chave ausente; na célula S1 sem falha, por exemplo, os 1 291 332 *logins* registrados foram todos recusados. Nenhum evento registrou código de erro. *Login* e validação somaram 75 % dos eventos em S1 e 98 % em S2. Quatro causas, todas de desenho da carga e da análise, explicam o resultado. A primeira é a identidade única de cada *thread*, que atingia o limite de tentativas e permanecia bloqueada, já que a mistura não continha desbloqueio. Essa identidade servia ao *login* e às operações O4 e O5. Com limiar de bloqueio igual a cinco, O5 preservando a marca de bloqueio e O7 ausente da mistura, o tempo esperado de absorção da cadeia do contador indica bloqueio permanente depois de cerca de 130 operações da *thread* em S1 e 500 em S2, em média, o que corresponde a frações de segundo no ritmo medido. Nessa cadeia, o incremento tem probabilidade 0,10 e a reposição 0,05 em S1; em S2, o incremento tem probabilidade 0,01, sem reposição. A segunda é a leitura, pelas validações e *logouts* isolados, de chaves sintéticas que nenhuma operação gravava. A terceira é a ausência de limpeza do *cluster* entre execuções, de modo que as identidades já estavam bloqueadas no primeiro evento. A quarta é o mascaramento: o programa registrava a chave ausente com o mesmo código de ausência de resultado, e o *script* de percentis contava recusas e ausências como sucesso, o que ocultou o problema nos resumos.

As latências medidas nessa bateria correspondem, portanto, à recusa de *login*, à leitura de chave ausente e às escritas do contador sobre identidades já bloqueadas, e não às operações que os rótulos indicam. Por isso, a versão anterior deste capítulo, que apresentava os percentis de latência dessa bateria por operação e condição de falha, foi revista: as tabelas foram retiradas, e nenhum número de latência de junho é usado como resultado ou será comparado com os da nova medição. A ausência de erros e de violações nessa bateria também não indica *failover* sem efeito, porque nenhuma sessão foi criada. A carga foi corrigida nos pontos descritos na Seção 3.2: limpeza e conferência dos *caches* antes de cada execução, identidades de *login* separadas das que recebem tentativas falhas,

chaves de sessão gravadas antes da medição, códigos distintos para sessão válida, inválida e ausente, e um verificador de sanidade que recusa execuções incoerentes com a mistura declarada. Os limiares desse verificador, definidos no diagnóstico da bateria de junho, foram fixados no programa de carga congelado antes do piloto da nova medição, de modo que não foram ajustados depois de observados os dados. A nova medição de referência, com essa carga, foi pré-registrada (Seção 3.2) e ainda não foi executada.

Os artefatos do piloto estão organizados em `runs/` com um diretório por célula (`S1-none`, `S1-F1`, `S1-F3`, `S2-none`, `S2-F1` e `S2-F3`), cada qual com o CSV de eventos crus por repetição, o `summary.json` por célula e o `comparacao.json` por contraste, gerados pelo *pipeline* de análise em `analysis/` (*numpy* e *scipy*).

A bateria plena, com os parâmetros T9 e T10 da Tabela 3 (30 repetições de 10^6 operações) e a carga em 75% do pico do piloto sem falhas, permanece como protocolo desenhado e escalável a partir do piloto aqui reportado. Esse protocolo deu lugar à nova medição de referência pré-registrada (Seção 3.2), que mede `DIST_SYNC` como controle e `DIST_ASYNC` como alvo na mesma campanha, em dez repetições de 300 s por célula, com F1 por queda silenciosa, e que ainda não foi executada. Nessa bateria, a duração da indisponibilidade do nó em F1 fica fixada em 60 s (T17), valor superior ao *timeout* padrão do detector de falhas FD_ALL3 do Infinispan (40 s, T16), o que garante a detecção do evento pelo *cluster* antes do término da janela. O atraso de canal de F3 segue calibrado para deslocar o percentil 99 em 50 ms (T18), via `tc-netem` (LUDOVICI; PFEIFER, 2011), valor de engenharia, sem fonte empírica, e calibrável. Os dois cenários previstos como ampliação opcional, partição (F2) e *state transfer* induzido (F4), ficaram reservados, no planejamento, para o Trabalho de Conclusão de Curso II. Esse plano foi modificado, e F4 foi redefinida (Seção 4.5). Os injetores de F2 e F4 passam pelas verificações sem carga de aplicação descritas a seguir, e a campanha com o cenário S3 os exercita sob carga.

Verificação dos injetores na máquina virtual

Antes de usar os injetores de F2 e F4 sob carga, cada um passa por uma sequência de verificações na máquina virtual, sem carga de aplicação, em que cada etapa só começa depois que a anterior é validada. Para F2, o nó `isn3` foi isolado dos outros dois por uma janela de cerca de 90 s em quatro repetições. O tempo entre o fim da janela e o retorno do *cluster* ao estado saudável com três nós foi medido por consultas a cada 2 s. Nas três repetições feitas em *cluster* recém-criado e com monitoramento mínimo (a primeira, a

terceira e a quarta), esse tempo foi de 48 s, 46 s e 44 s. A segunda repetição, feita em um *cluster* que já sofrera duas partições, com uma sonda que escrevia durante a partição e com coleta de monitoramento mais intensa, levou 188 s. Os registros mostram que o atraso ocorreu depois da fusão da vista, na resolução de conflitos de um dos *caches*, que esgotou o tempo-limite de 60 s. A repetição seguinte, em *cluster* novo, enfraqueceu a hipótese da sonda, mas as duas hipóteses restantes, histórico do *cluster* e coleta intensa, não foram separadas. A causa não foi isolada, e a repetição é registrada como limitação e reportada à parte. Esses tempos dizem respeito apenas à recomposição da topologia: sem carga de aplicação, nada indicam sobre divergência de dados ou perda de escritas.

Das oito etapas da sequência mais recente, cinco foram executadas. A primeira preparou, no hospedeiro, o pacote de código e seus resumos criptográficos. A segunda, na máquina virtual e sem alterar a rede, instalou esse pacote com conferência dos resumos, verificou as ferramentas e carregou os módulos do *kernel* usados por F4, executou na própria máquina virtual as suítes de teste dos injetores de F2 e F4 e leu o estado do *cluster*, que estava parado depois de uma queda do hospedeiro, com investigação da causa. A terceira verificou que o *kernel* aceita as disciplinas e o filtro de F4 sem afetar o tráfego, com filtro para um endereço reservado a testes. A quarta verificou que o temporizador do sistema armado pelo *script* de verificação dispara e aciona a limpeza do injetor de F4. A quinta repetiu a partição em *cluster* recém-criado. Essa repetição substituiu outra, planejada para o *cluster* com histórico, que perdeu o sentido quando uma queda do hospedeiro desfez o *cluster*, e seu tempo de recomposição está entre os citados acima. Faltam a primeira injeção real de F4, com perda de 20 % por 60 s e limpeza posterior, e duas verificações do injetor de F2, uma da recomposição manual depois de uma partição real e outra do temporizador de segurança, armado pelo *script* de verificação, disparando sozinho. O modo silencioso de F1 foi ensaiado apenas no *cluster* local, e seu ensaio na máquina virtual precede a recriação do *cluster* com os quatro *caches*.

4.4 Estado dos artefatos

Os artefatos versionados no repositório `tcc-desenvolvimento` cobrem, ao final do Trabalho de Conclusão de Curso I, três linhas de trabalho integradas, complementadas por um *pipeline* de análise estatística e por *scripts* de orquestração.

A primeira linha de trabalho é a configuração do *cluster* Infinispan. O arquivo `cluster/podman-compose.yml` orquestra três nós da imagem oficial fixada em

`quay.io/infinispan/server:15.0.21.Final` sobre uma rede *bridge* dedicada, e o arquivo `cluster/infinispan-cluster.xml` descreve os *caches* com `numOwners=2` e 64 segmentos, com parametrização baseada nas linhas T1 a T4, T15 e T16 da Tabela 3, exceto pelo número de segmentos, que em T4 é 256. Até outubro de 2026, a configuração tinha apenas os *caches* `sessions` e `counters`, ambos síncronos (`DIST_SYNC`). A política de partição adotada é `DENY_READ_WRITES` com *merge-policy* `PREFERRED_NON_NULL` (Infinispan Project, 2026a, seção 7), que define o comportamento padrão diante de partição involuntária no escopo do Trabalho de Conclusão de Curso I, dado que a injeção controlada de F2 estava reservada, no planejamento, para o Trabalho de Conclusão de Curso II. O injetor de F2 foi implementado depois (Seção 4.3).

A versão atual acrescenta `sessions-async` e `counters-async` em modo assíncrono, com os mesmos parâmetros, selecionados pela opção `--replication` do programa de carga. Os quatro *caches* são não transacionais. Essa versão foi testada no *cluster* local e só será instalada na máquina virtual quando o *cluster* for recriado para a nova medição de referência.

A segunda é o programa de carga em Java sobre Hot Rod, organizado em projeto Maven (Apache Software Foundation, 2026) (Java 17), com sete componentes:

- as operações O1 a O7 padronizadas sob o tipo único de retorno `OpResult`;
- um gerador de chaves Zipfian autocontido, baseado em amostragem por rejeição-inversão de complexidade $O(1)$ por amostra, com parâmetros $\rho = 0{,}99$ e universo de 100 000 chaves, conforme T5, T6 e T8;
- um seletor de cenários que codifica as misturas S1 (*50/50*) e S2 (*95/5*) de T7;
- uma política de aquecimento (T11) que estende a janela de não-medição em $\max(60\,\text{s},\ 10\,\% \cdot \text{duração})$ seguidos de 5 % do tempo restante;
- instrumentação de latência baseada em *HdrHistogram* (TENE, 2026) com despejo em CSV (T13);
- um auditor de invariantes que combina verificação imediata após cada operação para I1 a I4 com auditoria periódica em janelas de quiescência para I5 e I6;
- uma interface de linha de comando com onze opções nomeadas que parametriza T9, T10, T11 e T14 e produz mensagem de ajuda autogerada com validação por contrato.

Até outubro de 2026, o programa de carga recebeu os seguintes acréscimos:

- nas operações O1 a O7, a invalidação do *logout* e da cascata de bloqueio por escrita condicional à versão e códigos que distinguem erro de transporte (`ERROR_TRANSPORT`) de recusa por indisponibilidade (`ERROR_UNAVAILABLE`);
- nas misturas S1 e S2, a versão de *slots* e gerações, que é a usada nas medições (Seção 3.2);
- o cenário S3, com servidores de aplicação ligados a nós distintos, usuários virtuais, ator administrativo e conjunto de identidades para bloqueio por limite (Seção 3.2);
- uma fase de preparação que esvazia e confere os *caches* e grava a carga inicial de *slots*;
- o registro de cada evento em CSV, com marcas de tempo de início e fim em nanossegundos, código de retorno, servidor de conexão do cliente e, quando há marcadores de falha, fase da execução;
- no auditor, a classificação de I2 por marcos de bloqueio e um laço de convergência para I5 e I6, descrito a seguir;
- um verificador de sanidade que lê o CSV de eventos e emite veredito sobre a mistura observada;
- o pacote do controlador heurístico, descrito adiante, e opções nomeadas de linha de comando para duração, repetições, aquecimento, modo de replicação, mistura, cenário e semente.

O laço de convergência roda depois que a carga cessa, porque I5 e I6 só são verificáveis sob quiescência. A cada 5 s, suas rodadas comparam o estado das réplicas, inclusive o dono de cada sessão ($\mathsf{dono}_{r_1} = \mathsf{dono}_{r_2}$), até a convergência ou até um tempo-limite. O tempo de convergência vai do fim da carga até a primeira de três rodadas consecutivas sem divergência entre réplicas, com teto de 600 s. A nova medição de referência não usa o laço, e as regras de classificação e de contagem que ela adota estão na Seção 3.2.

A terceira é a especificação TLA+ `spec/SessionStore.tla`, com as operações O1, O3, O4, O5, O6 e O7 e os invariantes I1 a I6 declarados, acompanhada, no Trabalho de Conclusão de Curso I, de três configurações de cardinalidade (T21) e de um

runner de execução. A operação O2, por ser leitura pura, não tem ação dedicada no modelo, conforme registrado na Seção 4.2. Na versão atual, a especificação inclui as ações de falha e é acompanhada do módulo de verificação `spec/MCSession.tla` e de quatro configurações de cardinalidade, descritos nas Seções 4.2 e 4.3.

Os artefatos complementares incluem o *pipeline* de análise estatística em Python (`analysis/percentis.py`, `analysis/bootstrap_ic.py` e `analysis/compare_scenarios.py`, com base em *numpy* (HARRIS *et al.*, 2020) e *scipy* (VIRTANEN *et al.*, 2020), cobrindo T13 e T22) e quatro *scripts* de orquestração: `scripts/inject-crash.sh` para a falha F1 conforme T17, `scripts/inject-jitter.sh` para a falha F3 conforme T18 (com remoção idempotente da disciplina `tc-netem` via *trap* em `EXIT`, `INT` ou `TERM`), `scripts/collect-metrics.sh` para coleta periódica do *endpoint* OpenMetrics nos três nós conforme T19 e T20 (escrito, mas não integrado, como registrado adiante), e `scripts/run-baseline.sh` como orquestrador da bateria cenário $\times$ falha $\times$ repetição. Cada *script* aceita o argumento `--dry-run` que imprime o plano e os comandos previstos sem dependência de Podman, de *cluster* ativo ou da *capability* `NET_ADMIN`.

Na versão atual, os *scripts* de injeção cobrem as quatro classes de falha: `inject-crash.sh` (F1, nos modos de parada ordenada e de queda silenciosa), `inject-partition.sh` (F2), `inject-jitter.sh` (F3) e `inject-netdegrade.sh` (F4, com disciplina de prioridade, `netem` por banda e filtro por endereço de destino). As falhas de rede usam `tc-netem` (LUDOVICI; PFEIFER, 2011), aplicado a partir do hospedeiro no *network namespace* do *container*, porque a imagem do Infinispan não traz a ferramenta `tc`. Os injetores de F2 e F4 gravam marcadores de início e fim, removem as disciplinas de rede ao terminar, por rotina de limpeza que roda também quando recebem sinal de término, e oferecem um modo de limpeza avulso. Nas verificações na máquina virtual, os *scripts* de verificação armam ainda um temporizador do sistema que aciona essa limpeza se o injetor for morto. Completam o conjunto o coletor de recursos `collect-resources.sh` (Seção 3.2), os orquestradores `run-baseline.sh` e `run-rebaseline.sh`, o gerador da matriz da nova medição e os *scripts* de análise em Python (`rebaseline.py`, `fases.py`, `referencia_p99.py`, `energia_proxy.py` e `comparar_bracos.py`). Das quatro classes de falha com injetor, as três primeiras também aparecem no modelo formal, F1 e F2 como ações e F3 como atraso no canal (Seção 4.2).

O coletor das métricas secundárias expostas pelo Infinispan (Seção 3.2), `scripts/collect-metrics.sh`, foi escrito em junho de 2026, mas existe apenas em ramos separados do repositório e nunca foi integrado ao ramo principal nem à versão atual, de modo que não foi usado nas baterias. O coletor de recursos mede o uso de CPU de cada *container*, fora do Infinispan. Como a máquina virtual não dá acesso aos contadores de energia do processador, o custo energético é estimado a partir desse tempo de CPU e de um modelo de potência declarado (FAN; WEBER; BARROSO, 2007; KANSAL *et al.*, 2010), conforme a Seção 3.2. O coletor de recursos e o *script* de estimativa de energia estão implementados e testados com dados simulados, mas nenhum consumo foi medido em campanha, e a nova medição de referência não coleta uso de CPU.

A vertente formal foi executada em cinco rodadas do TLC nas três configurações da Tabela 3 (T21); a configuração `tiny` foi acrescentada depois, nas verificações de setembro de 2026. Os resultados estão descritos na Seção 4.3. A vertente experimental compila e empacota como *jar shaded* de aproximadamente 18 MB e dispõe de suíte de testes JUnit (JUnit Team, 2026), com seleção determinística da *seed* para evitar variação amostral entre rodadas. Uma verificação determinística da suíte, conduzida sobre dez rodadas consecutivas em ambiente Linux com OpenJDK Temurin 17 e Maven 3.9, registrou aprovação em todas as rodadas. Na versão atual, o *jar* está congelado para a nova medição, e os componentes têm suítes de testes automatizados em Java, Python e *shell*, estas com simuladores dos comandos de sistema para rodar sem *cluster*. O *pipeline* de análise estatística foi validado em conjunto sintético de 30 000 operações distribuídas entre sete tipos de operação, com detecção, pelo teste de Mann-Whitney por percentil, do efeito-na-cauda esperado para F1 em relação ao *baseline*. O programa de carga foi executado contra o *cluster* vivo nos cenários F1 e F3, em máquina virtual Linux com o módulo `sch_netem` disponível, com o *pipeline* de análise consumindo os CSV crus de cada repetição, conforme reportado na Seção 4.3. O diagnóstico dessa execução, descrito na mesma seção, mostrou que ela não mediu o ciclo de vida de sessões.

Controlador heurístico: projeto e implementação

O controlador heurístico previsto no Trabalho de Conclusão de Curso I (Seção 2.5) foi projetado e implementado em parte na etapa seguinte, e as configurações pré-definidas tomaram a forma de perfis entre os quais ele alterna. Na terminologia MAPE-K, o monitoramento corresponde ao agregador de janelas e à sonda de topologia, a análise à compa-

ração dos sinais com limiares, o planejamento à máquina de estados que escolhe o perfil e a execução à aplicação do perfil aos parâmetros do cliente, componentes descritos a seguir. Os contraexemplos produzidos pelo TLC (Seção 4.3) orientam a escrita do programa de carga e o desenho do controlador.

O controlador executa como *thread* dentro do processo Java do programa de carga, externo ao *data grid*: não roda no servidor, não reconfigura o servidor, apenas lê a saúde dos nós pela API REST, e não grava estado replicado. O núcleo de decisão fica em um pacote que não depende do Infinispan nem do código de sessão, restrição conferida por teste, de modo que o mesmo código pode decidir ao vivo ou sobre registros gravados. O núcleo usa como tempo o fim de cada janela de observação, e não o relógio do sistema, o que o torna determinístico e testável sem espera real. No desenho aprovado, cada janela gerará uma linha em arquivo JSONL, com os observáveis, os limiares em vigor, a decisão e seus motivos, como trilha de auditoria, e o controlador terá três modos, selecionados por opção de linha de comando: desligado, em que nenhum objeto novo é criado; *shadow*, em que observa e decide sem atuar; e *active*, em que aplica o perfil decidido. A trilha e os modos dependem da ligação ao laço de carga, ainda não implementada.

O monitoramento agrega os eventos do cliente em janelas contíguas de 5 s e consulta a saúde de cada nó a cada 2 s. O Quadro 4 resume os sinais. O sinal de latência é a fração de eventos da janela com latência acima do p99 de referência da operação, obtido de execuções sem falha do mesmo cenário e modo de replicação. Para uma única operação, a fração ultrapassa 1 % exatamente quando o p99 da janela supera o de referência. Somada sobre operações, ela funciona com poucas amostras por operação. Janelas com amostra abaixo do mínimo tornam o sinal indeterminado. As violações de I5 e I6 ficam fora dos gatilhos, porque só são verificáveis sob quiescência, e entram na avaliação como desfecho.

Quadro 4 – Sinais observados pelo controlador heurístico

Sinal	**Definição na janela**	**Papel**
Latência (S_{lat})	fração de eventos com latência acima do p99 de referência da operação; mínimo de 100 eventos	primário; exige 2 janelas seguidas
Erro (S_{err})	fração de eventos com `ERROR_TRANSPORT` ou `ERROR_UNAVAILABLE`; mínimo de 50 eventos	primário; exige 2 janelas seguidas
Topologia (S_{top})	consulta de saúde falhou, menos de três nós ou algum *cache* fora do estado saudável	primário; basta 1 amostra
Violação (S_{viol})	novas violações de I1, I4 ou I2 classificada como real	secundário; severidade máxima

Fonte: O autor

A decisão é uma máquina de dois estados, NORMAL e CONSERVADOR, avaliada uma vez por janela por quatro regras. Pela regra R1, uma violação leva ao perfil conservador imediatamente, mesmo durante o intervalo mínimo entre transições (*cooldown*), porque, em segurança, reagir tarde custa mais que oscilar uma vez. Pela regra R2, fora do *cooldown*, uma anomalia de topologia, ou latência ou erro acima do limiar de entrada em duas janelas seguidas, leva ao perfil conservador. Pela regra R3, no perfil conservador, qualquer sinal alto zera a contagem de janelas calmas. Pela regra R4, o retorno ao perfil normal exige, ao mesmo tempo, *cooldown* expirado, seis janelas calmas seguidas, topologia estável há um intervalo mínimo e nenhuma violação recente. A histerese vem de limiares de saída menores que os de entrada. Os valores atuais são provisórios, fornecidos por arquivo de parâmetros ou pelo conjunto provisório padrão, sem valor fixo na lógica de decisão: limiar de latência de 0,05 na entrada e 0,02 na saída, de erro de 0,02 e 0,005, *cooldown* de 30 s, estabilidade de topologia de 30 s e retenção após violação de 120 s. A calibração final seguirá regra fixada antes de ver os dados de calibração.

O perfil conservador atua por quatro parâmetros do cliente. O primeiro limita a admissão, reduzindo a 50 % o número de operações em curso. O segundo reduz o ritmo, com pausa de 5 ms entre operações em S1 e S2 e tempo de pensar multiplicado por 1,5 em S3. O terceiro adia as escritas administrativas de S3 (bloqueio, desbloqueio e alterações do contador) até a saída do perfil conservador ou até 30 s, sem descartá-las. O quarto dobra a espera base entre retentativas de S3, com variação aleatória de $\pm 50\,\%$, sem mudar o número de retentativas. Todos apenas escolhem quando o cliente emite uma operação que já emitiria: nenhuma operação é criada ou suprimida, nenhum estado replicado é tocado e a configuração do servidor não muda. Essa restrição mantém os comportamentos do sistema dentro dos admitidos pelo modelo formal e preserva o modo de replicação como variável do experimento. O modo de replicação e o tempo-limite do cliente Hot Rod ficaram fora desta primeira etapa porque alteram o sistema medido ou exigem reconstruir o cliente.

Estão implementados e testados na versão local do código que integra esses componentes, com relógio simulado e capturas reais da saúde do *cluster*: o núcleo de decisão, com as regras, a histerese e o *cooldown*; o agregador de janelas, ligado ao registro de eventos e ao auditor por um ponto de escuta opcional que, desligado, deixa a saída do programa idêntica à anterior; a sonda de topologia; o controle de emissão de operações, com os quatro parâmetros; o coletor de recursos; e os *scripts* de análise por fase, de referência de p99, de estimativa de energia e de comparação pareada entre braços. Falta ligar

o controlador ao laço de carga: a interface de linha de comando ainda não tem a opção que o ativa, e por isso o controlador não foi executado com o programa de carga e não há resultado sobre seu efeito. Também faltam a reprodução das decisões sobre registros gravados, os marcadores de início e fim de F1 e F3 nos registros, uma proteção contra parâmetros de energia inválidos, o pré-registro com execução curta de teste, a calibração e a avaliação descrita na Seção 3.2.

A atuação do controlador sobre a configuração do *cluster*, prevista no objetivo específico (4), é uma segunda etapa planejada, posterior à ligação do controlador ao laço de carga e à avaliação da atuação no cliente. Ela vem depois por três razões. Durante a nova medição de referência e a campanha com o cenário S3, a configuração do servidor é variável do experimento e não pode mudar. Só a avaliação da etapa cliente indica se atuar no servidor acrescenta algo. Por fim, parte das ações possíveis produz comportamentos que o modelo formal não admite.

A etapa começa por uma verificação, na documentação e em *cluster* local, de quais atributos do Infinispan 15 podem ser alterados em tempo de execução sem reiniciar o *cache*. Os candidatos iniciais, hipótese de partida ainda não verificada, são o tempo-limite de chamadas remotas e o tempo-limite de aquisição de trava, que só alteram quando uma operação termina; a suspensão do rebalanceamento, que altera o momento da convergência; e a mudança forçada da disponibilidade de uma partição, que permitiria escrita na partição minoritária, comportamento fora do modelo. O modo de replicação, o número de cópias, o número de segmentos e a política de partição e de fusão devem exigir a recriação do *cache*, com perda dos dados, o que a mesma verificação vai confirmar, e ficam fora. A primeira etapa permanece restrita ao cliente e é avaliada como tal; a segunda terá pré-registro e avaliação próprios. A lista final de parâmetros, a admissão de ações que alteram a convergência ou a semântica sob partição, estas com extensão do modelo TLA+, e o tempo de máquina dependem de decisão posterior do autor.

4.5 Resultados parciais e limitações conhecidas

O resultado parcial central desta etapa é a coerência entre o conjunto de invariantes alvo, o material teórico levantado pelo mapeamento sistemático e os contraexemplos produzidos pela verificação formal. A síntese inicial do mapeamento sustenta a contribuição pretendida do trabalho em duas dimensões complementares. A primeira diz respeito à métrica de avaliação. Nos cerca de trinta trabalhos lidos na íntegra até esta etapa, a taxa

de violação de invariantes não aparece como métrica primária de avaliação experimental, pois a literatura se concentra em métricas de desempenho agregadas, como latência média, vazão e percentis de cauda, e em métricas de disponibilidade do sistema; a classificação dos 165 estudos do mapeamento ainda precisa confirmar essa constatação. A segunda diz respeito ao método. A especificação formal acoplada a invariantes específicos do domínio de sessão não é tratada como objeto central no mesmo conjunto de fontes, e o ponto comparativo mais próximo encontrado, Nguyen *et al.* (2019), combina consistência relaxada com monitoramento e recuperação reativos, sem verificação formal a montante. Os contraexemplos de I1 e I2 obtidos pelo TLC, comentados na Seção 4.3, fornecem o catálogo inicial de padrões de violação que o controlador heurístico previsto para o Trabalho de Conclusão de Curso II deve mitigar. Na forma global, porém, as violações de I1 e I2 são consequência direta da divergência transitória entre réplicas e não informam sobre o protocolo; as propriedades avaliadas réplica a réplica, descritas na mesma seção, separam esse efeito. Na etapa atual, além da inclusão de falhas no modelo formal (Seção 4.2), os injetores das quatro classes de falha foram implementados e a carga foi corrigida depois do diagnóstico da primeira bateria. O controlador está projetado e implementado em parte, ainda sem ligação ao laço de carga (Seção 4.4). Sua avaliação está desenhada (Seção 3.2), mas não foi pré-registrada nem executada. O pré-registro, com os valores finais, vai preceder qualquer execução.

Três limitações conhecidas devem ser registradas. A primeira é a execução parcial dos artefatos: A vertente formal completou as cinco rodadas planejadas, e o *baseline* experimental teve os cenários de parada ordenada de nó (F1) e de atraso com *jitter* de canal (F3) executados em piloto reduzido sobre máquina virtual Linux, cujo diagnóstico, na Seção 4.3, mostrou que a bateria não mediu o ciclo de vida de sessões; a bateria plena nos parâmetros T9 e T10 segue como protocolo a escalar a partir do piloto, e a nova medição de referência que a substitui foi pré-registrada e ainda não foi executada. O pré-registro do protocolo (NOSEK *et al.*, 2018), feito em outubro de 2026 e acrescido de dois complementos antes de qualquer execução, não admite ajuste depois de observados os dados. O piloto da nova medição, que precede a campanha, também não foi executado. A segunda é a abstração da rede no modelo formal, que, no Trabalho de Conclusão de Curso I, capturava atraso e reordenação mas não modelava perdas permanentes nem partições, escopo da evolução prevista para o Trabalho de Conclusão de Curso II. Na versão atual, o modelo inclui queda, recuperação, partição e fusão como ações, e a perda de mensagens foi verificada apenas em cópia separada da especificação, sem retransmissão (Seção 4.2). A

terceira é a execução do mapeamento sistemático em uma única plataforma agregadora, com a complementação por busca manual nas bases originais formalizada entre as ameaças à validade na Seção 3.1 e integrada às atividades remanescentes para o Trabalho de Conclusão de Curso II.

Revisões em relação ao planejamento

O plano da Proposta e do Trabalho de Conclusão de Curso I sofreu revisões durante a execução. Esta subseção registra, para cada revisão, o plano, a mudança e o motivo, e remete às seções com os detalhes técnicos. A renovação de sessão constava do objetivo da Proposta, mas não entrou nas sete operações (O1 a O7, Seção 3.2) definidas no recorte do Trabalho de Conclusão de Curso I. Como ficou fora do escopo desde esse recorte, a ausência não resulta de revisão durante a execução, e nem o modelo formal nem o programa de carga têm operação de renovação. Os objetivos específicos (Seção 1.3) foram revistos em três pontos. O objetivo (3) passou a prever a arquitetura experimental nos dois modos de replicação, síncrono e assíncrono, com injeção de queda de nó, partição, atraso e degradação parcial da rede. O objetivo (4) foi dividido em duas etapas: a primeira atua apenas sobre parâmetros do cliente, e a atuação sobre a configuração do *cluster* é uma segunda etapa planejada, posterior à avaliação da primeira (Seção 4.4). O objetivo (5) passou a incluir, além das métricas previstas, o tempo de CPU por operação útil, com a estimativa de energia correspondente, ausente do plano original (Seção 3.2).

Na verificação formal, o plano previa um modelo sem falhas no Trabalho de Conclusão de Curso I e a inclusão de falhas no Trabalho de Conclusão de Curso II. As duas etapas foram cumpridas. As revisões de estrutura feitas ao longo da execução, com seus motivos, estão na Seção 4.2, e seus efeitos sobre os resultados de I1 a I6, na Seção 4.3.

Na vertente experimental, quatro revisões alteram a leitura dos dados. A primeira é o modo de replicação: a linha T1 da Tabela 3 fixa `DIST_ASYNC` como alvo, mas todas as execuções na máquina virtual até outubro de 2026 ocorreram em `DIST_SYNC` (Seção 4.3), achado que levou aos *caches* assíncronos (Seção 4.4) e à medição dos dois modos na mesma campanha. A segunda é a carga, que na bateria de junho não criou sessões e foi corrigida (Seção 3.2) antes de uma nova medição de referência pré-registrada. O cenário S3, acrescentado em outubro de 2026 e externo à nova medição, exercita o bloqueio administrativo e a escrita em nó isolado (Seção 3.2). A terceira é a falha F1. Em outubro de 2026, antes de qualquer execução da campanha, a queda silenciosa descrita na

Seção 3.2 substituiu o `podman stop` usado na bateria de junho e nas verificações intermediárias, porque a parada ordenada não exercita a detecção de falhas. Com `SIGTERM`, o servidor fecha o canal JGroups e pede a saída ao coordenador, que instala a nova visão sem que nenhum detector suspeite do nó (BAN, 2026, seções 3.10.13 e 6.8.7). Na queda silenciosa, a interface de rede é desativada antes do `SIGKILL` porque, com as conexões TCP fechadas pelo sistema operacional, o FD_SOCK2 detectaria a queda em cerca de um segundo, e a detecção passaria a não depender do FD_ALL3 (Seção 3.2). O estado do ensaio desse modo está na Seção 4.3. A quarta é o conjunto de falhas, tratado adiante.

O protocolo da nova medição também se afasta da Tabela 3 em pontos declarados. A justificativa de 10^6 operações por execução e de 30 repetições, dada na tabela, refere-se ao desenho original e não se aplica ao protocolo pré-registrado em outubro de 2026. As dez repetições por célula substituem as 30 de T10, por limite de tempo de máquina, com a execução como unidade de análise. Essa redução diminui o poder estatístico dos contrastes entre células, de modo que diferenças pequenas entre condições podem não ser detectadas. A duração fixa de 300 s substitui o volume de 10^6 operações de T9 e as durações de T14, e o aquecimento mínimo passa a 30 s (T11). A bateria de junho e as verificações intermediárias usaram `numSegments` igual a 64, e não 256 como em T4, e a nova medição mantém esse valor. Ele veio da configuração inicial do *cluster*, sem fonte que o justifique, e foi mantido para preservar a comparabilidade entre as baterias. A carga não segue o alvo de 75 % do pico de T12: o programa de carga opera em laço fechado, com quatro *threads* que emitem a operação seguinte assim que a anterior termina, sem limitação de vazão. A vazão de cada execução é medida e reportada, e não imposta. Os afastamentos de T4 e de T12 foram declarados em complemento ao registro do protocolo, antes da coleta. Na nova medição, a análise de T22 passa a reamostrar execuções no *bootstrap* e a aplicar o teste de Mann-Whitney, como análise secundária, sobre os p99 por execução, e não sobre eventos agrupados de várias execuções.

A quarta revisão da vertente experimental é o conjunto de falhas. As três classes de evento planejadas na Introdução (Capítulo 1) deram lugar a quatro classes: além da queda de um nó seguida de *failover* e da partição de rede, a terceira é o atraso com *jitter* no canal entre os nós, e a quarta é a degradação parcial da rede, definida adiante, no lugar da mudança de topologia com *state transfer* do modelo de falhas planejado (Seção 3.2). F2 deixou de ser reservada e ganhou injetor. Em F3, a distribuição lognormal do plano (T18) deu lugar à normal na execução, porque o `tc-netem` traz tabelas prontas para distribuições como a normal e a de Pareto, mas não para a lognormal, que exigiria gerar

e instalar um arquivo próprio no ambiente. A calibração está descrita na Seção 4.3, e a lognormal fica como análise de sensibilidade. Essa calibração é do atraso por pacote, e não da latência das operações, que ainda não foi medida sob F3 com carga válida.

Em outubro de 2026, o autor decidiu testar, além da partição total, a perda de pacotes e a degradação parcial, e o rótulo F4 passou a designar a degradação parcial da rede. Falhas de rede reais nem sempre cortam toda a comunicação: ao lado de partições completas, o levantamento de Bailis e Kingsbury (2014, p. 50–53) relata perdas de 50 % a 100 % dos pacotes, perturbações que afetam só alguns grupos de nós e uma placa de rede que descartava os pacotes recebidos, mas não os enviados. Na nova definição de F4, a comunicação entre o nó alvo e seus pares sofre perda, atraso, duplicação ou reordenação de pacotes, de forma simétrica ou assimétrica, sem corte completo. O tráfego do cliente não é afetado, porque o filtro casa apenas os endereços dos pares do *cluster*. As severidades de perda vão de N1 a N5 (1 %, 5 %, 20 %, 50 % e 99,9 %), e a mais alta deve aproximar uma partição, hipótese a confirmar experimentalmente. O rebalanceamento e o *state transfer*, que alteram o mapeamento entre chaves e nós (Infinispan Project, 2026c), continuam presentes como consequência da reentrada do nó em F1 e da fusão em F2, mas não têm injetor próprio.

No controlador, o plano previa um processo externo que lesse observáveis em arquivo e passasse ao perfil conservador quando a taxa de violação superasse um limiar. O projeto aprovado em outubro de 2026 o executa como *thread* dentro do processo do programa de carga (Seção 4.4), porque o observável primário é por evento e a serialização para outro processo custaria CPU, que é métrica avaliada. O gatilho primário passou a combinar latência de cauda relativa, taxa de erro e topologia, e as violações ficaram como gatilho secundário, por três motivos. Primeiro, I5 e I6 só são verificáveis sob quiescência. Segundo, o bloqueio por limite produziria registros de I2 esperados por desenho (Seção 4.3). Terceiro, nas paradas de nó da bateria de junho nenhum erro foi registrado, o que sugere que a detecção dependa de latência e topologia. Essa observação tem valor apenas indicativo, porque a bateria não criou sessões (Seção 4.3). A atuação sobre a configuração do *cluster*, prevista na Proposta e no objetivo (4), ficou para a segunda etapa mencionada acima.

Os critérios de avaliação do controlador também mudaram. O desenho aprovado em outubro de 2026 mantém a margem de 10 % e a rastreabilidade, mas troca a unidade de análise, do evento para a execução, e substitui o teste de Mann-Whitney, previsto na Seção 3.2, pelo desenho em blocos pareados da Seção 3.2. O teste aplicado a eventos

trata como independentes observações de uma mesma execução e é sensível a diferenças no corpo da distribuição, não só na cauda. O teste de Mann-Whitney sobre os p99 por execução permanece apenas na nova medição de referência, como análise secundária.

5 CONSIDERAÇÕES PARCIAIS

O trabalho avançou ao longo do Trabalho de Conclusão de Curso I em três linhas de trabalho complementares. A primeira foi a delimitação do problema, com a formulação da questão central, dos invariantes alvo e dos critérios de avaliação a partir da literatura normativa e da Proposta original do trabalho. A segunda foi o mapeamento sistemático da literatura, que produziu um protocolo replicável, executou a busca em uma plataforma agregadora e gerou um conjunto de 165 estudos primários para leitura completa, com o esquema de classificação descrito na Seção 3.1. A terceira foi a construção dos artefatos da vertente experimental e a execução da verificação formal: o repositório `tcc-desenvolvimento` versiona o *cluster* Infinispan parametrizado, o programa de carga em Java sobre Hot Rod, a especificação TLA+ `SessionStore.tla` com os invariantes I1 a I6 e o *pipeline* de análise estatística em Python, todos detalhados no Capítulo 4. A verificação formal foi executada em cinco rodadas do TLC, com contraexemplos produzidos para I1, I2, I5 e I6 e ausência de violação para I3 e I4 nas configurações exploradas, conforme reportado na Seção 4.3. Os cenários de parada ordenada de nó (F1, por `podman stop` e `podman start`) e de atraso com *jitter* de canal (F3) do *baseline* experimental foram executados contra o *cluster* vivo nesta etapa, em máquina virtual Linux. Essa primeira bateria de medição, de junho, não mediu o ciclo de vida de sessões, por defeitos da carga e da análise identificados em outubro (Seção 4.3).

Na versão parcial do Trabalho de Conclusão de Curso II, o trabalho chega com a vertente formal em estágio mais avançado que a experimental. A Seção 4.5 reúne as mudanças em relação ao plano original e seus motivos. Na vertente formal, o modelo TLA+ do protocolo de sessão inclui quedas, recuperações e partições. Na verificação pelo TLC, as fórmulas globais de I1 e I2 são violadas por construção em um modelo com várias réplicas, o que levou a propriedades avaliadas réplica a réplica. No cenário semeado, a irreversibilidade da invalidação por réplica só foi violada sob partição, coerente com a perda, na fusão, de escritas feitas na partição menor. Com *login* habilitado, ela também é violada sem falha, por artefatos do modelo. As formas de I3 e I4 declaradas no modelo, de faixa e de tipo, não foram violadas, com verificação exaustiva apenas na menor configuração, `tiny`, acrescentada nas verificações de setembro de 2026, depois das cinco rodadas do Trabalho de Conclusão de Curso I, e a imutabilidade do motivo, avaliada réplica a réplica, é violada. I5 e I6 são violados por dois mecanismos: o empate de marcas de tempo, defeito do modelo cuja correção está pendente, e, sob partição, a perda, na fusão, de uma

escrita ainda não propagada, com divergência permanente (Seção 4.3).

Na vertente experimental, o ambiente está montado e documentado (Seção 4.1): o *cluster* de três nós, o programa de carga com os cenários S1, S2 e S3, os injetores das quatro classes de falha e os *scripts* de análise. Os defeitos da primeira bateria foram corrigidos, e uma nova medição de referência, com os modos síncrono e assíncrono, foi pré-registrada, mas ainda não executada. Todas as execuções na máquina virtual até aqui ocorreram em `DIST_SYNC`, de modo que o comportamento do modo assíncrono, alvo do trabalho, ainda não foi medido. O injetor de partição foi verificado sem carga de aplicação, e o de degradação parcial passou apenas pelas verificações que não afetam o tráfego, sem injeção real até aqui.

O controlador heurístico está projetado e implementado em parte (Seção 4.4). O gatilho primário combina latência de cauda relativa, taxa de erro e topologia, e as violações de I1, I2 e I4 formam o gatilho secundário. A convergência (I5 e I6) entra apenas como desfecho. A primeira etapa atua apenas sobre parâmetros do cliente, e a atuação sobre a configuração do *cluster* é uma segunda etapa planejada, cujos parâmetros candidatos ainda não foram verificados. O controlador ainda não foi ligado ao laço de carga, e sua avaliação, desenhada em blocos pareados (Seção 3.2), depende da nova medição de referência e da campanha com S3.

O conjunto de invariantes alvo, irreversibilidade de invalidação por *logout*, consistência de bloqueio, monotonicidade local do contador, unicidade do motivo de invalidação, convergência do estado lógico e convergência do contador, encontra respaldo direto em Temoshok *et al.* (2025) e em OWASP Foundation (2026) para as propriedades de sessão e na literatura formal sobre replicação otimista, em particular Shapiro *et al.* (2011) e Nair, Petri e Shapiro (2020), para as propriedades de convergência. A escolha do par de métricas latência de cauda e taxa de violação como contribuição principal é sustentada pela síntese inicial do mapeamento sistemático: a primeira métrica é o padrão consolidado da literatura para sistemas com cauda longa, e a segunda métrica não aparece como métrica primária de avaliação experimental nos cerca de trinta trabalhos lidos na íntegra até esta etapa, constatação que a classificação dos 165 estudos do mapeamento ainda precisa confirmar. O ponto comparativo mais próximo, Nguyen *et al.* (2019), combina consistência relaxada com monitoramento e recuperação reativos, sem verificação formal a montante. Os contraexemplos do TLC fornecem o catálogo inicial de padrões de divergência a serem mitigados pelo controlador heurístico previsto para a etapa seguinte. Até a correção do empate de marcas de tempo, os contraexemplos de I5 e I6 que não passam

pela fusão não informam sobre o protocolo.

Três limitações merecem registro nesta etapa. A primeira é a execução do mapeamento sistemático restrita a uma plataforma agregadora, com a complementação por busca manual nas bases originais formalizada como ameaça à validade na Seção 3.1 e integrada às atividades remanescentes para o Trabalho de Conclusão de Curso II. A segunda é a leitura na íntegra das referências citadas, que não alcançou todas. Por restrição de acesso no período, oito das fontes citadas foram consultadas apenas pelo resumo ou por trechos: Newcombe (2014), Liu *et al.* (2015), Naskos, Gounaris e Katsaros (2017), Wu, Sreekanti e Hellerstein (2020), Gambi, Toffetti e Pezzè (2013), Rosa, Rodrigues e Lopes (2013), Enea *et al.* (2024) e BEN MOKHTAR *et al.* (2024); as afirmações que dependem delas se limitam ao que os resumos sustentam. A resolução final dessas exceções compõe a curadoria bibliográfica do Trabalho de Conclusão de Curso II. A terceira é a execução parcial dos artefatos. A vertente formal completou as cinco rodadas planejadas do TLC nas três configurações da Tabela 3 (T21); a configuração `tiny` foi acrescentada depois, nas verificações de setembro de 2026 (Seção 4.3). A configuração do *cluster*, o programa de carga em Java, os *scripts* de injeção de falhas e o *pipeline* de análise estatística estão implementados. O coletor de métricas OpenMetrics (OpenMetrics Project, 2024) foi escrito, mas não foi integrado ao código principal. O *baseline* experimental teve os cenários de parada ordenada de nó (F1) e de atraso com *jitter* de canal (F3) executados contra o *cluster* vivo, em máquina virtual Linux com o módulo `sch_netem` disponível, na forma de um piloto reduzido em relação à bateria plena dos parâmetros T9 e T10, que segue como protocolo a escalar. Essa bateria não mediu o ciclo de vida de sessões, e nenhum número de latência de junho é usado como resultado (Seção 4.3).

No Trabalho de Conclusão de Curso I, a abstração do canal de replicação, no modelo formal, capturava atraso e reordenação mas não modelava perdas permanentes ou particionamento. Essa simplificação era proposital naquela etapa, dado que a verificação visava o fluxo nominal sob entrelaçamento concorrente; o refinamento com falhas explícitas integrava o plano para o Trabalho de Conclusão de Curso II (Apêndice A, etapas 3 e 4). No Trabalho de Conclusão de Curso II, o modelo ganhou as ações de queda, recuperação, partição e fusão (Seção 4.2). O canal do modelo não perde mensagens na versão integrada, e a variante com perda tem caráter ilustrativo.

Limitações e ameaças à validade

Na vertente formal, os resultados de I3 e I4 nas configurações maiores valem só até o tempo-limite das execuções, e as cardinalidades verificadas são pequenas, recorte apoiado pela observação de que 98 % das falhas de produção analisadas se manifestam com três nós ou menos (YUAN *et al.*, 2014), mas que não exclui defeitos em escalas maiores. Os contraexemplos de I5 e I6 que não passam pela fusão decorrem do empate de marcas de tempo e não terão interpretação enquanto ele não for corrigido.

Na vertente experimental, nenhum dado de latência ou de violação é ainda utilizável: a bateria de junho não criou sessões, e a nova medição não foi executada. Os F1 já executados foram paradas ordenadas, e não quedas silenciosas, e o modo silencioso não foi ensaiado na máquina virtual. A partição foi exercida apenas sem carga de aplicação, e o tempo de recomposição da segunda repetição, de 188 s, obtido em condições diferentes das demais (*cluster* com histórico e coleta de monitoramento intensa), tem causa não isolada entre duas hipóteses registradas. A nova medição usa dez repetições por célula, e não as trinta do plano, o que reduz o poder estatístico. Ela mantém ainda 64 segmentos, sem fonte que justifique o valor, e usa distribuição normal em F3, no lugar da lognormal. Os parâmetros do cenário S3 são premissas de modelagem, e a estimativa de energia depende de um modelo de potência e do tempo de CPU virtual, com as limitações da Seção 3.2.

O ambiente de execução é uma ameaça à validade de outra natureza. O computador hospedeiro registrou dezessete desligamentos não ordenados desde setembro de 2026, dois deles com parada do sistema por erro registrada. O autor atribui os desligamentos a quedas no fornecimento de energia elétrica na região. Uma dessas quedas, em 18 de outubro, desfez o *cluster* entre duas etapas da sequência de verificações. O histórico que uma das repetições de partição pretendia preservar se perdeu, e a repetição foi redefinida. Outra queda, em 28 de outubro, interrompeu a primeira tentativa dessa repetição. As regras pré-registradas da nova medição tratam execuções interrompidas, que são descartadas e repostas, mas não compensam o atraso que novas quedas imporiam ao cronograma.

5.1 Reflexões sobre as atividades do Trabalho de Conclusão de Curso I

O cronograma apresentado (Tabela 5), referente à disciplina de Trabalho de Conclusão de Curso I, foi planejado com espaço de tempo de 12 semanas, organizado em seis etapas quinzenais.

Tabela 5 – Cronograma do Trabalho de Conclusão de Curso I

Tópico **Semanas:**	**1–2**	**3–4**	**5–6**	**7–8**	**9–10**	**11–12**
Definir pergunta de pesquisa e escopo	X					
Protocolo do mapeamento sistemático (bases, strings, critérios)	X					
Busca e triagem (título/resumo)		X				
Seleção por leitura completa, extração e classificação			X			
Síntese do mapeamento e recorte do protótipo			X			
Preparar ambiente experimental com Infinispan e observabilidade mínima				X		
Implementar programa de carga e checagens de invariantes				X		
Modelo formal com transições e canal assíncrono				X		
Model checking e análise de contraexemplos					X	
Baseline experimental com falhas selecionadas e métricas p95/p99					X	
Consolidação de resultados parciais e ajustes						X
Escrita da monografia	X	X	X	X	X	X

Fonte: O autor

Nas semanas 1 e 2, o objetivo era estabelecer a pergunta de pesquisa, o protocolo do mapeamento sistemático e o esquema inicial de extração e classificação. Nas semanas 3 e 4, a meta era executar a busca e a triagem inicial, e estabilizar os critérios de inclusão e exclusão. Nas semanas 5 e 6, a intenção era concluir a seleção por leitura completa, extrair dados e produzir a síntese do mapeamento, e definir o recorte implementável do protótipo. Nas semanas 7 e 8, o foco seria na preparação do ambiente experimental, com Infinispan, programa de carga e instrumentação mínima, e iniciar a especificação formal do modelo com invariantes básicas. Nas semanas 9 e 10, seria realizada a execução do *model checking* inicial, sucedida por um *baseline* experimental sem e com falhas selecionadas para obter dados preliminares. Nas semanas 11 e 12, previa-se consolidar resultados parciais, ajustar o recorte conforme evidências e finalizar a escrita do trabalho com metodologia, resultados parciais e planejamento das próximas etapas. A escrita do texto ocorreria em paralelo durante todo o período definido para o trabalho.

Algumas atividades desviaram do previsto e merecem comentário. O mapeamento sistemático, planejado para as semanas 5 e 6 até a extração e a classificação por leitura completa, avançou nesta etapa apenas até a seleção e a triagem: a busca correu por uma plataforma agregadora única (OpenAlex), ao passo que a consulta manual às bases originais e a classificação dos 165 estudos primários pelas dimensões D1 a D8 permaneceram para o Trabalho de Conclusão de Curso II. A leitura das fontes também ficou parcial,

com cerca de trinta trabalhos lidos na íntegra; parte das fontes hospedadas atrás de autenticação foi citada a partir de resumos institucionais, e a consulta ao texto integral foi remetida à etapa seguinte. A leitura integral concentrou-se nas referências que sustentam a fundamentação e o desenho experimental. A consulta às bases originais (ACM Digital Library, IEEE Xplore, Scopus e SpringerLink) será feita por execução manual via Portal de Periódicos CAPES. O ambiente experimental registrou o desvio de maior impacto operacional: o piloto de F1 correu sobre WSL2, mas o cenário F3 exigiu o módulo `sch_netem`, ausente nesse ambiente, o que motivou a migração para uma máquina virtual Linux (Hyper-V, Ubuntu Server 24.04), decisão não antecipada no planejamento inicial.

As demais atividades cumpriram o previsto, com ressalvas de escopo. O *model checking* concluiu as cinco rodadas planejadas do TLC, com contraexemplos para I1, I2, I5 e I6 e ausência de violação para I3 e I4 nas configurações exploradas. O *baseline* experimental foi conduzido como piloto reduzido, limitado aos cenários de parada ordenada de nó (F1) e de atraso com *jitter* de canal (F3), cujo diagnóstico consta da Seção 4.3, e não como a bateria plena dos parâmetros T9 e T10 (30 repetições de 10^6 operações), que segue como protocolo a escalar. Os cenários de partição (F2) e de *state transfer* induzido (F4) ficaram reservados, no planejamento do Trabalho de Conclusão de Curso I, ao Trabalho de Conclusão de Curso II. O cronograma do Trabalho de Conclusão de Curso I foi cumprido com duas ressalvas: a leitura completa e a classificação dos estudos do mapeamento não foram concluídas, e o *baseline* das semanas 9 e 10 mostrou-se inválido como medida (Seção 4.3). No Trabalho de Conclusão de Curso II, o conjunto de falhas foi revisto (Seção 4.5).

5.2 Próximas etapas e plano para o Trabalho de Conclusão de Curso II

O planejamento para a continuação do trabalho está organizado conforme a Tabela 6 e o detalhamento das atividades previstas para o Trabalho de Conclusão de Curso II consta do Apêndice A.

Nas semanas 1 e 2, o objetivo é consolidar o modelo formal do sistema de sessões com canal assíncrono, formalizar o conjunto inicial de invariantes e executar a primeira rodada de *model checking* para obtenção de contraexemplos. Nas semanas 3 e 4, o objetivo é evoluir o modelo, incorporar falhas explícitas, refinar as invariantes e analisar os contraexemplos gerados, e delimitar quais padrões de falha devem ser reproduzidos

Tabela 6 – Cronograma do Trabalho de Conclusão de Curso II

Tópico **Semanas:**	**1–2**	**3–4**	**5–6**	**7–8**	**9–10**	**11–12**
Modelo formal: transições de sessão e canal assíncrono; invariantes iniciais; execução inicial de *model checking*	X					
Modelo formal: inclusão de falhas; refinamento de invariantes; análise de contraexemplos		X				
Baseline com falhas: cenários; cálculo de p95/p99, erro e violações			X			
Baseline com falhas: cenário de partição e, quando aplicável, rebalanceamento com *state transfer*; consolidação do conjunto			X			
Alinhamento formal-experimental: mapear contraexemplos para padrões observáveis; ajustes mínimos e reexecuções pontuais				X		
Controlador heurístico: definição de observáveis, regras com histerese e *cooldown* e perfis normal e conservador				X		
Execução com controlador: repetir cenários do *baseline*; coletar métricas equivalentes; registrar decisões do controlador					X	
Análise comparativa: *baseline vs.* controlador e correlação com contraexemplos formais					X	
Escrita da monografia	X	X	X	X	X	X
Consolidação final da monografia						X

Fonte: O autor

experimentalmente. Nas semanas 5 e 6, o objetivo é executar o *baseline* experimental sob falhas selecionadas (parada silenciosa de nó com reentrada, latência e *jitter*, partição e, quando aplicável, rebalanceamento com *state transfer*), calcular métricas como p95 e p99, taxa de erro, tempo de recuperação e taxa de violações, e consolidar o conjunto de referência. Nas semanas 7 e 8, o objetivo é alinhar modelo formal e experimento, mapear contraexemplos para padrões observáveis, realizar ajustes mínimos no programa de carga e na instrumentação para comparabilidade, e implementar o controlador heurístico integrado ao ambiente. Nas semanas 9 e 10, o objetivo é executar os mesmos cenários do *baseline* com o controlador, coletar métricas equivalentes e realizar a análise comparativa, com correlação entre os resultados experimentais e os contraexemplos formais. Nas semanas 11 e 12, o objetivo é consolidar a avaliação crítica, sistematizar limitações e conclusões, e finalizar a escrita do trabalho com descrição do método, resultados, discussão e preparação para entrega e defesa. A escrita do texto ocorre em paralelo durante todo o período definido para o trabalho.

O cronograma da Tabela 6 foi modificado durante a execução. No Trabalho de

Conclusão de Curso II, as etapas das semanas 1 a 4 foram cumpridas, com a inclusão de falhas no modelo. A etapa das semanas 5 e 6 deu lugar à correção da carga e ao pré-registro da nova medição de referência, ainda não executada. O controlador, previsto para as semanas 7 e 8, foi projetado e implementado em parte antes da medição de referência, para que seu desenvolvimento não dependesse da máquina virtual. As etapas restantes, sem datas fixas, estão no Apêndice A.

Em síntese, a primeira evolução prevista era a inclusão de falhas explícitas no modelo formal, com análise dos contraexemplos gerados e mapeamento desses contraexemplos para padrões observáveis nos experimentos. A segunda evolução era a expansão do *baseline* experimental para os cenários de partição e *state transfer* induzido, e a coleta do conjunto de referência completo. A terceira evolução era o desenho e a implementação do controlador heurístico, com regras determinísticas com histerese e intervalo mínimo entre acionamentos, atuando sobre um conjunto finito de perfis pré-definidos. A última etapa era a análise comparativa entre o *baseline* e a configuração com o controlador, com correlação entre os contraexemplos formais e os padrões empíricos. Desse plano, as falhas já estão no modelo formal e o controlador está projetado e implementado em parte. A segunda evolução foi modificada: o *state transfer* não terá injetor próprio, e a partição e a degradação parcial serão exercidas sob carga na campanha com o cenário S3 (Seção 4.5). O parágrafo seguinte reúne o que falta, inclusive a análise comparativa.

As etapas restantes, detalhadas no Apêndice A, seguem uma ordem de dependência. Na máquina virtual, falta concluir as verificações dos injetores, estabilizar o sistema operacional contra atualizações automáticas, ensaiar o F1 silencioso e recriar o *cluster* com os quatro *caches*. Em seguida, vêm o piloto e a campanha da nova medição de referência, que produzirão as primeiras tabelas de latência e de violação válidas, nos dois modos. Depois, a campanha com o cenário S3 exerce F2 e F4 sob carga. Em paralelo, o controlador precisa ser ligado ao laço de carga, calibrado e avaliado conforme o desenho da Seção 3.2, e a segunda etapa, de atuação sobre a configuração do *cluster*, começa pela verificação de quais parâmetros podem mudar em tempo de execução. Na vertente formal, faltam a correção de I5 e I6 e o alinhamento entre os contraexemplos e os padrões observados nos experimentos. No mapeamento, faltam a classificação dos estudos e o fluxograma definitivo.

REFERÊNCIAS

Apache Software Foundation. **Apache Maven**. 2026. Website. Acesso em: 30 jun. 2026. Available from Internet: <https://maven.apache.org/>.

ARMBRUST, M. *et al.* A view of cloud computing. **Communications of the ACM**, v. 53, n. 4, p. 50–58, 2010. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/1721654.1721672>.

AVIŽIENIS, A. *et al.* Basic concepts and taxonomy of dependable and secure computing. **IEEE Transactions on Dependable and Secure Computing**, v. 1, n. 1, p. 11–33, 2004. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1109/TDSC.2004.2>.

BAILIS, P.; KINGSBURY, K. The network is reliable. **Communications of the ACM**, v. 57, n. 9, p. 48–55, 2014. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/2643130>.

BAN, B. **Reliable group communication with JGroups: JGroups Manual, version 5.x**. 2026. Website. Acesso em: 3 nov. 2026. Available from Internet: <http://www.jgroups.org/manual5/index.html>.

BARROSO, L. A.; HÖLZLE, U. The case for energy-proportional computing. **Computer**, v. 40, n. 12, p. 33–37, 2007. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1109/MC.2007.443>.

BEN MOKHTAR, S. *et al.* Taming tail latency in key-value stores: A scheduling perspective. In: SOUSA, L.; ROMA, N.; TOMÁS, P. (Ed.). **Euro-Par 2021: Parallel Processing**. Cham: Springer, 2021. (Lecture Notes in Computer Science, v. 12820), p. 136–150. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-030-85665-6_9>.

BEN MOKHTAR, S. *et al.* A scheduling framework for distributed key-value stores and its application to tail latency minimization. **Journal of Scheduling**, v. 27, n. 2, p. 183–202, 2024. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/s10951-023-00803-8>.

BROWN, L. D.; CAI, T. T.; DASGUPTA, A. Interval estimation for a binomial proportion. **Statistical Science**, v. 16, n. 2, p. 101–133, 2001. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1214/ss/1009213286>.

CHANDRA, T. D.; TOUEG, S. Unreliable failure detectors for reliable distributed systems. **Journal of the ACM**, v. 43, n. 2, p. 225–267, 1996. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/226643.226647>.

COOPER, B. F. *et al.* Benchmarking cloud serving systems with YCSB. In: **Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC '10)**. Indianapolis, IN, USA: ACM, 2010. p. 143–154. Acesso em: 25 maio 2026. Available from Internet: <https://doi.org/10.1145/1807128.1807152>.

DEAN, J.; BARROSO, L. A. The tail at scale. **Communications of the ACM**, v. 56, n. 2, p. 74–80, 2013. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1145/2408776.2408794>.

DECANDIA, G. *et al.* Dynamo: Amazon's highly available key-value store. In: **Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)**. Stevenson, WA, USA: ACM, 2007. p. 205–220. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1145/1294261.1294281>.

EFRON, B.; TIBSHIRANI, R. J. **An Introduction to the Bootstrap**. New York: Chapman & Hall, 1993.

ENEA, C. *et al.* Model checking distributed protocols in Must. **Proceedings of the ACM on Programming Languages**, v. 8, n. OOPSLA2, p. 1900–1927, 2024. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/3689778>.

FAN, X.; WEBER, W.-D.; BARROSO, L. A. Power provisioning for a warehouse-sized computer. In: **Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA '07)**. San Diego, CA, USA: ACM, 2007. p. 13–23. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1145/1250662.1250665>.

FISCHER, M. J.; LYNCH, N. A.; PATERSON, M. S. Impossibility of distributed consensus with one faulty process. **Journal of the ACM**, v. 32, n. 2, p. 374–382, 1985. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/3149.214121>.

FISHER, R. A. **The Design of Experiments**. Edinburgh: Oliver and Boyd, 1935.

GAMBI, A.; TOFFETTI, G.; PEZZÈ, M. Assurance of self-adaptive controllers for the cloud. In: CÁMARA, J. *et al.* (Ed.). **Assurances for Self-Adaptive Systems**. Berlin, Heidelberg: Springer, 2013, (Lecture Notes in Computer Science, v. 7740). p. 311–339. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-642-36249-1_12>.

GAO, Y. *et al.* Model checking guided incremental testing for distributed systems. **Proceedings of the ACM on Software Engineering**, v. 2, n. ISSTA, p. 297–319, 2025. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/3728883>.

GEORGES, A.; BUYTAERT, D.; EECKHOUT, L. Statistically rigorous Java performance evaluation. In: **Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)**. Montreal, Canada: ACM, 2007. p. 57–76. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1145/1297027.1297033>.

GILBERT, S.; LYNCH, N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. **ACM SIGACT News**, v. 33, n. 2, p. 51–59, 2002. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1145/564585.564601>.

GOOD, P. I. **Permutation, Parametric and Bootstrap Tests of Hypotheses**. 3. ed. New York: Springer, 2005. (Springer Series in Statistics).

Google. **Service Level Objectives (SRE Book)**. 2016. Website. Acesso em: 19 mar. 2026. Available from Internet: <https://sre.google/sre-book/service-level-objectives/>.

HANLEY, J. A.; LIPPMAN-HAND, A. If nothing goes wrong, is everything all right? Interpreting zero numerators. **JAMA**, v. 249, n. 13, p. 1743–1745, 1983. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1001/jama.1983.03330370053031>.

HARRIS, C. R. *et al.* Array programming with NumPy. **Nature**, v. 585, n. 7825, p. 357–362, 2020. Acesso em: 30 jun. 2026. Available from Internet: <https://doi.org/10.1038/s41586-020-2649-2>.

HEO, T. **Control Group v2**. 2026. The Linux Kernel documentation, Admin Guide. Acesso em: 30 out. 2026. Available from Internet: <https://docs.kernel.org/admin-guide/cgroup-v2.html>.

HERLIHY, M. P.; WING, J. M. Linearizability: A correctness condition for concurrent objects. **ACM Transactions on Programming Languages and Systems**, v. 12, n. 3, p. 463–492, 1990. Acesso em: 17 maio 2026. Available from Internet: <https://doi.org/10.1145/78969.78972>.

HOEFLER, T.; BELLI, R. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In: **Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)**. Austin, TX, USA: ACM, 2015. p. 1–12. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1145/2807591.2807644>.

HOLM, S. A simple sequentially rejective multiple test procedure. **Scandinavian Journal of Statistics**, v. 6, n. 2, p. 65–70, 1979. Acesso em: 30 out. 2026. Available from Internet: <https://www.jstor.org/stable/4615733>.

HOLZMANN, G. J. **The SPIN Model Checker: Primer and Reference Manual**. Boston, MA: Addison-Wesley, 2004. ISBN 978-0-321-22862-8.

Infinispan Project. **Configuring Infinispan caches (Infinispan 15.0)**. 2026. Website. Acesso em: 27 out. 2026. Available from Internet: <https://infinispan.org/docs/15.0.x/titles/configuring/configuring.html>.

Infinispan Project. **Cross-site replication (xsite)**. 2026. Website. Acesso em: 19 mar. 2026. Available from Internet: <https://infinispan.org/docs/stable/titles/xsite/xsite.html>.

Infinispan Project. **Infinispan Documentation (stable)**. 2026. Website. Acesso em: 19 mar. 2026. Available from Internet: <https://infinispan.org/docs/stable/>.

Infinispan Project. **Infinispan Metrics**. 2026. Website. Acesso em: 19 mar. 2026. Available from Internet: <https://infinispan.org/docs/stable/titles/metrics/metrics.html>.

Infinispan Project. **Infinispan Performance Considerations and Tuning Guidelines**. 2026. Website. Acesso em: 25 mai. 2026. Available from Internet: <https://infinispan.org/docs/stable/titles/tuning/tuning.html>.

Infinispan Project. **Introduction: What is Infinispan?** 2026. Website. Acesso em: 19 mar. 2026. Available from Internet: <https://infinispan.org/introduction/>.

Infinispan Project. **Using Hot Rod Java clients (Infinispan 15.0), Section 1.4: Client Failover**. 2026. Website. Acesso em: 27 out. 2026. Available from Internet: <https://infinispan.org/docs/15.0.x/titles/hotrod_java/hotrod_java.html#hotrod_failover>.

JACKSON, D. **Software Abstractions: Logic, Language, and Analysis**. 2. ed. Cambridge, MA: MIT Press, 2012. ISBN 978-0-262-01715-2.

JAIMAN, V.; BEN MOKHTAR, S.; RIVIÈRE, E. TailX: Scheduling heterogeneous multiget queries to improve tail latencies in key-value stores. In: REMKE, A.; SCHIAVONI, V. (Ed.). **Distributed Applications and Interoperable Systems (DAIS 2020)**. Cham: Springer, 2020. (Lecture Notes in Computer Science, v. 12135), p. 73–92. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-030-50323-9_5>.

JUnit Team. **JUnit 5 User Guide**. 2026. Website. Acesso em: 30 jun. 2026. Available from Internet: <https://junit.org/junit5/>.

KANSAL, A. *et al.* Virtual machine power metering and provisioning. In: **Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC '10)**. Indianapolis, IN, USA: ACM, 2010. p. 39–50. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1145/1807128.1807136>.

KEPHART, J. O.; CHESS, D. M. The vision of autonomic computing. **Computer**, v. 36, n. 1, p. 41–50, 2003. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1109/MC.2003.1160055>.

LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. **Communications of the ACM**, v. 21, n. 7, p. 558–565, 1978. Acesso em: 14 maio 2026. Available from Internet: <https://doi.org/10.1145/359545.359563>.

LAMPORT, L. **Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers**. Boston, MA: Addison-Wesley, 2002. Acesso em: 18 jun. 2026. ISBN 978-0-321-14306-8. Available from Internet: <https://lamport.azurewebsites.net/tla/book.html>.

LEE, T. **TLA+ / TLC TL;DR**. 2021. Blog. Publicado em 24 abr. 2021. Acesso em: 16 jul. 2026. Available from Internet: <https://tomlee.co/2021/04/tla-tlc-tldr/>.

LEMOS, D. L. *et al.* Instrumentação das técnicas Árvore de decisão e kerastuner para autoajuste de redes neurais recorrentes gru na perspectiva de computação sustentável. In: **Anais da Escola Regional de Alto Desempenho da Região Sul (ERAD-RS)**. [s.n.], 2025. Acesso em: 19 mar. 2026. Available from Internet: <https://sol.sbc.org.br/index.php/eradrs/article/view/35364>.

LI, B. *et al.* Enjoy your observability: an industrial survey of microservice tracing and analysis. **Empirical Software Engineering**, v. 27, n. 1, p. art. 25, 2021. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/s10664-021-10063-9>.

LIU, S. *et al.* Quantitative analysis of consistency in NoSQL key-value stores. In: CAMPOS, J.; HAVERKORT, B. R. (Ed.). **Quantitative Evaluation of Systems (QEST 2015)**. Cham: Springer, 2015. (Lecture Notes in Computer Science, v. 9259), p. 228–243. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-319-22264-6_15>.

LIU, Y. *et al.* OnlineElastMan: self-trained proactive elasticity manager for cloud-based storage services. **Cluster Computing**, v. 20, n. 3, p. 1977–1994, 2017. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/s10586-017-0899-z>.

LUDOVICI, F.; PFEIFER, H. P. **tc-netem(8): Network Emulator**. 2011. Linux manual page (iproute2). Acesso em: 30 out. 2026. Available from Internet: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>.

MANN, H. B.; WHITNEY, D. R. On a test of whether one of two random variables is stochastically larger than the other. **The Annals of Mathematical Statistics**, v. 18, n. 1, p. 50–60, 1947. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1214/aoms/1177730491>.

MORÁN, M. *et al.* Exploring energy saving opportunities in fault tolerant HPC systems. **Journal of Parallel and Distributed Computing**, v. 185, p. 104797, 2024. Acesso em: 19 mar. 2026. Available from Internet: <https://doi.org/10.1016/j.jpdc.2023.104797>.

NAIR, S. S.; PETRI, G.; SHAPIRO, M. Proving the safety of highly-available distributed objects. In: MÜLLER, P. (Ed.). **Programming Languages and Systems (ESOP 2020)**. Cham: Springer, 2020, (Lecture Notes in Computer Science, v. 12075). p. 544–571. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-030-44914-8_20>.

NASKOS, A.; GOUNARIS, A.; KATSAROS, P. Cost-aware horizontal scaling of NoSQL databases using probabilistic model checking. **Cluster Computing**, v. 20, n. 3, p. 2687–2701, 2017. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/s10586-017-0816-5>.

NAVAUX, P. O. A.; LORENZON, A. F.; SERPA, M. d. S. Challenges in high-performance computing. **Journal of the Brazilian Computer Society**, v. 29, n. 1, p. 51–62, 2023. Acesso em: 23 jun. 2026. Available from Internet: <https://journals-sol.sbc.org.br/index.php/jbcs/article/view/2219>.

NEIVA, F. W.; SILVA, R. L. d. S. d. **Revisão Sistemática da Literatura em Ciência da Computação: Um Guia Prático**. Juiz de Fora: Departamento de Ciência da Computação, UFJF, 2016. (RelaTeDCC, 001/2016). Acesso em: 12 maio 2026. Available from Internet: <https://www.researchgate.net/publication/304215400>.

NEWCOMBE, C. Why Amazon chose TLA+. In: AIT AMEUR, Y.; SCHEWE, K.-D. (Ed.). **Abstract State Machines, Alloy, B, TLA, VDM, and Z (ABZ 2014)**. Berlin, Heidelberg: Springer, 2014. (Lecture Notes in Computer Science, v. 8477), p. 25–39. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-662-43652-3_3>.

NEWCOMBE, C. *et al.* How Amazon Web Services uses formal methods. **Communications of the ACM**, v. 58, n. 4, p. 66–73, 2015. Acesso em: 18 jun. 2026. Available from Internet: <https://doi.org/10.1145/2699417>.

NGUYEN, D. *et al.* Using weaker consistency models with monitoring and recovery for improving performance of key-value stores. **Journal of the Brazilian Computer Society**, v. 25, n. 1, p. art. 10, 2019. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1186/s13173-019-0091-9>.

NOSEK, B. A. *et al.* The preregistration revolution. **Proceedings of the National Academy of Sciences**, v. 115, n. 11, p. 2600–2606, 2018. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1073/pnas.1708274114>.

ONGARO, D.; OUSTERHOUT, J. In search of an understandable consensus algorithm. In: **Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)**. Philadelphia, PA: USENIX Association, 2014. p. 305–319. ISBN 978-1-931971-10-2. Acesso em: 17 maio 2026. Available from Internet: <https://raft.github.io/raft.pdf>.

OpenMetrics Project. **OpenMetrics: A Cloud-Native, Highly Scalable Metrics Protocol**. 2024. Cloud Native Computing Foundation (CNCF). Especificação em https://github.com/prometheus/OpenMetrics. Acesso em: 30 jun. 2026. Available from Internet: <https://openmetrics.io/>.

Oracle. **Java Platform, Standard Edition: Java Management Extensions (JMX) Guide**. 2023. Oracle Java SE Documentation. Acesso em: 30 jun. 2026. Available from Internet: <https://docs.oracle.com/en/java/javase/21/jmx/introduction-jmx-technology.html>.

OWASP Foundation. **Session Management Cheat Sheet**. 2026. OWASP Cheat Sheet Series. Acesso em: 12 maio 2026. Available from Internet: <https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html>.

PAGE, M. J. *et al.* The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. **Systematic Reviews**, v. 10, n. 1, p. art. 89, 2021. Acesso em: 19 mar. 2026. Available from Internet: <https://systematicreviewsjournal.biomedcentral.com/articles/10.1186/s13643-021-01626-4>.

PETERSEN, K. *et al.* Systematic mapping studies in software engineering. In: **Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering (EASE 2008)**. BCS Learning & Development, 2008. (Electronic Workshops in Computing). Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.14236/ewic/EASE2008.8>.

Podman Project. **Podman Documentation**. 2026. Website. Sítio oficial: https://podman.io. Acesso em: 30 jun. 2026. Available from Internet: <https://docs.podman.io/>.

Prometheus Authors. **Prometheus Documentation**. 2026. Website. Acesso em: 30 jun. 2026. Available from Internet: <https://prometheus.io/docs/>.

Red Hat. **Red Hat JBoss Data Grid 7.0: Administration and Configuration Guide**. 2016. Website. Acesso em: 16 jul. 2026. Available from Internet: <https://docs.redhat.com/en/documentation/red_hat_data_grid/7.0/html-single/administration_and_configuration_guide/index>.

ROSA, L.; RODRIGUES, L.; LOPES, A. Self-management of distributed systems using high-level goal policies. In: LEMOS, R. *et al.* (Ed.). **Software Engineering for Self-Adaptive Systems II**. Berlin, Heidelberg: Springer, 2013, (Lecture Notes in Computer Science, v. 7475). p. 162–190. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-642-35813-5_7>.

RUIZ-FUERTES, M. I.; PALLARDÓ-LOZOYA, M. R.; MUÑOZ-ESCOÍ, F. D. Consistency in scalable systems. In: **On the Move to Meaningful Internet Systems: OTM 2012**. Berlin, Heidelberg: Springer, 2012, (Lecture Notes in Computer Science, v. 7566). p. 549–565. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-642-33615-7_7>.

SALHI, H. *et al.* Benchmarking and performance analysis for distributed cache systems: A comparative case study. In: **Performance Evaluation and Benchmarking for the Analytics Era (TPCTC 2017)**. [s.n.], 2018. (Lecture Notes in Computer Science). Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/978-3-319-72401-0_11>.

SHAPIRO, M. *et al.* Conflict-free replicated data types. In: DÉFAGO, X.; PETIT, F.; VILLAIN, V. (Ed.). **Stabilization, Safety, and Security of Distributed Systems (SSS 2011)**. Berlin, Heidelberg: Springer, 2011. (Lecture Notes in Computer Science, v. 6976), p. 386–400. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1007/978-3-642-24550-3_29>.

STEELE JR., G. L.; LEA, D.; FLOOD, C. H. Fast splittable pseudorandom number generators. In: **Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14)**. Portland, OR, USA: ACM, 2014. p. 453–472. Acesso em: 27 out. 2026. Available from Internet: <https://doi.org/10.1145/2660193.2660195>.

STUDENT. The probable error of a mean. **Biometrika**, v. 6, n. 1, p. 1–25, 1908.

TEMOSHOK, D. *et al.* **Digital Identity Guidelines: Authentication and Authenticator Management**. Gaithersburg, MD: National Institute of Standards and Technology, 2025. (NIST Special Publication, 800-63B-4). Substitui a SP 800-63B (2017). Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.6028/NIST.SP.800-63b-4>.

TENE, G. **HdrHistogram: A High Dynamic Range (HDR) Histogram**. 2026. Website e repositório de software. Repositório: https://github.com/HdrHistogram/HdrHistogram. Acesso em: 30 jun. 2026. Available from Internet: <https://hdrhistogram.github.io/HdrHistogram/>.

TIAN, Y. *et al.* Cache your prompt when it's green: Carbon-aware caching for large language model serving. **Proceedings of the ACM on Measurement and Analysis of Computing Systems**, v. 10, n. 1, p. 1–28, 2026. Apresentado na ACM SIGMETRICS 2026. Acesso em: 23 jun. 2026. Available from Internet: <https://doi.org/10.1145/3788087>.

ULLAH, A. *et al.* A control theoretical view of cloud elasticity: taxonomy, survey and challenges. **Cluster Computing**, v. 21, n. 4, p. 1735–1764, 2018. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/s10586-018-2807-6>.

VIRTANEN, P. *et al.* SciPy 1.0: fundamental algorithms for scientific computing in Python. **Nature Methods**, v. 17, n. 3, p. 261–272, 2020. Acesso em: 30 jun. 2026. Available from Internet: <https://doi.org/10.1038/s41592-019-0686-2>.

WU, C.; SREEKANTI, V.; HELLERSTEIN, J. M. Autoscaling tiered cloud storage in Anna. **The VLDB Journal**, v. 30, n. 1, p. 25–43, 2020. Acesso em: 12 maio 2026. Available from Internet: <https://doi.org/10.1007/s00778-020-00632-7>.

YU, Y.; MANOLIOS, P.; LAMPORT, L. Model checking TLA+ specifications. In: PIERRE, L.; KROPF, T. (Ed.). **Correct Hardware Design and Verification Methods (CHARME '99)**. Berlin, Heidelberg: Springer, 1999. (Lecture Notes in Computer Science, v. 1703), p. 54–66. Acesso em: 30 out. 2026. Available from Internet: <https://doi.org/10.1007/3-540-48153-2_6>.

YUAN, D. *et al.* Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems. In: **Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)**. Broomfield, CO, USA: USENIX Association, 2014. p. 249–265. Acesso em: 30 out. 2026. Available from Internet: <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan>.

APÊNDICE A — PLANO DE TRABALHO PARA O TCC II

Este apêndice detalha o plano de trabalho previsto para o Trabalho de Conclusão de Curso II. O cronograma quinzenal aparece na Tabela 6 da Seção 5.1, e cada uma das seis etapas é explicitada a seguir, com entregáveis, dependências e riscos. O plano foi revisado durante a execução, e as atividades que restam para concluir o trabalho constam, ao final deste apêndice, nas Etapas A a F.

Etapas 1 e 2: consolidação do modelo formal

As duas primeiras semanas reúnem três atividades. A primeira consolida o modelo formal do sistema de sessões com canal assíncrono, partindo da especificação inicial descrita na Seção 4.2 e adicionando os refinamentos identificados nas execuções do final do Trabalho de Conclusão de Curso I. A segunda formaliza o conjunto inicial de invariantes I1 a I6, com tradução das propriedades de convergência (I5 e I6) para um predicado de quiescência reconhecível pelo verificador. A terceira executa a primeira rodada completa de *model checking* sobre o modelo consolidado, com configurações de cardinalidade crescente, e registra todos os contraexemplos gerados.

Etapas 3 e 4: introdução de falhas explícitas no modelo

A evolução central das semanas 3 e 4 é a incorporação de falhas explícitas ao modelo formal. As transições adicionadas representam, respectivamente, *crash* silencioso de um nó, recuperação após *crash* e partição de rede entre subconjuntos de nós. Cada falha admite parametrização sobre o número de nós afetados e a duração relativa. O conjunto de invariantes pode ser refinado, com decisão registrada caso o autor opte por adicionar invariantes derivados (por exemplo, unicidade de sessão ativa por identidade, atualmente fora do escopo). Os contraexemplos gerados são analisados e delimitam quais padrões de falha devem ser reproduzidos experimentalmente.

Etapas 5 e 6: *baseline* experimental sob falhas

Nas semanas 5 e 6, o *baseline* experimental é executado sob falhas selecionadas. O conjunto cobre os quatro cenários (F1 a F4) já planejados para a linha experimental e, quando aplicável, o cenário F5 de *cross-site*. A replicação entre localidades (*cross-site replication*) saiu do recorte durante o Trabalho de Conclusão de Curso II, e o cenário F5 não será executado. Cada cenário é executado o número de vezes definido pelo plano de análise estatística, com cálculo de p95, p99, taxa de erro, tempo de recuperação e taxa de violação de cada invariante. O conjunto de referência produzido alimenta a comparação subsequente com o controlador.

Etapas 7 e 8: alinhamento formal-experimental e controlador heurístico

A primeira atividade das semanas 7 e 8 alinha o modelo formal e o experimento. Os contraexemplos identificados nas etapas anteriores são mapeados para padrões observáveis no *baseline*, com ajustes mínimos no programa de carga e na instrumentação para garantir comparabilidade. A segunda atividade implementa o controlador heurístico. O controlador é externo ao *data grid*, lê observáveis em janelas temporais e decide ações conforme regras determinísticas com histerese e intervalo mínimo entre acionamentos. Dois perfis pré-definidos coexistem: um perfil normal, otimizado para latência, e um perfil conservador, ativado quando a taxa de violação aumenta acima de um limiar. O projeto do controlador foi revisado no Trabalho de Conclusão de Curso II: o gatilho primário combina latência de cauda relativa, taxa de erro e topologia, e as violações de I1, I2 e I4 formam o gatilho secundário (Seção 4.4).

Etapas 9 e 10: execução com controlador e análise comparativa

As semanas 9 e 10 executam os mesmos cenários do *baseline* com o controlador ativo. Métricas equivalentes são coletadas e as decisões do controlador são registradas com o respectivo gatilho. A análise comparativa estabelece, por cenário, a diferença entre o *baseline* e a configuração com controlador para cada métrica, com testes estatísticos não paramétricos e intervalos de confiança. A correlação com os contraexemplos formais aproxima os achados experimentais dos padrões previstos pelo modelo.

Etapas 11 e 12: consolidação e entrega

As duas últimas semanas concluem a avaliação crítica, sistematizam as limitações remanescentes e finalizam a escrita do trabalho. A descrição do método, os resultados, a discussão e a preparação para entrega e defesa são integradas ao texto. A escrita ocorre em paralelo a todas as etapas, e a consolidação final consiste em integrar as parciais já escritas, revisar a coerência entre as vertentes formal e experimental, e finalizar elementos pré-textuais e pós-textuais.

Plano de trabalho restante

O plano das Etapas 1 a 12 foi modificado durante a execução do Trabalho de Conclusão de Curso II. Esta parte do apêndice detalha as atividades que restam para concluir o trabalho, na ordem imposta pelas dependências entre elas. Não há datas fixas: a duração de várias etapas depende de tempo de máquina, e a estimativa para as três campanhas de medição (nova medição de referência, campanha com o cenário S3 e avaliação do controlador) é de ao menos cinco a sete noites de execução sem falha do hospedeiro, além de sessões diurnas de preparação. As etapas que exigem a máquina virtual são executadas uma de cada vez, com validação antes da seguinte. As demais podem correr em paralelo.

Etapa A: preparação da máquina virtual

A etapa conclui a sequência de verificações dos injetores descrita na Seção 4.3: a primeira injeção real de degradação parcial, com perda de 20 % por 60 s sem carga de aplicação, e as duas verificações restantes do injetor de partição, a recomposição manual depois de uma partição real e o disparo do temporizador de segurança armado pelo *script* de verificação. Em seguida, o sistema operacional da máquina virtual é protegido contra atualizações automáticas de *kernel* e reinícios não planejados, porque uma troca de *kernel* no meio de uma campanha alteraria o ambiente medido. A etapa termina com o ensaio do F1 silencioso na máquina virtual e com a recriação do *cluster* com os quatro *caches*, dois síncronos e dois assíncronos.

Etapa B: nova medição de referência

A etapa executa o protocolo pré-registrado da Seção 3.2. Uma rodada piloto com as doze células, de dia, verifica o orquestrador, os critérios de recusa e os tempos; seus dados não entram em resultado. A campanha de 120 execuções roda em duas noites, com autorização do autor antes de cada uma. A análise pré-registrada produz os percentis por execução, os contrastes entre condições de falha e entre os modos síncrono e assíncrono e as contagens de violação, que formarão as primeiras tabelas de resultados válidas do trabalho. Os números da bateria de junho não serão comparados com os novos.

Etapa C: falhas sob carga e cenário S3

O orquestrador de experimentos passa a disparar F2 e F4, e o comportamento do modo assíncrono é verificado na documentação e empiricamente, com escrita em um nó e leitura imediata em outro, nos dois modos. Com isso, prepara-se o pré-registro da campanha com o cenário S3, em que `DIST_ASYNC` é o alvo, com todas as condições de falha, e `DIST_SYNC` entra em células de referência. A estimativa atual é de cerca de 120 execuções. Essa campanha mede as hipóteses sobre o modo assíncrono, como violações de I1 e I4 mesmo sem falha e perda de escritas confirmadas sob partição, e fornece os dados de calibração do controlador.

Etapa D: controlador heurístico

O desenvolvimento do controlador não depende da máquina virtual até a avaliação. Falta ligá-lo ao laço de carga, com as opções de linha de comando e a gravação das trilhas de decisão; implementar a reprodução das decisões sobre registros gravados, que confere a equivalência entre a decisão ao vivo e a reprodução; acrescentar marcadores de início e fim de F1 e F3 aos registros; e proteger a análise de energia contra parâmetros inválidos. Com os dados da etapa C, os limiares são calibrados pela regra fixada antes, e a avaliação é pré-registrada, testada em execução curta e executada conforme a Seção 3.2, com cerca de 80 execuções.

A segunda etapa do controlador trata da atuação sobre a configuração do *cluster* (Seção 4.4). A verificação de quais atributos do Infinispan 15 podem ser alterados em

tempo de execução sem reiniciar o *cache* parte da lista de candidatos, que é hipótese de partida, e é feita na documentação e em *cluster* local. Ela pode começar depois da ligação do controlador ao laço de carga, enquanto a implementação, o pré-registro e a execução vêm depois da avaliação da etapa cliente. Antes da implementação, definem-se a lista final de parâmetros, a admissão ou não de ações que alteram a convergência ou a semântica sob partição, estas com extensão do modelo TLA+, e o tempo de máquina, e a etapa recebe pré-registro e avaliação próprios.

Etapa E: vertente formal e alinhamento

A correção do empate de marcas de tempo, por desempate entre réplicas ou por canal com entrega em ordem, depende de decisão do autor e é seguida de nova verificação de I5 e I6. O mecanismo da fusão de partições não depende dessa correção, e as violações correspondentes devem continuar. As variantes com perda de mensagens e com relógio atualizado pelo máximo são integradas ou descartadas com registro do motivo. O catálogo que relaciona operação, falha e invariante é então confrontado com os resultados experimentais das etapas B e C, para indicar quais contraexemplos do modelo têm correspondente observado e quais não têm.

Etapa F: mapeamento sistemático e consolidação

O mapeamento sistemático é fechado com a complementação da busca nas bases originais, a classificação dos 165 estudos selecionados nas dimensões D1 a D8 e o fluxograma PRISMA definitivo, o que permite posicionar o trabalho frente a cada agrupamento de estudos. O código desenvolvido, hoje em versões locais, é integrado à versão principal do repositório. A escrita final integra os resultados das campanhas, a discussão e as conclusões, revisa a coerência entre as vertentes formal e experimental e conclui os elementos pré-textuais e pós-textuais. A escrita ocorre em paralelo a todas as etapas.